
AXION

Development of a digital productivity tool with smart attention management features, designed to help users manage their time effectively by restricting access to distracting applications and websites.

Master Thesis
The Technical University of Denmark
Computer Science & Engineering

Total characters: 199.680
Normal pages (2400 c/p): 55.2 (excl. appendix)
Total physical pages: 119
Appendix pages: 46
Hand-in date: 27/07/2025
ECTS points: 35
Supervisor: Thomas James Howard



Christoffer Hartig Braunstein
s194475



Frederik Kjærgaard
s194610

Contents

Prologue **5**
1.1 Abstract . . . 6
1.2 Acknowledgements . . . 6
1.3 Introduction . . . 7

Methodology **8**

The Problem **10**
3.1 Introduction . . . 11
3.2 Addictive Products as a Business Model . . . 11
3.3 How Technology Companies Engineer Addiction . . . 11
3.4 The Prevalence and Consequences of the Problem . . . 12
3.5 A Critique of the Media Discourse: Why a Lack of Awareness is Not The Problem 13
3.6 Problem Formulation . . . 13

The Solution **14**
4.1 Introduction . . . 15
4.2 Conceptual Idea . . . 15
4.3 Blue Ocean Strategy - Theory . . . 15
4.4 Blue Ocean Strategy - Applied . . . 16
4.5 Final Value-Offer . . . 18
4.6 Target Audience . . . 18

Requirements **19**
5.1 Minimum Viable Product . . . 20
5.2 System Requirements . . . 20
5.3 Non-Functional Requirements . . . 21

Project Management & Execution **22**
6.1 Introduction . . . 23
6.2 Incorporation of AxionHQ ApS . . . 23
6.3 Monetization & Business Model . . . 23
6.4 Fundraising . . . 23
6.5 Financial Projection . . . 24
6.6 Board of Advisors . . . 26
6.7 Key Performance Indicators . . . 26
6.8 Legal Requirements . . . 27
6.9 First Marketing Campaign & Feedback . . . 27
6.10 Agile Development . . . 28
6.11 Development Guidelines & Principles . . . 29

Product & Development **30**
7.1 Introduction . . . 31
7.2 Technical Stack Selection . . . 31
7.2.1 Application Layer . . . 32
7.2.2 Backend Layer . . . 32
7.2.3 Database Layer . . . 33
7.2.4 Error Monitoring . . . 33
7.2.5 Analytics . . . 33
7.2.6 Automation & Version Control . . . 33

7.2.7 System Architecture Overview . . . 34
7.3 System Architecture - Android Application . . . 35
7.3.1 MVVM and Jetpack Compose . . . 35
7.3.2 UI Design . . . 35
7.3.3 Introduction & Tutorial . . . 37
7.3.4 Accessibility Service & Blocking Functionality . . . 37
7.3.5 Advanced Blocking . . . 39
7.3.6 Category Blocking . . . 40
7.3.7 Data Reporting . . . 42
7.3.8 Global Exception Handler . . . 44
7.3.9 Caching . . . 45
7.3.10 Deployment of Application . . . 45
7.3.11 Development & Production Environment . . . 46
7.4 System Architecture - Backend . . . 47
7.4.1 Monolithic Structure versus Micro-services . . . 47
7.4.2 REST API . . . 48
7.4.3 Account Management . . . 50
7.4.4 Development & Production Environment . . . 51
7.4.5 Unit Testing . . . 51
7.5 System Architecture - Azure . . . 51
7.5.1 Standard Directory . . . 53
7.5.2 Azure B2C Directory . . . 54
7.6 System Architecture - Database . . . 55
7.6.1 Conceptual Design . . . 55
7.6.2 Normalization . . . 56
7.6.3 Implementation . . . 57
7.7 Website . . . 57
7.8 Sentry . . . 58
7.9 Firebase Analytics & BigQuery . . . 58
7.10 Version Control & CI/CD . . . 59
7.11 Challenges and Unimplemented Features . . . 60
7.11.1 Internet Slowdown . . . 60
7.11.2 Category Blocking With DNS . . . 61

Results & Future Vision **62**
8.1 Results . . . 63
8.1.1 End User Perspective . . . 63
8.2 Future Vision . . . 64

Epilogue **65**
9.1 Discussion . . . 66
9.1.1 Problem Exploration . . . 66
9.1.2 Solution Innovation . . . 66
9.1.3 Development . . . 66
9.1.4 User Acquisition and Marketing . . . 67
9.1.5 Future Perspectives . . . 67
9.2 Conclusion . . . 68

Appendix **72**
10.1 Endpoints . . . 73
10.1.1 POST /api/accounts . . . 73
10.1.2 GET /api/accounts/{externalId} . . . 73

10.1.3 PUT /api/accounts/{externalId} . . . 74
10.1.4 DELETE /api/accounts/{externalId}/delete/request . . . 74
10.1.5 POST /api/accounts/{externalId}/delete/verify . . . 74
10.1.6 POST /api/accounts/{externalId}/email/request . . . 75
10.1.7 POST /api/accounts/{externalId}/email/verify . . . 75
10.1.8 POST /api/feedback . . . 76
10.1.9 GET /api/health . . . 76
10.1.10 POST /api/accounts/{externalId}/blocklists . . . 77
10.1.11 GET /api/accounts/{externalId}/blocklists . . . 78
10.1.12 DELETE /api/accounts/{externalId}/blocklists/{blockListId} . . . 78
10.2 Firebase Events . . . 79
10.3 Version Display . . . 82
10.3.1 Version 1.0.0 . . . 82
10.3.2 Version 1.1.0 . . . 82
10.3.3 Version 1.1.1 . . . 82
10.3.4 Version 1.2.0 . . . 83
10.4 Initial Project Plan . . . 83
10.4.1 Pre-Release Phase . . . 84
10.4.2 Post-Release Phase . . . 84
10.4.3 Unknown Phase . . . 85
10.5 BigQuery User Engagement Measurements . . . 86
10.5.1 Active Users . . . 86
10.5.2 Long Term Users . . . 87
10.5.3 Churn Rate . . . 88
10.5.4 User Growth Daily . . . 90
10.5.5 Long Term User Growth Daily . . . 91
10.6 DTU Flyer . . . 93
10.7 Application Tutorial Screens . . . 94
10.8 Board of Advisors . . . 98
10.8.1 Board Members . . . 98
10.8.2 Meeting Summaries . . . 99
10.9 MVP User Stories . . . 101
10.10 Sprint Planning . . . 102
10.11 Screenshots of the Application . . . 104
10.12 Class Diagram of Block . . . 115
10.13 Backend HTTP Statuses . . . 115
10.14 Accessibility Permission Alert . . . 116
10.15 Navigation Bar Comparison . . . 117
10.16 Supplementary Files . . . 118

PROLOGUE

"The best minds of my generation are thinking about how to make people click ads"
- Jeff Hammerbacher, Early Employee of Facebook

1.1 Abstract

In recent decades, compulsive use of screens have been on a rise. This project aims to examine the problem, innovate a solution, and make considerable progress towards its development. We explore the problem of compulsive and problematic technology use and its effects on individuals through existing literature and research on the topic. We map some of the current solutions to problematic technology use, in particular application-based solutions. Blue Ocean Strategy is used to analyze some of the companies in the current market and propose a novel alternative to existing solutions, which we call the product vision.

We describe how a company was founded with the goal of realizing the product vision. We specify the requirements needed to build a Minimum Viable Product, and formulate a detailed plan for the execution of the project using agile methodology. We describe the choices and considerations we have with regards to commercialization of the solution. Through a product oriented methodology, we go over the actions taken towards building the product vision, and explain the technical structure in detail. We document the architecture of the current implementation. The solution we made, which exceeds the requirements we originally defined for the Minimum Viable Product, is presented at its current stage. Then a discussion is provided. It details the choices made during the project and the future perspectives. Finally, we conclude on the project.

1.2 Acknowledgements

We would like to thank our supervisor from DTU, Professor **Thomas J. Howard**, for his expertise and guidance throughout the project. His knowledge of the area, combined with access to relevant resources, helped guide the project to completion.

A special thank you to our board of advisors — **Jan Borg**, **Morten Lahrmann** and **Tina Braunstein**. Their insights were crucial in shaping the project during its early stages through discussions and feedback. **Jan Borg**, with over 10 years of experience in IT and project management and educated as a cand.it, contributed valuable input on the development of the product. **Morten Lahrmann**, currently a manager at FujiFilm, brought experience from two previous startups and ongoing business development activities, offering guidance on business strategy. **Tina Braunstein**, Director at Greve Vand a private water utility with 11 employees, provided valuable strategic insight from an executive perspective.

We would also like to thank **David Ventzel** from Accelerace, a leading pre-seed investor and accelerator. As a partner and fund manager, he offered valuable feedback on our business and product ideas, sharing his experience with common pain points faced by early stage startups.

Thanks to **Magnus Nyborg Madsen**, co-founder of Reshape Biotech, which develops robotic platforms with incubation and automated imaging. Through several discussions and meetings, he shared experienced from his own startup journey and provided feedback on our business plan and product direction.

We appreciate the input from **Natasha Hougaard** and **Magnus Stjernborg Koch** from Skjoldet, a DTU-based startup developing a browser add-on for security against phishing attacks. Their insights into their own business model and development process further expanded our understanding and informed our approach.

We also thank DTU Associate Professor **Tyge Tiessen** for sparring on the security aspects and legal data compliance related to the project.

Additional thanks to **Jose Cayasso**, CEO of *SlideBean*, for providing valuable feedback on our original marketing plan.

Lastly, we would like to thank **Thomas Hartig Braunstein** and **Rikke Liljeroth Lindeberg Bille** for proofreading the report.

1.3 Introduction

This thesis is the result of a growing frustration with how difficult it can be to stay focused while working or studying. As students, we have experienced firsthand how much time is unintentionally spent on mobile devices, even when we know it is unproductive. The project began from that personal experience and the idea that we might be able to build something that could help not just ourselves, but others facing the same challenges.

The project takes place within the domain of problematic technology habits, with a focus on digital habit changes. It aims to explore whether software can support healthier usage patterns, without requiring users to rely purely on willpower. This exploration is mainly through the development of a proposed solution. The goal is to not just build something that technically works, but to explore what it takes to make such a system useful in practice.

The thesis follows a practical, product oriented methodology, with the main objective being to build a working solution and gather early usage insights. While the primary focus is on development, considerations about the business model supporting the product is also presented. The business sections follows a more theoretical approach, aiming to evaluate feasibility and potential.

Throughout the thesis, we will be referring to both sections in the appendix, external sources and supplementary files that has been uploaded alongside the thesis. We have collected and explained each of the supplementary files in the appendix 10.16 Supplementary Files.

The following chapter outlines the methodology, which defines the structure of the thesis and how each part is connected.

METHODOLOGY

"Move fast and break things"
- Mark Zuckerberg, Founder of Facebook

This project concerns problems arising from excessive technology use. The project aims to develop an adequate solution.

In Chapter 3, we examine the problem of excessive technology use on a societal and individual level. We argue for the existence, prevalence and severity of this problem, referencing existing academic knowledge and using deductive reasoning.

In Chapter 4, we consider possible solutions to the problem presented in Chapter 3. We argue that the current widespread solutions are insufficient by referring to existing literature on the subject. A software-based solution is proposed. Using Blue Ocean Strategy [1], we analyze the existing value offers in the related market. The *Remove-Reduce-Raise-Create* matrix from Blue Ocean Strategy is used to create a new, currently non-existing value offer as a solution to the problem from Chapter 3, serving as a long term product vision.

In Chapter 5, we scope the project using the product vision found in Chapter 4. We define the MVP. We provide specific system requirements and non-functional requirements.

In Chapter 6, we explain how the project was structured and executed according to the requirements defined in Chapter 5. This process was supported by a company which we founded to facilitate the completion of the product vision as defined in Chapter 4.

In Chapter 7, the product development process was documented according to computer science best practices and software industry standards. It was guided by the requirements defined in Chapter 5 and assisted by the process planning and workflow structure described in Chapter 6.

In Chapter 8, we show the state of the latest iteration of the product. A strategy for continuing the progression towards completion of the product vision is presented.

A discussion and conclusion is provided at the end of the report.

THE PROBLEM

"Privatize the Gains, Socialize the Costs"
- Unknown Origins

3.1 Introduction

Whether or not compulsory behaviors should be treated as addiction is disputed [2]. Some scientists prefer not using the word "addiction" about technology use as there exists some debate about the definition of addiction. Some authors argue that the term addiction "pathologizes everyday human behavior" [3]. One could argue against that view by stating that what is considered everyday human behavior changes over time, and that a particular behavior is not necessarily healthy by virtue of it being normal. Addiction is a powerful word, and technology habits that might not pass the threshold for addiction can still be problematic and deserving of a solution. "Problematic technology habits" will be used to describe any type of involuntary, irrational or compulsive use of technology in this report, to include nuances of damaging and irrational behavior happening below a pathologized threshold.

In this chapter, various arguments will be presented to support that problematic technology use is a serious public health problem. An argument for why addiction as a business model is prevalent in today's world will be presented. We will then introduce the concept of superstimuli as a possible way to explain problematic technology use. In section 3.4, data on how widespread the problem is and its consequences for individuals is covered.

Finally, in section 3.5, arguments will be presented for why the current popular discourse and view on these problems is insufficient to solve them. The usage of the product built later will depend on the public perspective, so the solution should take that into account.

3.2 Addictive Products as a Business Model

For a business, slowing the loss of customers is an effective strategy. If a product has addictive qualities, those alone may prompt customers to buy it repeatedly.

The potential success of addictive products solely by virtue of them being addictive, could provide a plausible explanation for why several significant industries uses them [4] [5].

Classic liberal economic theory suggests that competition benefits the customer in a multitude of ways [6]. The absolute nature of this claim has been disputed in recent years [7]. In the paper "Competition and Consumer Protection: A Behavioral Economics Account" [8], it is suggested that "the benefit (to consumers) of competition does not necessarily extend to a world with imperfectly rational consumers who systematically misperceive the benefits and costs of products". In the book "The Law and Economics of Irrational Behavior" [9] in chapter 5.2, it is discussed whether or not addiction is a rational choice, and it is concluded that addiction is at least partly irrational. If addiction is at least part irrational, and that the prevalence of imperfectly rational consumers in a market suggests that the competition in said market is not necessarily beneficial to consumers, it suggests that addictive business models presents a way for companies to operate successfully without necessarily being of benefit to customers. It seems plausible, that said companies may be competing on the addictiveness of their products, rather than qualities which would make rational customers purchase their product.

3.3 How Technology Companies Engineer Addiction

In a series of famous experiments, the Dutch ecologist, Niko Tinbergen in his book 'The Herring Gull's World: A Study of the Social Behavior of Birds' [10], found that across many species of animals, greatly exagerrated sources of natural stimuli was preferred. The most famous example is with the species herring gull, which focus on hatching the largest and greenest eggs in its nest, as these are perceived the most genetically promising. Tinbergen showed that artificially exaggerating the size and color to a degree never found in nature would make the birds leave

their nests full of real eggs to pursue nesting a large ball painted in a deep green color. As this phenomenon would never occur in the wild, the evolution of the herring gull did not have to select for any form of resistance against such clever manipulation.

In 'Vanedyr' by Nicklas Brendborg [11], Brendborg explains how companies use what he calls superstimuli to engineer addiction. He argues that technology companies do this by appealing to our needs for information, feeling of progress or sexual needs, depending on the industry in question. Sensationalist news can be said to be particularly stimulating [12]. It is hard for humans to resist temptations like these, as they are specifically engineered to exploit irrational biases and psychological weaknesses we naturally posses [11].

3.4 The Prevalence and Consequences of the Problem

According to an article from Simon Kemp from DataReportal [13], global adoption of the internet, social media, gaming and adult content has been on the rise for two decades. Average time of all internet users globally on social media (including those who do not use it at all) is by the end of 2024 2 hours and 21 minutes, with females spending significantly longer than males [13].

Smartphone usage is habit forming and people consistently underestimates their screen time [14]. Screen time may have different consequences dependent on the type. A study [15] on the relationship between self-reported screen time, actual screen time, self-reported stress, and actual stress, found several interesting insights, the most relevant summarized below:

- Screentime for entertainment was statistically significantly correlated with stress.
- A possible link between self-reported addiction and actual stress was found.
- Individuals using screens for entertainment were higher in perceptual stress, while individuals using screens mostly for work had lower perceptual stress and higher life-satisfaction.
- Self-reported addiction was correlated with higher screen time scores, suggesting that individuals' assessment of their screen time was quite accurate.
- Dependence on social media and gaming was in particular correlated with self-reported screen addiction.

While this was a study of correlations and therefore not conclusive to show that screen time for entertainment is the direct cause of higher levels of stress, it does highlight interesting nuances of screen usage. There is an opportunity cost associated with spending many hours on a screen each day for entertainment - the time must come from other activities, that one would have done instead. This opportunity cost could explain why social media use causes a reduction of in-person activities, as well as subjective well being [16].

Screen entertainment has potentially negative effects on the brain. Research finds that smartphone use in children causes a higher risk of developing Attention-Deficit Hyperactive Disorder [17] [18]. Results from an experiment done in the study [19] indicates that nomophobia, a specific way to measure smartphone addiction, has a persistent negative correlation with fluid intelligence whether or not the phone is present. This suggests that it is not merely phone presence, but long-term effects on the brain, which causes the lowered fluid intelligence [19].

With the modern knowledge that the brain is plastic, the idea that consuming several hours of fast-paced and highly stimulating content daily could reduce the ability to stay focused seems plausible.

3.5 A Critique of the Media Discourse: Why a Lack of Awareness is Not The Problem

Most of the media discourse around problematic technology use has centered around creating awareness of the problem, without offering concrete solutions.

In a survey conducted among 1251 Danish university students [20], 23% of student were at high risk of smartphone addiction. This was determined using the Smartphone Addiction Scale - Short Version (SAS-SV). Of the students with high risk, 74% identified their own behavior as problematic, and 91% had considered reducing their smartphone use. The author of the paper concludes that the findings suggests a desire among the students to reduce their smartphone usage. Smartphone users may be aware of habit formation but consume as if they are inattentive to it [14]. These findings suggest a lack of solutions rather than a lack of awareness.

3.6 Problem Formulation

There is significant evidence to suggest that addiction to entertainment technology and problematic use of technology carries a risk. It is a worthwhile goal to give individuals the option of better understanding and controlling their use of technology.

In this project, we will begin the construction of a solution to be used as a tool by individuals for improving their well-being and gain more time by understanding and controlling their digital usage.

THE SOLUTION

"The only way to beat the competition is to stop trying to beat the competition"
- Blue Ocean Strategy, W. Chan Kim

4.1 Introduction

This chapter will explore possible solutions to the problem of problematic technology use.

A straightforward way to solve the problem is for the user to simply use the phone the exact amount the user thinks it should be used. This is unlikely to work, as most users persistently fails to correctly estimate their actual screen time [14]. Even if successful, such a solution would still require the individual to continuously make decisions every time they had an urge to view their phone. Modern research in habit-formation suggests that environmental changes are successful strategies for habit change [21].

4.2 Conceptual Idea

Our idea, at a conceptual level, is to build an application and backend software, which empowers the user to have full information and control over their digital habits. It must be user friendly and understandable for a wide audience.

We believe that none of the current solutions achieve this in completion. If eliminating such a common and compromising problem as unhealthy technology use would be easy and simple, we think more people would use that solution.

4.3 Blue Ocean Strategy - Theory

For the development of our idea, we have used the methods in the book "Blue Ocean Strategy" [1] aiming to make direct and expensive competition more or less irrelevant by targeting new market segments with a new product.

The core idea of the book is to use a tool called the "Strategy Canvas" to map out the value-offer of either an industry or a single business. A value-offer, as denoted by the author, is the collection of important aspects of a given product or service. Frequently, the authors argue, businesses in an industry end up benchmarking themselves against one another, using the same Key Performance Indicators. This leads to harsh and expensive competition and falling profits as competition for the same customers intensifies.

The goal is to diversify from existing companies to access a different market without competition. The process of doing that requires four modifications to the existing value-offer:

- Remove
- Reduce
- Raise
- Create

By removing aspects of a value-offer which a subset of customers does not appreciate or directly dislike, the product is simplified and its cost is brought down. By reducing aspects that the industry consider benchmarks, but customers are not especially attached to, further simplicity and cost reduction can be achieved. By raising existing aspects which customers appreciate, the product is improved. Finally, after creating entirely new aspects, a new customer base can be attracted, and a new value-offer is created.

Several strategies for choosing the right aspects of a value-offer to scale up, down, remove or add, is proposed. One way is by combining the offer of two or more existing companies or industries. Another is by looking at why different groups of non-customers have chosen not to be customers. The latter is particularly useful as industries frequently attempt to compete on certain metrics which the customers are then "trained" to care about - thus, non-customers are frequently more precise at suggesting improvements to products than existing customers.

In the next section, we will utilize Blue Ocean Strategy to analyze the largest competitors and the rest of the industry as a whole. Then, we will use the *Remove-Reduce-Raise-Create* matrix to create a new value-offer which is not currently offered on the market.

4.4 Blue Ocean Strategy - Applied

There are a number of applications which offer some way of blocking other applications and websites. We chose to focus on three significant companies. They represent the broad market decently well, with different strengths and weaknesses.

- Freedom - Founded in 2011, available for iOS, Android, Windows and MacOS
- Opal - Founded in 2020, available for iOS and MacOS
- Cold Turkey - Founded in 2017, available for Windows

The value-offers of the three companies are plotted on the strategy canvas on figure 1. We tried the applications ourselves and discussed how they should be plotted. These are subjective judgements based on our experience using the applications. They are also not rated using any objective scale, but simply plotted visually. This is how the method is presented in the book. [1]

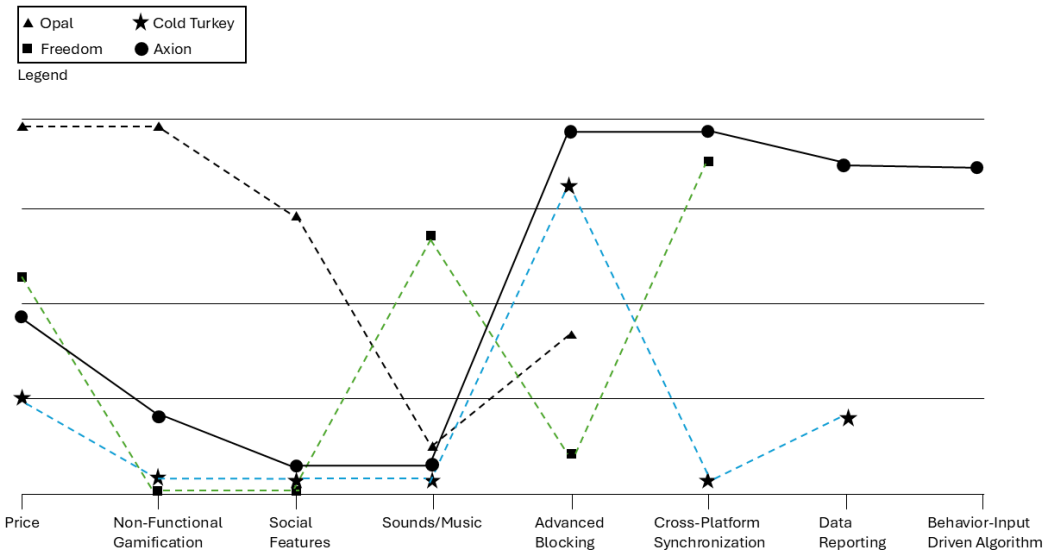


Figure 1: Blue Ocean Strategy Canvas

Our proposed solution is plotted as well. It was created by considering the value-offers of the three competitors and the rest of the industry and then using the *Remove-Reduce-Raise-Create* matrix, shown on figure 2.

Remove Social Features Sounds/Music	**Raise** Advanced Blocking Cross-Platform Synchronization
Reduce Gamification Price	**Create** Data Reporting Behavior-Input Driven Algorithm

Figure 2: Remove-Reduce-Raise-Create Matrix

Social features were removed because it seems redundant, almost gimmicky; to encourage users to create social media-like posts and scroll inside an application whose purpose is to reduce usage of other applications with such features. Sounds/Music was also removed, as it does not directly support the core purpose - it surprised us to see that Opal and Freedom, respectively, had spent 20 % of their navigation-bar space on these, in our minds, redundant features.

Gamification was heavily reduced, to the point where it was almost eliminated. While the application should feel great and responsive, and progress rewarding, Opal has gamified their application with a high number of arbitrary rewards - gems, streaks, shares and colors. This could potentially alienate some users and make the application seem gimmicky.

Price was reduced compared to the price of Freedom or Opal - but still with a subscription, hence the price is plotted higher than Cold Turkey.

Advanced blocking and cross-platform synchronization was raised. These two features are what is unique to Cold Turkey and Freedom, respectively. No current application combines them - Cold Turkey is a Windows-only product, and Freedom has very simple blocking options. Doing this already sets the new product clearly apart from existing ones with regard to the value-offer.

Data reporting was created. To our surprise, Freedom and Opal had no visualized screen-time statistics. Cold Turkey had a simple aggregation of minutes spent on different applications and websites.

To our knowledge, there currently exists no application which aids in configuring blocks for users. Thus, to successfully use any attention-managing tool, users must know themselves and their own behavior well enough to accurately estimate interventions needed. Furthermore, such interventions need to be constantly modified and updated, since user behavior may change.

A solution could be a behavior-input driven algorithm. The user sets goals - and the algorithm sets rules which helps the user achieve those goals based on current usage habits. The tool would continuously update the restrictions to match changes in behavior to fulfill a constant goal. This would be a powerful way to automate the process, significantly increasing the connection between what the user wants to achieve and what results they actually achieve.

4.5 Final Value-Offer

On figure 1, the final value-offer can be seen alongside the value-offers of Freedom, Opal and Cold Turkey.

Our solution could possibly carve out a market share by combining aspects of existing solutions in a new way. The solution is not in direct 1:1 competition with any existing solutions - users gained are not necessarily taken from other companies. The solution could create a "Blue Ocean" - a market which, for a time, does not face hard competition and where growth is therefore more likely.

Building the proposed solution with a satisfying quality is a large task. If the solution was to not just exist but be attractive to a broad audience, it would have to be commercially successful and be part of an economically sustainable business, which could maintain and develop it on a longer timeline. While the completion of all tasks are outside the scope of this report, the process is initiated and the progress documented. This is further described in Chapter 6.

4.6 Target Audience

Due to the lack of user data, it is difficult to pinpoint a specific target audience. To determine this accurately, data on potential users is needed to assess who would benefit most from the solution. Rather than making assumptions, the target audience will be kept at a high level, based solely on objective characteristics:

- People who subjectively believe they have a problem with excessive screen time
- People who are willing to take personal action to address the problem
- People who have the technical ability to use a technical solution

As the user base grows and more data becomes available, the target audience can be further refined. For the purpose of this thesis however, it will remain defined at this general level.



REQUIREMENTS

"Let's talk about Minimum Viable Product, or as I like to call it:
The art of building just enough to find out you're wrong"
- Agile Protocol: The Transformation Ultimatum, Scott M. Graffius

DTU

5.1 Minimum Viable Product

An acceptable Minimum Viable Product, from here on called MVP, should be able to provide answers to the riskiest assumptions a business has to make [22]. These encompasses both technical risks and market risks.

An application capable of controlling access to other applications requires direct communication to the operating system. It also must fit into strict regulations to be published on the primary application store on the platform. An MVP should at least be able to detect and block the use of other applications and websites. An MVP for an application which will later provide synchronization across mobile and computer platforms must also have a functioning and scalable account management system. The data structure of the programming objects must be independent, such that new functionality added to one object works with the other objects.

Relevant user engagement metrics should be defined, and analytics software utilized to obtain as much data as possible for improvement. Crashes and bugs should be noticed as quickly as possible after launch. Extensive measurements of usage metrics should be part of an MVP.

To decide which operating system was best for the MVP, a meeting was held with the Board of Advisors, introduced in section 6.6 Board of Advisors. The board agreed that a mobile application would be the ideal place to start. Additionally, Jan Borg (cand.it), stated that while iOS was more popular in Denmark, Android has more developmental freedom with regards to interacting with the operating system. If a good application was first made for iOS, innovation could be restricted by the operating system, whereas starting with Android would allow for the highest developmental freedom. Then, when expanding to iOS, workarounds and restrictions could be applied to the existing Android solution. Hence, it was decided that the MVP should be built on the Android operating system.

5.2 System Requirements

The application must support user authentication and account management, as it is central to storing and retrieving user data. This functionality is a prerequisite to implement cross-platform functionality in the future. Users must be able to sign up, sign in and sign out using an account management system that adheres to security guidelines. The authentication must be secure and persistent, allowing users to stay signed in between sessions unless manually signed out.

Setting up and enabling blocks is the core feature of the product. Users must be able to create new blocks by selecting specific applications or website to restrict access to. While the block is active, the application must monitor phone usage to prevent the user from accessing blocked applications and websites. If the user attempts to do so, they must be redirected back to the home screen and reminded of their ongoing block.

The application must collect and upload relevant data for analytics, such as application launches, user sign ups, block creations, etc. These events form the foundation for measuring engagement and usability of the application. The application must collect error data for debugging purposes.

On the backend, endpoints should be exposed to allow the application to fetch and store data. All requests must be authenticated and validated using tokens issued by the account management system. The backend must be designed for scalability to support growth and increased traffic. It must be deployed on a secure and reliable cloud infrastructure.

To further understand how the application will be used from the perspective of an end user, the system requirements are broken down into user stories. See appendix 10.9 MVP User Stories.

5.3 Non-Functional Requirements

The non-functional requirements outline the quality attributes and operational expectations of the product. These requirements describe how the system should perform and behave under different conditions and workloads. They ensure that the product is secure, reliable, maintainable and compliant with the relevant regulations.

The application must adhere to the General Data Protection Regulation (GDPR) [23]. Users must be informed about what data is collected and for what purpose. Users must have the option of requesting deletion of their data at all times.

All data transmitted between the application and backend must be encrypted using industry standard protocols. Authentication tokens must be securely stored and validated on every request to prevent unauthorized access.

The application must be reliable and capable of handling unexpected behavior without crashing or becoming unstable.

The application should uphold high usability and accessibility standards, ensuring good user experience across a variety of Android devices and screen sizes. Performance should be optimized to avoid noticeable lag or increased battery consumption.

The backend infrastructure must be scalable and capable of handling increased traffic without a decrease in service performance. Cloud-based deployment should ensure availability, monitoring and logging of system activity for future security auditing.

Storing user data in any form should adhere to standardized security best practices and uphold a high level of user privacy. This applies especially to sensitive data, like passwords.

The cloud infrastructure should maintain an up-time of at least 99%, ensuring that the backend remains operational and accessible with minimal interruptions.

PROJECT MANAGEMENT & EXECUTION

"Vision without execution is just hallucination"
- Thomas Edison, Inventor and Businessman

DTU

6.1 Introduction

This chapter dives into how the project was planned, managed and executed. It covers the key performance indicators, legal considerations and the agile development process used to structure the work. It also presents an overview of the individual sprints, along with the development guidelines that ensured consistency and quality throughout the project. It also covers the comercial aspects of the project.

6.2 Incorporation of AxionHQ ApS

AxionHQ ApS is the legal name of the company started to facilitate continued work on the project and eventual completion of the long-term vision.

Before incorporating, the type of company, legal aspects and finances was considered. In Denmark, there are several possible legal business types. We considered two different ones:

- Personally owned businesses (Single-person business or I/S)
- Limited Liability Company (Anpartsselskab, ApS)

As the name suggests, the Limited Liability Company offers the owners protection against claims which exceed the value of the company. Additionally, it is easier for potential investors to join in ownership of the company, an advantage for a startup. For these reasons primarily, a Limited Liability Company was founded and the starting capital of 20.000 DKK was invested. The CVR number is 45460495. The incorporation was completed on 12th of March 2025.

6.3 Monetization & Business Model

For a product to be sustainable and in continuous improvement, it is usually a necessity to monetize it. Several monetization options were considered. Among those was B2B subscriptions for the application to make people more productive at work, and advertisements on the application, at least in the beginning.

We ended up with a pure B2C product with a freemium model. The users will have the option to use the application for free, but may pay to unlock more advanced features. This is a difficult model to get started with. Generally, while selling to businesses can be more difficult in terms of compliance and trust, selling to end customers often results in a lower customer retention month over month, and thus a low Customer Lifetime Value (CLV). It presents high requirements for the application to be stable and user-friendly. B2C subscription application businesses are challenging to market, and require a strong effort in marketing from the founding team, unless the product is inherently viral (like a social media which benefits from the network effect).

There were some clear advantages however in choosing a classic subscription model. We can observe how existing businesses who offer similar products have at the very least some success with this business model, making it a viable option. The problem is also very widespread, as documented in Chapter 3, and the fact that customers could potentially save hours per day suggests a promising benefit, possibly enough to justify a subscription for the user. A classic subscription model also offers predictable revenue and more aspects to work with, as retention is not the only goal but also conversion rate.

6.4 Fundraising

We consider this company a startup, the type of company with a scalable product, high-risk, and often in need of external capital to survive the early stages. It will not be possible to pay below-market founder salaries from subscription revenue for a while, let alone market salaries

to hire external engineers. Hence, for the business to survive, it was necessary to seek external funding.

Startup funding usually happens in certain rounds - the round raised is determined by the stage and maturity of the company. Various solo investors and venture capital funds specialize in investing in specific rounds. It is important to know which stage the company is at when looking for investment and knowing what it takes to raise a certain round. The amount raised should be carefully considered. It must be enough to get the company from where it is at the time of the round closing, to the stage it needs to be in to raise the next round of funding. Otherwise, the company could find itself in a situation where it is not mature enough for the next group of investors, but too mature for the previous group. This can lead to a necessity of accepting an investment at a poor valuation in order to survive, or ultimately, bankruptcy. Some companies finding themselves in this situation may have the opportunity to lower their expenses and become profitable.

The first two rounds of funding are called Pre-Seed and Seed, and are meant for early stage companies with a strong growth potential. Later rounds of funding are called Series-A and Series-B, and are meant for more established and mature companies. They will not be discussed in detail here, as this company is at the Pre-Seed stage.

Raising a Pre-Seed round requires a clear idea and strategy, a team with all the necessary core skills to develop the product and gain traction, and ideally, an MVP. When budgeting the round, the estimated cost of reaching the milestones necessary to raise the Seed round should be considered. The requirements for Seed is generally said to be having a product-market fit, usually manifesting through increased month over month growth, a high month-over-month customer retention rate, and a monthly recurring revenue of 30000 - 120000 DKK.

Both private investors and public funding programs were potential funding options. We had a meeting with Accelerace Capital Fund [24] after showing them our pitch-deck, but ultimately decided to apply for the public funding program Innofounder by Innovationsfonden, a Danish state-funded program. To see the latest version of our pitch-deck, refer to the supplementary file `Pitch Deck.pptx`. Innofounder is for Pre-Seed stage companies with innovative solutions that fit into one of three categories, deemed important for the progress of Danish society [25]:

- Environmental Technology and Innovation
- Life science, health and welfare technology
- Strategic and challenge-driven innovation in new technological spaces

Our product fits into the category of life science, health and welfare technology, specifically mental health. Refer to Chapter 3 where we argue that the problem is a public health issue.

For aid with the application, we hired the management consulting firm Thursday Consulting [26], who has a section which specialize in services related to public funding applications. They have high success-rate, and the far majority of the cost of their work is conditional on the Innofounder being granted, minimizing our risk.

6.5 Financial Projection

As stated earlier, it is important to ensure that the amount raised in the Pre-Seed round suffices to reach the milestones necessary for raising the Seed round without running out of capital. We made a series of 18-month forecasts, which all assume that we successfully get the Innofounder, or another equivalent amount of Pre-Seed funding. The purpose of the forecasts is to have a plan for reaching Seed metrics before the money runs out in the event of a successful Pre-Seed round.

When attempting to project startup revenue growth, it is important to not assume growth without properly allocating time and money to drive said growth. As stated earlier, depending on the natural virality of the product alone is not sufficient for most products, and certainly not subscription products. Hence, we mapped the avenues we could use, primarily organic, to drive user and customer growth. Among them was various organic marketing strategies, such as our own YouTube channel and potentially getting featured on the news.

These are plausible but time consuming ways of growth. Our strategy will be to try out several strategies, and then continue with the ones giving the highest return on the time spent. Predicting the growth from these sources is hard, as we have not started marketing efforts yet and therefore have no data. To keep costs low in the beginning however, we applied for Microsoft for Startups [27]. This allowed us to get an initial 5000$ in credits which we are using to run for the cloud computing setup. Most of the cloud computing services from Microsoft has the payment feature "pay-as-you-go", meaning we only pay for the services which are used. The price will increase with a growing user base, but for now, the cost is minimal and the initial credits will be more than sufficient to run the services without extra cost.

To make realistic estimates, we mapped five different assumption-based scenarios. All of them are based in the freemium subscription business model, and assume the following starting metrics: A conversion rate of 2% from free to paid, a monthly free user churn of 40% (how many users are lost each month) and a paid customer churn of 20% per month. These numbers are considered fairly conservative, even for B2C products. The scenarios begin in July 2025 and assume successful funding of at least 430.000 DKK granted in January 2026. Before January 2026, no salaries will be withdrawn. The results in terms of users, customers and revenue by December 2026, as well as the remaining account balance is described for each scenario.

- Scenario 1: University deals and promotion to students go well. The product revenue is 27.300 DKK/Month, and the product has 2169 users of which 1290 are paying customers (through the discounted deal with the universities). The reason for the high ratio of customers/user in this scenario, is due to the successful university deal, where the individual users do not have to pay. The remaining bank-balance is 122.000 DKK.
- Scenario 2: Organic marketing (SEO, YouTube channel) and news stories are successful. The final product revenue is 18.100 DKK/Month, and the application has 7238 users of which 452 is paying customers. The remaining bank-balance is 216.000 DKK.
- Scenario 3: The application development goes well, and we manage to create some virality and significantly lowering churn. The product revenue is 16.000 DKK/Month, and the application has 12120 users of which 398 are paid. The final bank balance is 194.000 DKK.
- Scenario 4: Best case, all of the initiatives go well. In this scenario, the final revenue is 69.000 DKK/Month, and the application has 27000 users of which 2611 are paying customers. In this scenario, the company balance at the end of the period is 140.000 DKK, and the company has had the opportunity to hire one additional developer.
- Scenario 5: Worst case, none of the initiatives pay off well. In this scenario, the final revenue is 5.700 DKK/Month, and the application has 3896 users, 151 of which are paying customers. The bank balance is 158.000 DKK.

The projections show that in scenario 1 and 4, the company will be at a stage where raising Seed capital is realistic in the end of 2026. In scenario 2 and 3, a little more growth will be needed, but since the company has a healthy bank balance and low expenses, it should be possible to reach at least 30.000 DKK in revenue to be able to raise Seed capital, before the company runs out of money. In scenario 5, the revenue is not sufficient to reach Seed capital, and it is possible

we would have to pivot or raise another Pre-Seed round if we found ourselves at the end of 2026 with these metrics.

The financial projections show, that if some or all of our growth initiatives go as planned or better, we will reach Seed metrics before running out of money. Getting the Innofounder grant would therefore be enough Pre-Seed funding, provided we can reach the assumptions.

To see the full projection and the calculations, refer to the supplementary file `User Growth & Revenue Projection.xlsx`.

6.6 Board of Advisors

We assembled a board of advisors who has a range of knowledge to aid strategic decision making. We had four board meetings, both physical and online. Information about the board members and meeting summaries can be found in appendix 10.8 Board of Advisors.

6.7 Key Performance Indicators

It is important for a business to know and measure its Key Performance Indicators (KPI's). We have taken inspiration from other subscription B2C businesses to understand which metrics are important.

- Churn - How many customers in % are lost on a monthly basis. The most important KPI for our business, but also one that is hard to measure since it requires acquiring a significant number of customers and then measure the rate at which they stop using the product.
- CLV - Customer Lifetime Value - Calculated by multiplying the subscription payment of a customer with the amount of time they stay in the business. It is necessary to know the churn to accurately estimate CLV.
- CAC - Customer Acquisition Cost - This KPI is usually tied to a specific marketing channel. It describes how much money the business must spend on the channel to gain one customer. This is difficult for us to measure directly as most of the marketing strategies we mentioned are organic, and it is less clear how many customers come from each channel and how expensive the channels are to run. A way to do the calculation instead, could be to use time spent instead of money, to measure efficiency of organic marketing strategies.
- Conversion Rate - How many users go from being users to being customers in %. Maximizing the conversion rate is important to monetize the user base more efficiently.
- Free User Churn - How many free users are lost per month. Less important than the other metrics, but the chance of a user converting to a customer increases if they do not stop using the product all together.

These are the most important KPI's. A lot of data is needed to estimate them correctly, and for some of them, like churn, some time is needed as well. Early tracking of the changes in the KPI's before the long-term data is obtained speeds up the process of iteratively building, tracking, learning, ideating and building again. For example, higher engagement with premium-only features could suggest the customer is gaining value from the premium subscription, and may be less likely to leave.

At this point, we do not have data enough to estimate the KPI's, but we set up measurements on Firebase analytics, Googles own analytics system, for engagement metrics which serves as indicators of interest:

- Active Users - Defined as having the application installed, accessibility services turned on, and having had at least one active block within the last seven days for more than 20 minutes.
- Long Term Users - Active users that have been active users for at least 14 days.
- Churn Rate - % of long term users lost, measured over a given period.
- User Growth Daily - How many new active users do we get per day.
- Long Term User Growth Daily - How many new long term users do we get per day.

6.8 Legal Requirements

It is mandatory to have a Privacy Policy and Terms of Service available for the user to read. To have the application published on Google Play Store, the documents would have to be available on a web URL owned by the publishing company. To follow GDPR rules about transparency regarding the gathering and use of data [28], users are also required to read and accept them, before creating an account.

We purchased the domain `axionapp.io`, built a landing page and put our Privacy Policy and Terms of Service on the website. As no users are currently funneled from the website to the application, it mostly serves as a landing page and identity for the company, and a place to store our documents for users to read. For more information on the website, refer to section 7.7.

The application uses a special API called Accessibility Services. It is this API that enables the application to enforce rules outside of the application, and read what actions the user performs. For a non-accessibility application (an accessibility application would be a text-to-voice reader) it is mandatory to ask permission from the user and explain thoroughly what the service will be used for, to avoid exploitation of users and secure compliance with European GDPR laws. We have put alerts in our application and on our Google Store Listing following these guidelines, and we were granted the permission from Google to use the service.

6.9 First Marketing Campaign & Feedback

To get initial users and feedback, we conducted a field marketing campaign at DTU when the application was launched. The purpose was to engage directly with students for first hand feedback, but also to increase the initial user base to collect data for identifying potential bugs, improvements and missing features. The campaign involved approaching students at DTU, presenting the concept and encouraging them to download the application and share their impressions.

Flyers were also distributed at key locations around campus where students typically study and work. To further increase effectiveness, the campaign was carried out during the exam period and the flyer emphasized the message "Ace Your Exam". The flyer can be found in appendix 10.6 DTU Flyer.

The flyer included a QR code. To monitor engagement and redirect users properly, we used the website Linkly [29] to generate the QR link. Linkly allowed us to track the number of scans, the devices used and the scan timestamps. It also handled redirection based on device type, where Android users were sent directly to the Google Play Store listing, while users on iOS, Windows or other platforms were redirected to our website, which explains that the application is currently only for Android, but coming to iOS and Windows.

The campaign was launched on the 23rd of June. From that date until now, a total of 41 scans have been registered. Of these scans, 21 was on Android, 19 on iOS and 1 on Windows.

As part of the campaign, we directly approached students and encouraged them to download and try the application. This method, often referred to as concept testing, allowed us to gather unstructured, real time feedback on the user experience.

These testing sessions revealed consistent issues. Users generally found the application confusing upon launch. At the time, no tutorial was implemented, so users were immediately met with a sign in or sign up screen without any context. This led to uncertainty and hesitation, as users did not understand the purpose of the application before making an account. A few users also encountered a crash when attempting to enable a block. This negatively impacted how users perceived the application and reduced the willingness to continue exploring its features.

These insights were crucial in guiding improvements. They directly influenced the development of the tutorial screen, detailed in section 7.3.3 Introduction & Tutorial, and the implementation of the global exception handler, described in section 7.3.8 Global Exception Handler.

6.10 Agile Development

Agile development is a methodology for managing software projects in a flexible and adaptive manner. It supports iterative planning while allowing for changes throughout the development process. Rather than creating a fixed project plan from the start of the project, the process is divided into smaller, more easily manageable iterations called sprints. This approach was chosen for the project as it aligns well with software development, where requirements and focus often evolve over time, especially after the launch of an application.

Each sprint contains a defined set of tasks, that has been planned before starting the sprint. By the end of each sprint, the product must be in a functional and tested state. In other words, all tasks within a sprint must be completed in full, unless they have been pushed to the backlog. If they are not pushed, partial implementations or incomplete features are not accepted. This ensures continuous delivery of a usable and complete product at every stage. If tasks were not completed within the sprint, due to changing priorities or time constraints, they were moved to a backlog. During the planning of the following sprint, the backlog was reviewed to determine which tasks should be prioritized or removed.

Working in an agile environment allowed us to respond quickly to unexpected developments. This included failed implementations and new insights and ideas gained through meetings and user feedback. These events influenced the planning of the next sprint. With six total sprints, the MVP was completed in sprint three. This means, that after sprint three, the plan and requirements started to be loosely defined. New features and requirements were instead discussed and planned before each sprint, taking user feedback into account. To see the full sprint plan, see appendix 10.10 Sprint Planning. The six total sprints, equals five total iterations that was completed. Each sprint concludes with a new iteration of the product. Sprint six however focuses on the completion of the thesis and minor bug fixes, therefore not being regarded as a full iteration. With the MVP being done in sprint three, we have completed two more iterations, that further improved upon the initial idea of the MVP.

To further expand the agile environment to not just the development process, the sprint structure included other areas as well. This ensured that all areas contributed toward the overall product goals and development, during each sprint. The task categories used in the sprint planning were:

- **Technical Tasks** – Software development and implementation work.
- **Business & Finance Tasks** – Internal business related tasks, including budgeting and operational decisions.
- **Organizational Tasks** – Tasks related to project planning, such as sprint planning and internal meetings.

- **Customer Success Tasks** – Tasks such as gathering user feedback and defining KPIs.
- **Sales & Marketing** – Tasks aimed at user acquisition and the conversion to long-term users.

6.11 Development Guidelines & Principles

To ensure code quality and maintainability, the development process focused on several principles and best practices.

The main branch was kept deployable at all times, with all changes developed on feature branches and merged via pull requests after testing, see section 7.10 Version Control & CI/CD for further details.

We followed the principle of separation of concerns, structuring the codebase into well defined modules. This is also referred to as Hexagonal Architecture [30]. It was done to simplify testing and extensibility.

Security best practices were followed. Secrets like passwords and tokens were managed via environment variables or Azure Key Vault, see section 7.5.1.6 Key Vault - axion-cloud-vault for further details. Sensitive data was never stored in code.

The overall project structure followed established software engineering principles, particularly SOLID and DRY. SOLID is a set of five guidelines that improve the maintainability and flexibility of codebases. While these principles are especially valuable in larger projects, research shows that they should be applied thoughtfully and can be adapted when needed to balance performance and complexity [31]. The following outlines each principle based on this research.

S – Single Responsibility Each class or module should have one clearly defined responsibility. This is the sane as the principle of separation of concerns.

O – Open/Closed Principle Software components should be open for extension but closed for modification. Existing code remains stable while new functionality is added via extensions, rather than modifications.

L – Liskov Substitution This principle states that, when building classes in a derived manner, the child class should behave the same as the parent class, without modifications. The child class should always extend the functionality, to minimize confusion and keep the integrity of the functionality, for any classes that has been derived from the parent class.

I – Interface Segregation Interfaces should be specific to individual needs. It is better to use multiple small, but more focused interfaces than a single, general purpose one.

D – Dependency Inversion High-level modules should depend on abstractions, not fundamental implementations. This decouples the system and enables easier swapping of components. This is especially useful, when using third-party APIs or libraries.

Using the SOLID principles supports the building of a flexible and maintainable system design. The DRY (Don't Repeat Yourself) principle was applied to avoid code duplication and instead reuse logic. These combined efforts contributed to a scalable and clean codebase.



PRODUCT & DEVELOPMENT

”Bad programmers worry about code.
Good programmers worry about data structures and their relationships”
- Linus Torvalds, Creator of Linux and Git

DTU

7.1 Introduction

This chapter describes the important choices and considerations we made during the product development process.

It begins by explaining the selection of the technical stack. Then, it will describe the architecture of each major system component; the mobile application, the backend, the database and the cloud infrastructure. The chapter also goes over supporting tools such as error monitoring, analytics, version control and CI/CD. In the end of the chapter, we will describe challenges and unimplemented features.

Before delving into the chapter, we will first go over the concept of a **block**. This concept is central to the application and is referenced extensively throughout the chapter. Alongside the block, other related concepts are also defined and explained for clarity.

A **block** is an object which has the ability to prevent access to specific parts of the user's device. A block contains a list of applications and a list of websites. A block contains a value called **activation**, which defines whether the block is enforced. A block also contains a value called **lock**, which defines whether the user may edit, turn off or delete the block. The user will be able to configure the applications, websites, activation and lock of a block.

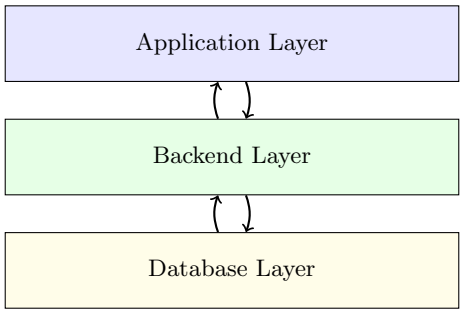
Each block contains several configurable variables. The most important includes:

- `isOn` – Indicates if the block is turned on
- `isActive` – Indicates whether the block is currently active
- `isLocked` – Indicates whether the block is currently locked
- `websiteList` – A list of websites
- `applicationList` – A list of applications
- `lock` – The method of controlling the value isLocked
- `activation` – The method of controlling the value isActive

For a full overview of the data structure of a block, see appendix 10.12 Class Diagram of Block.

7.2 Technical Stack Selection

Three main components is required to build the solution; the application layer, the backend layer and the database layer. A layered architecture is important when implementing functionality [32].



The application layer contains the application itself. The backend layer takes care of any operation that requires data being stored, updated or deleted. The database layer is where the data is stored.

When a user registers, the application layer receives the necessary information and sends it to the backend layer. The backend layer process the data and stores it in the database layer. We explain this subject in section 7.2.7 System Architecture Overview.

7.2.1 Application Layer

The system chosen for the application layer must support an Android application which performs continuous background tasks and interacts with accessibility services to allow system level control. This requires a development environment which provides native system access.

Table 1 lists the candidates evaluated for Android development.

Candidate	Notes
Kotlin	Native Android, modern language with better exception handling
Java	Native Android, long-established but potentially more volatile
Flutter (Dart)	Cross-platform (Android, iOS, desktop), single codebase, less targeted

Table 1: Evaluated candidates for the Android application layer

In selecting the mobile development environment for our application on the Android platform, we evaluated Kotlin, Java and the framework Flutter, as potential candidates. Kotlin and Java are both native languages for the Android platform, while Flutter offers a single codebase across Android, iOS and desktop platforms. While Flutter would shorten development time, the deep level of integration with the operating systems needed to implement blocking functionalities requires using a language native to the operating system.

Java has for a long time been the standard of Android development. Kotlin has over the last five years evolved into a modern Android language, and has better software safety, error handling and stability [33]. Kotlin was chosen due to the stability advantages.

7.2.2 Backend Layer

The backend needs to be cloud based and scale easily. It must to offer a robust and secure REST API, handle tasks asynchronously and support frequent updates with high reliability. The REST API is explained in section 7.4.2.

Table 2 lists the candidates evaluated for the backend.

Candidate	Notes
Java	Aligns with AWS ecosystem, mature cross-platform frameworks
C#	Native integration with Azure (Founders Hub credits, .NET ecosystem)

Table 2: Evaluated candidates for the backend layer

C# offer different language features, such as asynchronous programming via async/await, strong typing and integrated support for dependency injection. It also offers the integration with the .NET framework. Java has advantages in ecosystem size and cross platform server frameworks. Besides the languages, we considered that Java aligns with Amazons AWS services and C# integrates natively with Microsoft Azure services. Microsoft has the "Founders Hub" program [27]. The program gives startups credits, which can be used to run code on Azure's cloud

services. The synergy between C# and Azure and the Founder's Hub program, made it the optional choice.

7.2.3 Database Layer

The database layer requires a relational model to manage structured data. This ensures consistency and integrity through schema enforcement, foreign keys and normalization. This is further detailed in section 7.6.

Table 3 lists the candidates we evaluated for the database.

Candidate	Notes
MySQL	Native integration on Azure, simple setup, widely used
PostgreSQL	Extensive JSON support, extension frameworks
MongoDB	NoSQL, flexible schema but complex transactions

Table 3: Evaluated candidates for the database layer

MySQL was selected for its native Azure support and its alignment with the relational data model. While PostgreSQL offers advanced features with better JSON support and extension frameworks, these are unnecessary. NoSQL options like MongoDB or GraphQL could complicate schema management, making it less suitable. The maturity and industry adoption of MySQL made it a clear choice.

7.2.4 Error Monitoring

Further requirements included real-time error monitoring across all system layers to detect and resolve issues in production.

To meet these needs, Sentry was chosen as the error monitoring platform. It offers a free plan for Kotlin and C#, real-time alerts and detailed error reports with context. SonarQube was also considered as it excel at static code analysis, but its licensing lacks a free tier covering both programming languages. The capabilities of Sentry allowed efficient detection of issues in both the application and the backend without extra cost.

7.2.5 Analytics

Another requirement was the ability to collect application usage data, in order to evaluate feature usage and guide future development.

Firebase Analytics was chosen to capture user interaction events and session metrics. Firebase Analytics was the optimal choice, as it is a tool with no platform restrictions, allowing us to track behavior across all current and future platforms.

7.2.6 Automation & Version Control

To ensure reliable collaboration between us and consistent delivery of new features, version control and automation were required.

GitHub was chosen for version control, as it is the most widely used and supported system for software development. With its maturity and global adoption, it was a natural choice.

Using GitHub, it made sense to incorporate GitHub Actions as the continuous integration and deployment service. GitHub Actions automated builds, tests and Azure deployments in

controlled workflows. Its native integration with GitHub repositories minimized configurations and ensured that new features and fixes could be delivered to users with minimal manual work.

7.2.7 System Architecture Overview

Figure 3 shows the entire system and the technical stack for each component. The solid arrows represent calls between the components, where the dotted lines with arrows represents the response that the action will give. The `Backend Layer` and `Database Layer` is wrapped inside a box, showing that the components are part of `Azure`.

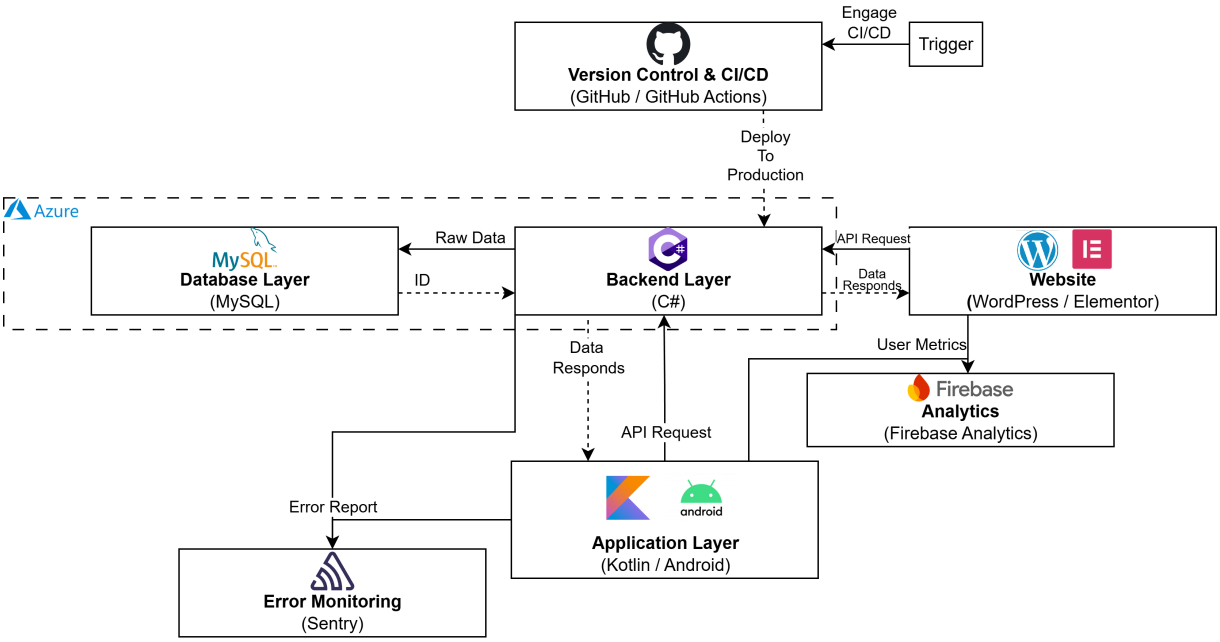


Figure 3: Overview of Architecture

The backend acts as the main connector between the website, the database and the application. The application and website can both communicate with the backend via HTTP requests to its API. The backend handles all database interactions, ensuring data is stored correctly by sending raw data. Upon successful insertion, the id of the entry is returned. This can be used to identify and manage individual entries.

GitHub is used for version control and CI/CD automation. More on this in section 7.10. Only the backend is part of the CI/CD pipeline, deploying to Azure via a manual trigger. Google Play integration for the Android application was planned, but restricted by third-party upload limitations from Google.

Firebase Analytics collects user metrics from both the application and website. The backend does not report user metrics, as it lacks individual user context. Sentry handles error monitoring for the application and backend. The website is excluded due to minimal complexity.

The architecture centralizes all cross-platform logic in the backend, maintains a single source of truth in the MySQL database and uses managed services for analytics, error monitoring and CI/CD.

7.3 System Architecture - Android Application

This section will go through the development of the Android application, both detailing the implementation and considerations.

7.3.1 MVVM and Jetpack Compose

The UI toolkit we used is called Jetpack Compose. It is a declarative framework [34], and works by "composing" the individual UI functions, called "composables". Jetpack Compose only needs to update the elements which has been changed - unlike older toolkits, it does not recreate the entire screen every time data changes.

It is good practice to store the application data properly, so that each variable only exists in one place. A repository is a class that functions as the true source of data. Letting the UI elements call the repository directly is not ideal, as it does not allow for temporary changes in data. For example, if the user wants to edit an object and they make changes, they cannot click cancel and delete their changes, as the data in the repository has already been changed.

To handle temporary data, we use an MVVM, Model-View-Viewmodel, architecture. In the MVVM architecture, the viewmodel is a class that communicates with the UI elements and the repository. If the user clicks a button to change data, the button composable calls the viewmodel to change its data.

If the change needs to be saved, the viewmodel calls the repository to transfer the changes. This separates concerns, and provides a flexible architecture with which updating the application will be unlikely to conflict with existing data structures. On figure 4, the manner in which data flows down and events flow up through the data hierarchy can be seen.

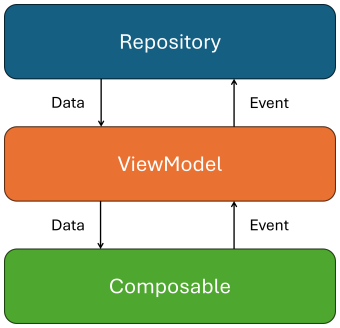


Figure 4: Data Hierarchy

7.3.2 UI Design

Every individual component on the screen is visualized with a composable function. Composables can consist of other composables, as the entire screen is a function composed of the individual UI elements. This structure makes designing larger composables simple. This can be seen on figure 5.

The individual composables call the viewmodel to access the data they use to create the UI. For more information on this process, refer to section 7.3.1 MVVM and Jetpack Compose.

Designing the user interface is crucial. A well-designed UI clearly communicates functionality, enabling users to intuitively navigate and understand where to find features.

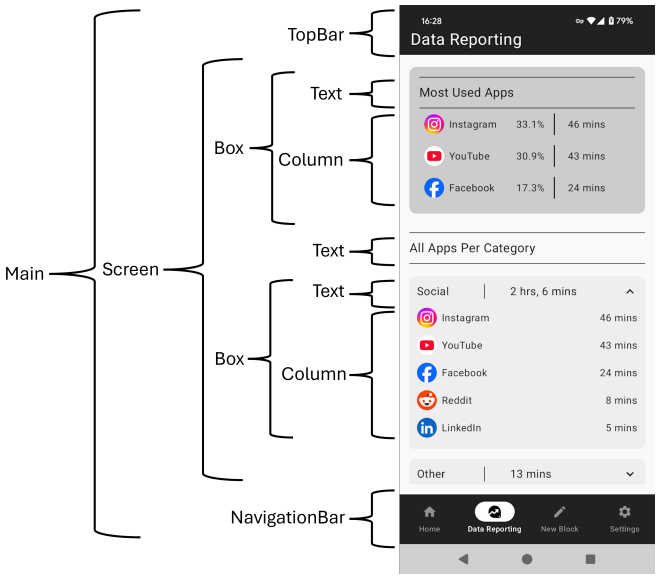


Figure 5: UI Architecture

One of the most important design choices we faced was about the structure of the navigation bar. For a minimalistic design and simplicity, we believed there should only be 4 tabs, which is visualized in figure 6.

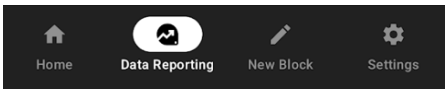


Figure 6: Bottom Navigation Bar

The 4 screens which make up the general user-perceived structure in the application are:

- Home: Consists of an overview of existing blocks. From this screen the user can turn on blocks and set their advanced blocking options.
- Data Reporting: This screen gives a concise overview of the users screen time, spread among applications. It is prioritized as the second screen, as insights from usage data are essential for driving behavior change. By first understanding their usage patterns, users can find inspirations for block setups.
- New Block: This screen is for creating a new block. On creation of the block, only the content (applications, websites and categories) of the block is defined.
- Settings: Holds personal settings, account management and developer feedback buttons. When the payment integration update is finished, in-app subscription management will also reside in this screen.

The 4 screens respectively are for managing and turning blocks on and off, overviewing usage data, creating new blocks and managing account and settings. This gives a clear and user friendly structure to the application. For a detailed navigation bar comparison to some of our competitors, see appendix 10.15 Navigation Bar Comparison.

The application uses a dark, grey, white, and blue colour palette to emphasize a sleek and modern design. Top and bottom bars are dark, while the main screen background is white with

grey text boxes. The highlights when creating new block and dialog box which opens when the user turns on a block, are both blue. In that way, blue is the primary action colour in the application, letting the user associate blue with changes. Buttons are dark against the white background to make them stand out. Active blocks show green switches, following standard mobile UI conventions, while inactive elements appear faded to clearly distinguish between interactive and non-interactive components.

7.3.2.1 Responsive Design

Snackbars are short, temporary messages that appear at the bottom of the screen. They provide immediate feedback, confirming actions or prompting the user when additional input is needed. For example, snackbars appear when a user attempts an action with incomplete data or successfully edits data, ensuring clarity and responsiveness in the UI.

To improve responsive feel, recently created or edited blocks are automatically moved to the top of the list and briefly highlighted in blue. Turning on a block also moves it to the top. This ensures that new, modified or recently activated blocks remain easily accessible.

7.3.3 Introduction & Tutorial

The introduction and tutorial for the application, was implemented when we ran the first marketing campaign, explained in section 6.9 First Marketing Campaign & Feedback. Based on user feedback, we chose to implement a tutorial screen, that the users encounter first instead of the sign up screen, aiming to lower the barrier of entry for new users. As this feature was not part of the MVP, requirements were defined. The tutorial should present the core features of the application, explained with both images and minimal text. It should be easy to understand and guide the user to handle the application, without using it first. It should present the user with the permissions the application requires in a transparent and open way. Introducing the permissions to the user before having them make an account helps the user understand the purpose of the application, before facing the first barrier of entry. The full implementation of the tutorial can be seen in appendix 10.7 Application Tutorial Screens.

7.3.4 Accessibility Service & Blocking Functionality

Blocking functionality on Android requires special system level permissions, to detect and respond to changes in foreground activity. Foreground activity is what the user can see. This is achieved using an `AccessibilityService`, which is a specialized Android component that can observe and interact with system visual elements. The accessibility service gives control over the system and therefore requires a special permission from the user. Google also requires an explanation for the usage of the service, before it can be published to the Play Store. The alert to inform the user to adhere to the guidelines from Google can be seen in appendix 10.14 Accessibility Permission Alert.

The main responsibility of the service is to track which application is currently in use and compare it with the list of currently blocked applications and websites. The lists are aggregated lists of all applications and websites currently in an active block. When a blocked application is detected as the current foreground application, the service immediately launches a blocking activity. This activity forces the user back to their home screen and sends a notification. This is visualized in figure 7, which outlines the sequence of checks and actions performed.

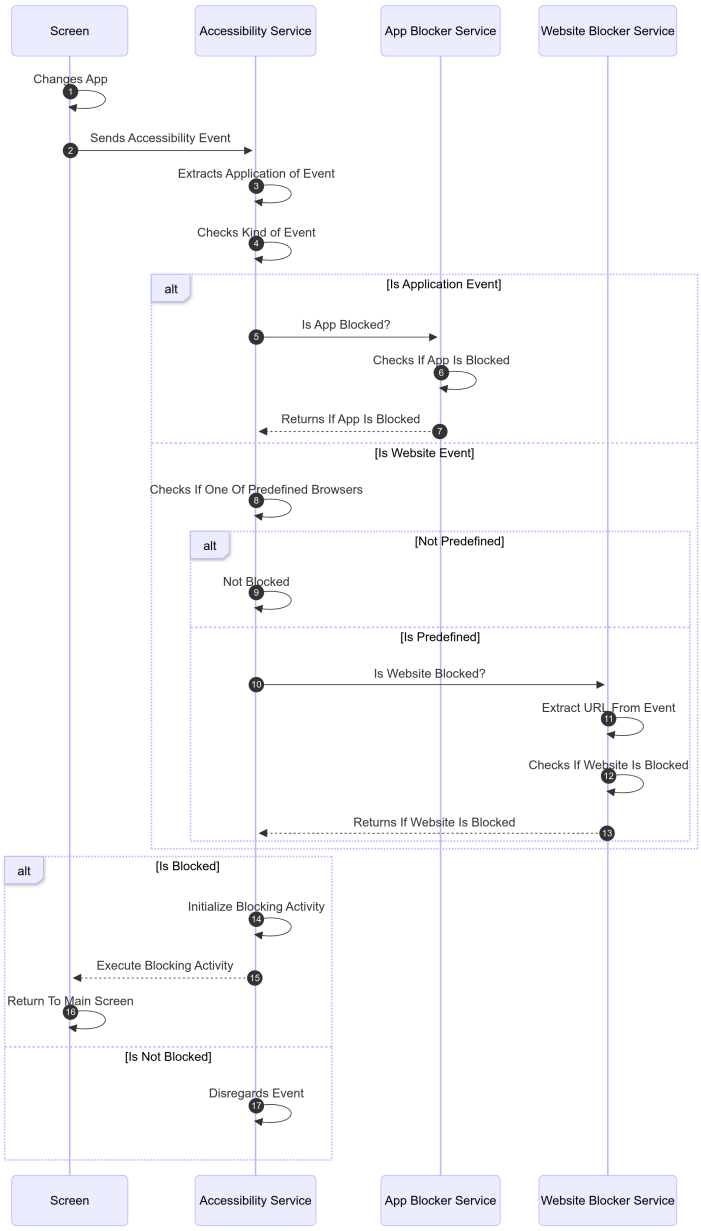


Figure 7: Sequence Diagram showing how it is determined whether the event is blocked

To ensure minimal performance overhead, the service is only active when blocking is enabled. It runs in the background and is triggered by accessibility events of a specific type. The type of event indicate that a new application or activity has come to the foreground. From the event the service can then extract the package name of the active application and compare it against the list of blocked applications, websites and categories.

When a match is found, the blocking activity is started with a specific flag. The flag ensures that the activity executed will force the user back to their home screen. Users can only access the application once the block is no longer active or if they turn off their accessibility service.

To prevent the user from blocking essential applications, the application is fitted with a predefined list of system essential package names. Applications on that list cannot be blocked.

7.3.5 Advanced Blocking

In the 2nd and 3rd iterations of the application, upon turning on a block, the user was faced with a pop-up box with three choices; cancel, lock without picking a time and pick a time. If the user was to pick a time, they could not disable the block before the timer expired. To ensure that the user did not block an application or website for an extensive amount of time by accident, the maximum duration was limited to 14 days.

This timer-based approach aimed to introduce commitment, a core feature of the application. One of the major problems with the operating system native solutions for managing screen time (for example Apple Screen Time) is they do not allow the user to set an unbreakable rule.

However, time-locking is just one method. Other applications have had success with allowing the users to set a short delay before opening an application. Freedom and Opal offer weekly scheduling of blocks, such that a user could block certain applications while at work or studying.

In comparison, our initial time-lock implementation seems too simplistic. Users should be able to tailor their blocking strategy based on their needs.

We had a brainstorm, where we mapped all the ways of blocking we could imagine. Examples would be by time, with password, with a delay to open, with a small challenge, with restrictions on the amount of opens per day, etc. We then tried to group these ideas into categories, as not being able to edit a block for a certain time is not the same as the block being enforced.

The result of the innovation exercise was a three-variable system: Locking, Blocking and Access. Blocks have a variable for each of the three variables, and the user must select one value for each variable:

- Locking: When can the block be edited?
- Blocking: When is the block enforced?
- Access: How is the block enforced?

When a block is turned on, a dialog box is opened. See figure 8.

The dialog allows the users to easily combine the three settings. For a responsive feel, the currently selected options are shown in the bottom of the dialog. In Iteration five (the current version of the application), there is four options for locking, two for blocking and one for access. The setup allow us to quickly add new options for each category if needed. The locking options are:

- None - The block may be edited or turned off anytime
- Timed - The block is locked for a certain period
- Delay - The block is locked, but can be unlocked any time with a delay. When the user turns off the block, a timer with the chosen delay will start, and the block will turn off once the time has passed
- When Active - The block is locked only when it is active according to the blocking schedule. This is to let the user edit blocks when they are not active, but not turn off the block to access entertainment right away

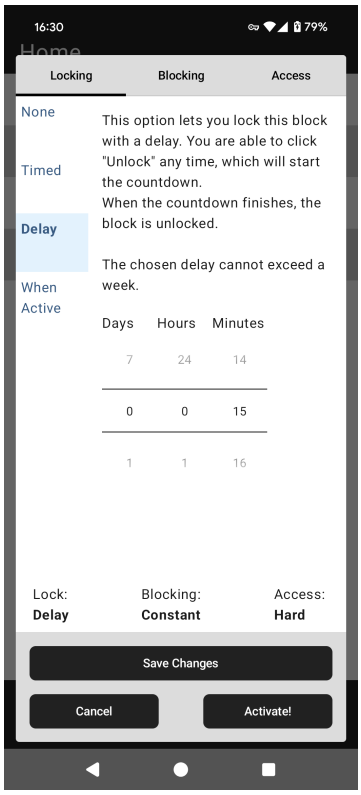


Figure 8: The Block Options Dialog Box

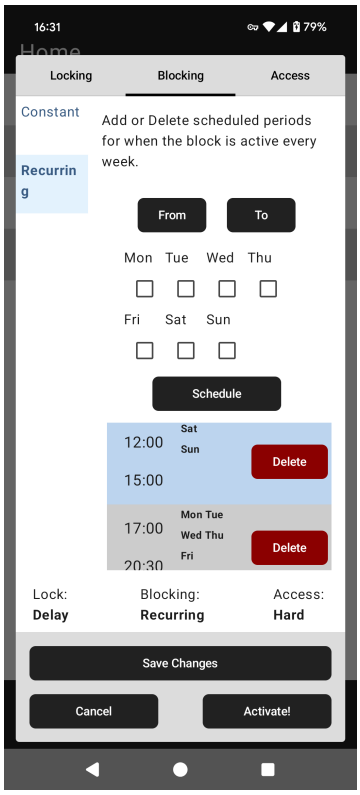


Figure 9: Creation of a Weekly Recurring Block

These locking options may be combined with the following two blocking options:

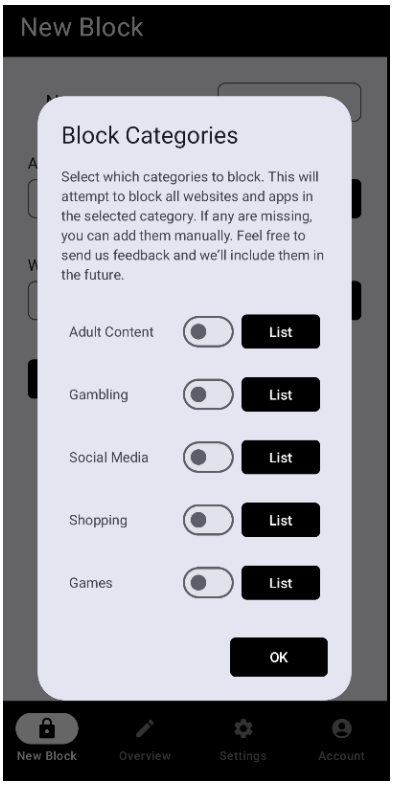
- Constant - The block is always active and enforcing the chosen access setting
- Recurring - The user chooses any number of weekly recurring periods. The block will be active during the chosen periods only. The interface of time and weekday selection for this type of block can be seen on figure 9.

Note, that to avoid locking a block permanently, it is not possible to combine the "When Active" locking with the "Constant" blocking. This is clearly communicated to the user if they try.

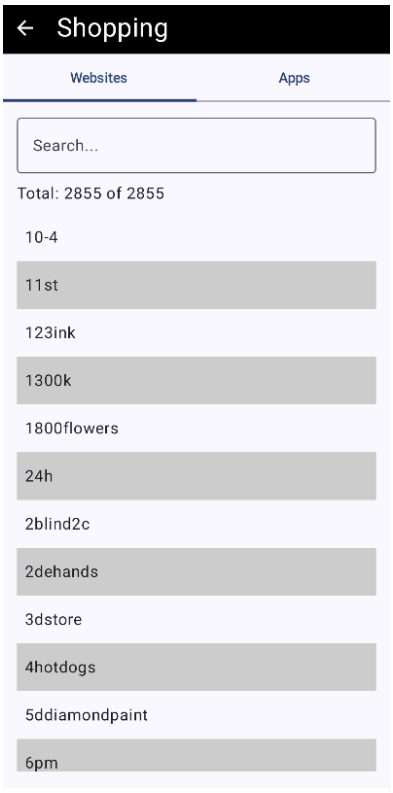
7.3.6 Category Blocking

To improve usability, a feature was added allowing users to block entire categories of websites and applications with a single click, instead of specifying each individually. Although not part of the original MVP, it was inspired by a meeting with the "Center for Ludomani" [35], who highlighted the need for an easy way to block all gambling related content. While the meeting did not result in direct collaboration, the feature was still implemented. The minimum supported categories included adult content, shopping and social media to address a broader audience.

The original plan involved using a VPN-based DNS filtering approach for more accurate categorization, but this proved unfeasible, see section 7.11.2. A simpler method was instead used. The application currently supports blocking the categories: **Shopping**, **Games**, **Gambling**, **Social Media** and **Adult Content**. Games and gambling were added after further discussion. A visual example of this feature is shown in figure 10.



(a) Category Blocking Dialog



(b) List View of Shopping Domains

Figure 10: Category Blocking UI

7.3.6.1 Websites

Blocking websites by category is best done using third-party resources rather than building our own system. As discussed in section 7.11.2, large companies use advanced tools like large language models and web scrapers for accurate categorization, which were not feasible to construct for this project. Instead, we used open-source projects that provide community maintained domain lists, originally created for blocking on personal routers.

The main sources for domain lists were:

- Corbin Davenport – international shopping domains [36]
- Steven Black – gambling and social media domains [37]
- Irek Szef – game related domains [38]
- Emilio Dalla Torre – adult content domains [39]

To tailor the lists for Danish users, we added domains from:

- Licensed gambling sites from Spillemyndigheden [40]
- Danish online business sites from WebshopListen [41]
- Major Danish businesses from Wikipedia [42]

These sources were combined and supplemented with custom entries to create as comprehensive category lists as possible. Table 4 shows the number of website domains per category.

Category	Number of Entries
Shopping	2855
Games	7437
Gambling	3617
Social Media	30
Adult Content	8684

Table 4: Number of Domain Entries in Each Category

To maintain performance, the domain lists are converted into hashmaps, when the application first loads. Each domain is hashed at application startup, enabling $O(1)$ constant time lookup, rather than $O(n)$ linear time searches. This optimization makes sure that the feature is scalable in performance as the number of entries might increase in the future.

7.3.6.2 Applications

Unlike websites, no open-source lists exist, to our knowledge, for application categories. We instead rely on the categorization from Google Play Store, which requires a look up for each installed application. This could potentially be costly for performance, dependent on the total number of installed applications.

To optimize this, the system first checks if the application is already categorized and cached locally, avoiding repeated look ups. If not, it checks a manually maintained list for categorizing applications with known incorrect categorizations. Afterwards, system applications are excluded via a restricted list. If still not categorized, a web scrape of the Google Play Store page using the library `Jsoup` extracts the category based on keywords like "social" or "games." If still unidentified, a keyword match on the application label with the domain lists is attempted. Applications failing all steps will default to the "other" category.

Figure 11 illustrates this categorization flow.

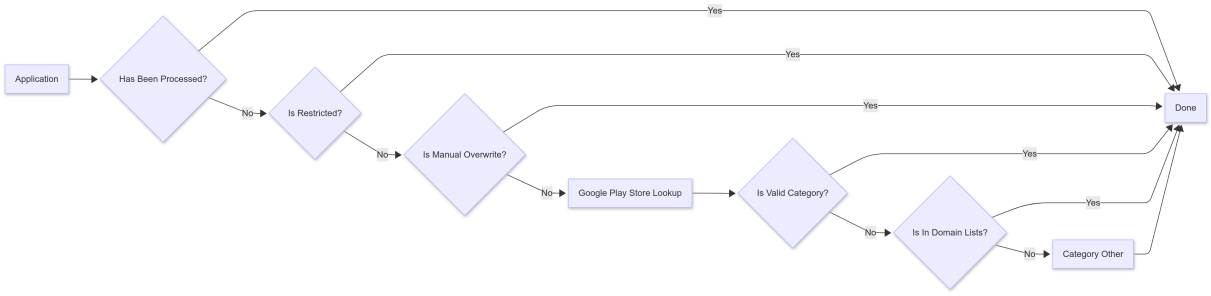


Figure 11: Visualization of Categorization Flow

7.3.7 Data Reporting

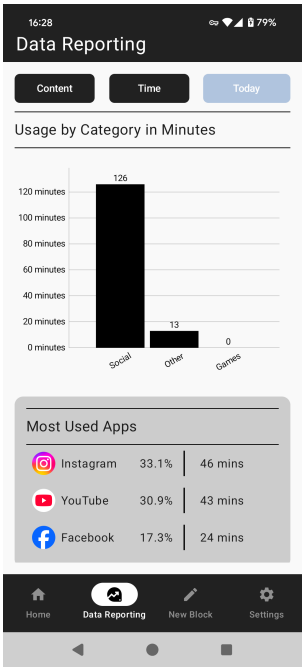
The data reporting feature gives users insight into their application usage. While it is currently a just simple overview, this feature is intended to be used for letting the user automatically generate blocks based on individual user habits. This would remove the need for users to manually identify their own patterns.

This feature was not part of the original MVP, but was agreed upon to be developed due to the future potential of the the feature. The requirements were minimal for the implementation. The requirements was kept simple for this project; the data reporting tool only needs to pull the usage data, categorize the data and visualize it with charts.

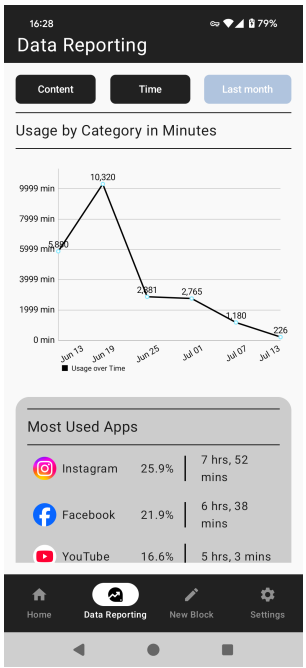
To visualize usage data, two types of charts were implemented: a bar chart and a line chart. The bar chart highlights what the user spends time on, grouped by categories. Applications that do not fit into a predefined category are grouped under the category `Other`. The bar chart, shown in figure 12a, displays usage across categories.

The line chart focuses on usage trends over time. As seen in figure 12b, it shows total application usage over the past month.

A drop-down button in the top right corner allows the user to change the time range between: Today, Last Week, Last Month, Last Year or Custom, where the user can set a start and end date.



(a) Bar chart Showing Application Usage Today



(b) Line chart Showing Application Usage Over Last Month

Figure 12: Example of Charts

Two additional components are included below the charts: `Most Used Apps` and `All Apps Per Category`. The `Most Used Apps` highlights the most time-consuming applications during the selected time range, helping users identify potential candidates for blocking.

The `All Apps Per Category` component shows all applications used for more than one minute in the selected period. These are grouped by category using accordions. An example of this is shown in figure 13.

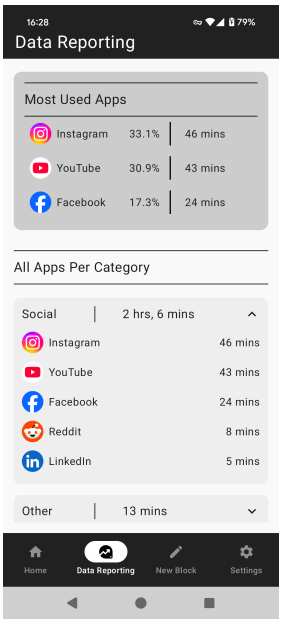


Figure 13: Accordion Showing Application Usage Today

7.3.8 Global Exception Handler

After the initial launch of the application, user feedback, both in person and through Sentry, showed that the application occasionally crashed. These unhandled crashes could cause serious issues, such as broken functionality or even loss of the blocks that the user had created. To address this issue, a global exception handler was implemented.

The handler starts a new dedicated thread that continuously listens for uncaught exceptions. If an unhandled error occurs, the handler recognizes it, allowing the main thread to crash safely while the listener thread takes over. This makes sure the application remains in a usable state and keeps user data.

To improve user experience and transparency, a generic error message is shown in an alert, as shown in figure 14.

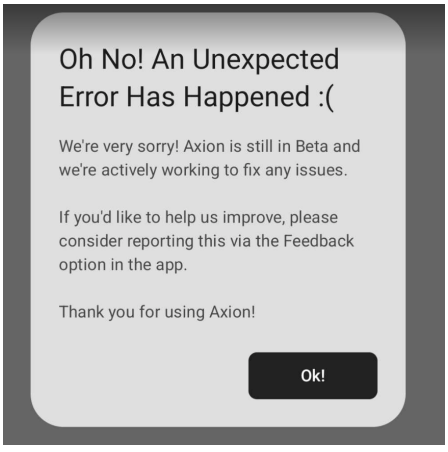


Figure 14: Alert for Generic Error Message

7.3.9 Caching

Caching is used to store persistent data locally on the device, improve performance and avoid repeated backend requests. Cached data remains until manually cleared by the user.

In the application, caching is mainly used for application categories, user credentials (email and access token) and analytics data. For example, categorizing installed applications, a performance heavy operation explained in section 7.3.6, only needs to be performed once. It is then stored for reuse.

The implementation uses the Android API `SharedPreferences`. A cache namespace is created called `internal_prefs`. Data is stored as structured key-value pairs. For example, categorized applications are saved under the key `processed_pkgs` as a JSON object, enabling fast retrieval. Figure 15 visualizes this.

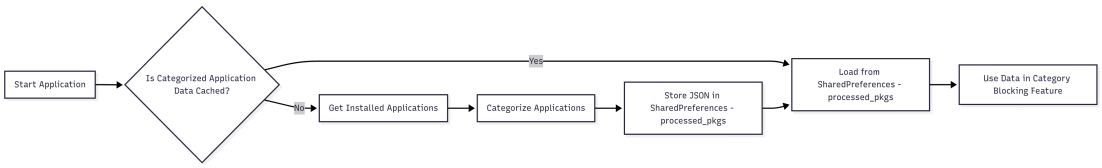


Figure 15: Caching of Application Categories

7.3.10 Deployment of Application

To deploy the application to Google Play Store and release a new update, we follow a step-by-step process.

7.3.10.1 Update What's New? File

What's New? is an industry adopted term. It is a standard page listing all application versions with update details. Each version highlights three areas: new features, improvements to existing features and bug fixes. Figure 16 shows the update alert for version 1.2.0, which appears the first time a user opens the applicaton after updating.

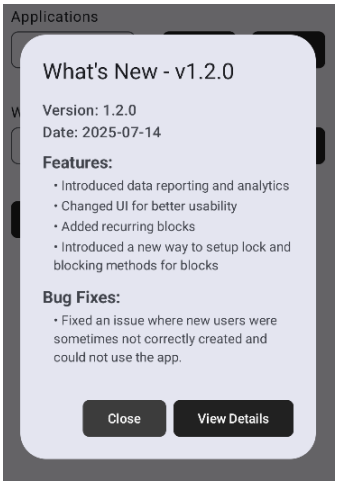


Figure 16: Alert That Shows What's New on Version 1.2.0

7.3.10.2 Version Update

Version numbers follow Semantic Versioning [43], a widely adopted system created by Tom Preston-Werner, co-founder of GitHub. Versions use the format X.Y.Z, where only one number increases per update. When X is increased, Y and Z are reset and when Y is incremented Z is reset:

- MAJOR (X) - When creating and changing features, that makes the application be backwards incompatible. This means that the user will have to relearn the application and understand it anew.
- MINOR (Y) - When creating and changing features, but the application is still backwards compatible. This means that the users still understand the application, but simply has new features or individual changes to learn.
- PATCH (Z) - When only making improvements and bug fixes on the application, where it is still backwards compatible.

For a detailed version history of the application, see appendix 10.3 Version Display. The alert in the application shows a shortened "What's New?" summary, while the appendix provides the full details.

7.3.10.3 Compiling of AAB

To release a new version on Google Play Store, an AAB (Android App Bundle) file is required. This package contains the compiled code and all necessary application information.

The AAB file is created using the build tool of Android Studio, which uses a keystore, to keep the signature hash that Google Play Store uses whenever the application is released or updated. The signature hash is used to identify our application.

7.3.10.4 Review & Release

After everything is done, the application is submitted for review. Google ensures all guidelines are met and requested permissions are justified to protect users from malicious applications. During this process, the countries where the update should be available are selected. Once approved, the update is released to the users.

7.3.11 Development & Production Environment

To ensure system stability and protect the functionality and data of the production environment, it is crucial to setup a separate development and production environment. By using the native editor of Android Studio, it is possible to start the application in a production or development environment. When developing new features, these are exclusively developed and tested in the development environment, to ensure stability in production.

To make this separation, the build configuration for the application has a set of two setups: **production** and **development**. Depending on which environment is compiled and used, it will load the corresponding configuration. This configuration makes sure that in the development environment, it does not send analytics data to Firebase, nor does it send error data to Sentry. To enable local development with Azure B2C, we use a development specific signature hash that is unique for each of our machine. This is different from the production signature hash, Google Play Store uses.

7.4 System Architecture - Backend

The backend operates as the core layer between the application and the database and other external systems. It is responsible for serving HTTP requests, managing authentication, handling business logic and ensuring that data consistency is preserved across devices. While currently implemented as a monolithic system, the architecture was intentionally designed with dependency inversion to allow future decomposition into micro-services as the system scales.

The system exposes a RESTful web API as the entry point for all application interactions. A REST (Representational State Transfer) API is a standardized architecture style for building web APIs. It uses stateless HTTP requests and maps operations to clearly defined resources, as defined by the original creator Roy Thomas Fielding in his dissertation [44]. These resources could be presented with `/accounts`, `/users` or `/blocklists`, with predictable behavior based on the HTTP methods (GET, POST, PUT, DELETE). The REST structure is widely adopted and understood among developers. Being easy to use with strong conventions and wide support in different frameworks and languages, the REST API structure decreases onboarding time of new developers.

The backend itself is structured using a layered architecture of **controller** → **service** → **repository**. Controllers expose API endpoints and parse requests. Services contains the business logic and repositories are responsible for direct communication with the database layer. This separation of concerns makes for better testing, maintainability and easier debugging.

7.4.1 Monolithic Structure versus Micro-services

The entire backend currently has a single entry point called `ApiService`. This is due to it being build as a monolithic system. The `ApiService` exclusively defines the REST API and leaves request handling to internal services. For example, a POST request to `/accounts` would trigger the `postAccount()` function within the `ApiService`, which then calls the corresponding function in the `AccountService` namespace. That service in turn uses the appropriate repository to save the data. This sets up a direct flow for later replacing internal function calls with HTTP requests to the different services, when changing from monolithic to micro-service architecture.

Each namespace can be split into its own deployable micro-service. Figure 17 illustrates the current monolithic design, where the `API Controller` routes all logic internally. As shown in figure 18, these internal calls can be swapped for HTTP requests.

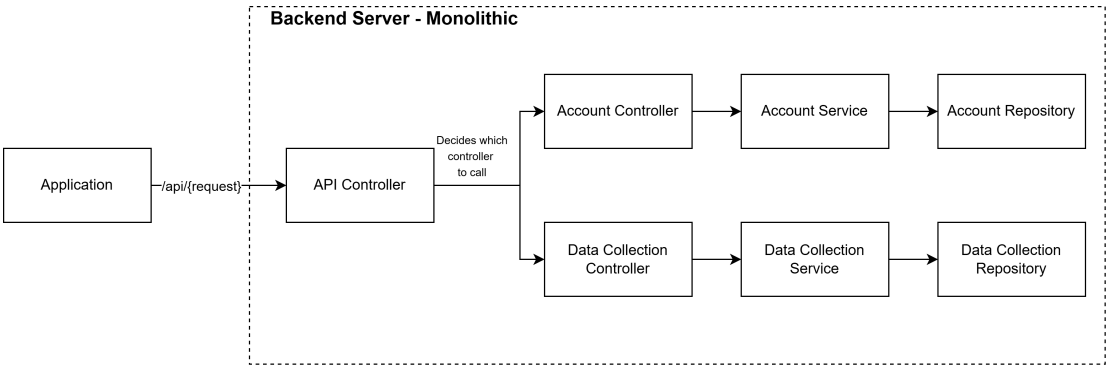


Figure 17: Monolithic design of backend

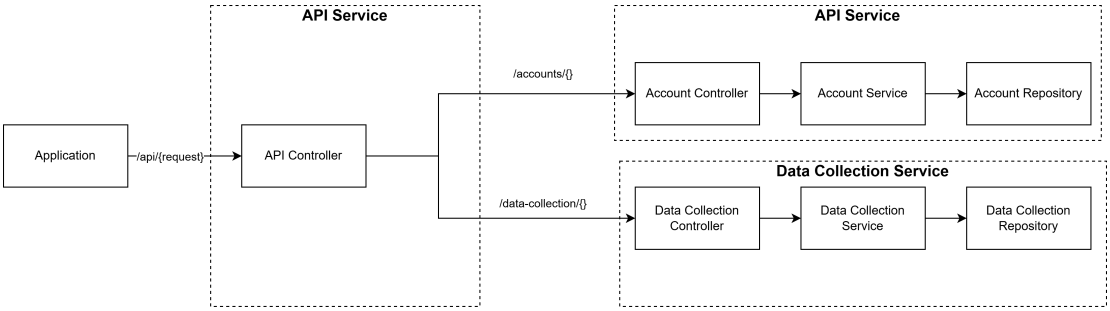


Figure 18: Micro-service design of backend

While micro-service architecture offers better error handling, isolation and horizontal scaling, they also introduce operational complexity, such as individual deployments, individual server handling and a need for secure communications between the services [45]. Given the current computational needs where most of the logic and computation takes place on the application, it is more efficient to run the backend as a monolithic architecture. By maintaining dependency inversion for each of the services within the code base, the backend remains scalable and ready for decomposition when the traffic load increases.

7.4.2 REST API

A RESTful API provides a stateless, resource oriented interface over HTTP, using standard methods (POST, GET, PUT, DELETE) to create, retrieve, update and delete resources. In our implementation, each route corresponds to a resource. The backend does not retain any client state between requests. The rest API is implemented using the framework ASP.NET, to hosts the endpoints. It listens for incoming HTTP requests and return the appropriate HTTP status codes and potential JSON responses.

Endpoints
All available endpoints, HTTP methods and descriptions are listed in appendix 10.1 Endpoints.

HTTP Status Codes
We follow the standard status ranges of HTTP status codes and their meanings:

- **2xx** (Success): 200–299
- **4xx** (Client Error): 400–499
- **5xx** (Server Error): 500–599

To see what HTTP statuses the backend can currently return and their meanings, refer to appendix 10.13 Backend HTTP Statuses.

HTTP Methods
Each endpoint uses one of the following standardized HTTP methods:

- **POST** – create a new resource
- **PUT** – update an existing resource
- **GET** – retrieve a resource
- **DELETE** – remove a resource

Authentication
All endpoints require a valid access token, before a request can be made. Should this access

token be invalid or not provided, the server will respond with status 401 and deny the request. Token management and authentication is covered in section 7.4.3 Account Management.

DTO Validation
Incoming request bodies are de-serialized into Data Transfer Objects (DTO). The DTO is an object that can specify which fields are required in the object. If any required fields are missing, the serialization will throw an error, resulting in a status 400. The response also details what fields are missing.

Below on figure 19, the flow of de-serializing an incoming request for creating a user, processing it, saving it to the database and returning the ID is visualized.

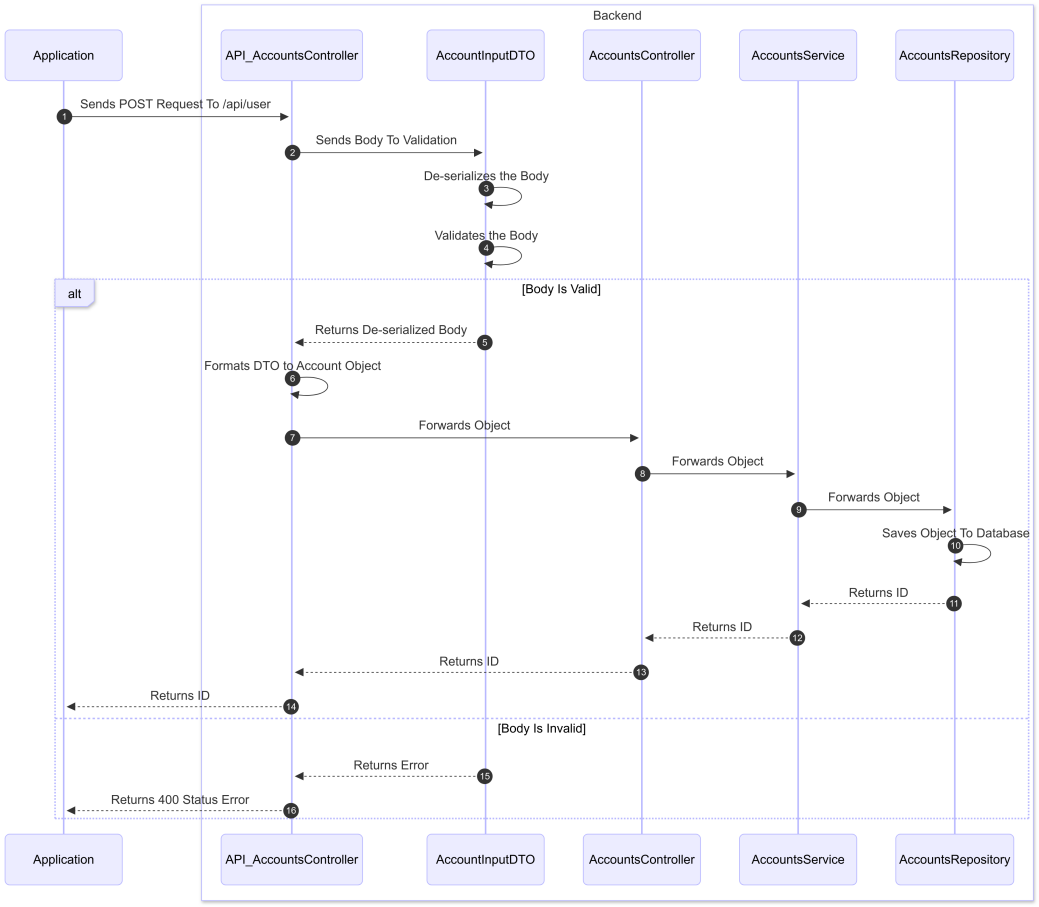


Figure 19: Creating User Flow With DTO Validation

Error Handling
Unhandled exceptions in controllers or middleware are caught by a global exception handler, which returns a status 500 (Internal Server Error) response. This prevents the application from crashing due to unexpected errors while still informing clients of a server error.

Rate Limiting
To mitigate abusive behavior and reduce DOS (Denial of Server) attack risk, we apply per IP rate limiting. Currently, it is limited to 10 requests per 10 seconds per endpoint. While this helps against DOS attacks, as they come from a single to few IPs, true DDOS (Distributed

Denial of Service) attack protection requires advanced request monitoring and analysis. It is therefore marked for future implementations.

7.4.3 Account Management

When implementing account management, several security concerns must be addressed, from safely storing credentials to ensuring that only authenticated users gain access. The following provides an overview of the complexities and outlines potential approaches to address them.

Password Storing – Confidentiality
Securely storing passwords is necessary to keep online users safe. Rather than storing passwords in plain text, they can be hashed with a one-way algorithm implemented with salt and pepper (values added to prevent certain security attacks). This ensures the original passwords remain encrypted if the database is compromised.

Authentication – Authenticity
Verifying that each request comes from the claimed user is also critical. Authentication establishes identity, typically through bearer tokens issued after login transmitted over HTTPS. Proper token management with principles like token families and token rotation ensures that users can only access their own data and that stolen tokens have limited impact.

Azure B2C
Due to the complexity of security, account management was outsourced to Azure Active Directory B2C [46]. Azure B2C handles registration, password storage, multi-factor authentication and token validation. We integrated a middleware layer in the backend to verify B2C tokens. This ensured valid access, reduced development time and relied on the compliance guarantees from Microsoft. Figure 20 shows how B2C validates tokens. More details are covered in section 7.5.2 Azure B2C Directory.

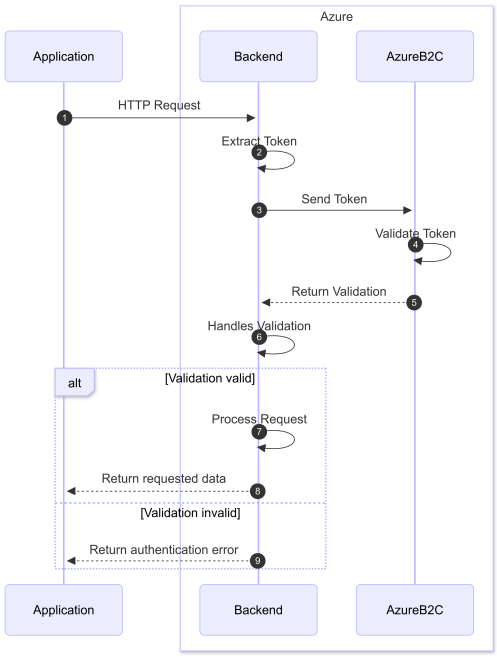


Figure 20: Azure B2C and Backend Interaction With Token Validation

7.4.4 Development & Production Environment

To ensure system stability and protect production data, the backend can change between development and production environments using an environment variable called `ASPNETCORE_ENVIRONMENT`. When the server starts, it tries to retrieve this variable. If it is set to **Production**, then it starts the server in production mode, otherwise it starts the server in development mode. This separation is crucial to prevent accidental changes, service interruptions or data deletion, when developing new features. New features and changes are tested only in development to ensure they are safe and functional before being deployed.

Two configuration files are used for the two environments: `appsettings.json` for developments settings and `appsettings.Production.json` for production specific settings. These files control environment dependent settings such as database connections, external service keys, etc.

In production, the system connects to the remote database in Azure, initializes error tracking via Sentry and can fetch passwords from Azure Key Vault. These services are, by design, disabled in development to avoid unnecessary risks and disable the sending of data that can skew the production data and analysis.

During development, a MySQL database runs on our local devices. This provides a realistic and isolated environment, allowing us to test features without affecting production data.

7.4.5 Unit Testing

Unit testing involves writing automated tests for individual units of code, typically functions or methods, to ensure they behave as expected. Unlike manual testing, unit tests are repeatable, fast and can be run automatically on each change, making them both reliable and efficient.

All functions and branches across the backend are covered by unit tests to ensure that logic is correct and that edge cases are handled. This helps catch bugs and errors early in the development process. They make sure, that already existing functionality still works as intended.

7.5 System Architecture - Azure

This section will describe the setup of the Azure architecture. Azure is responsible for hosting the backend, the database, email service and the account management system. These are all crucial elements for the system to function correctly.

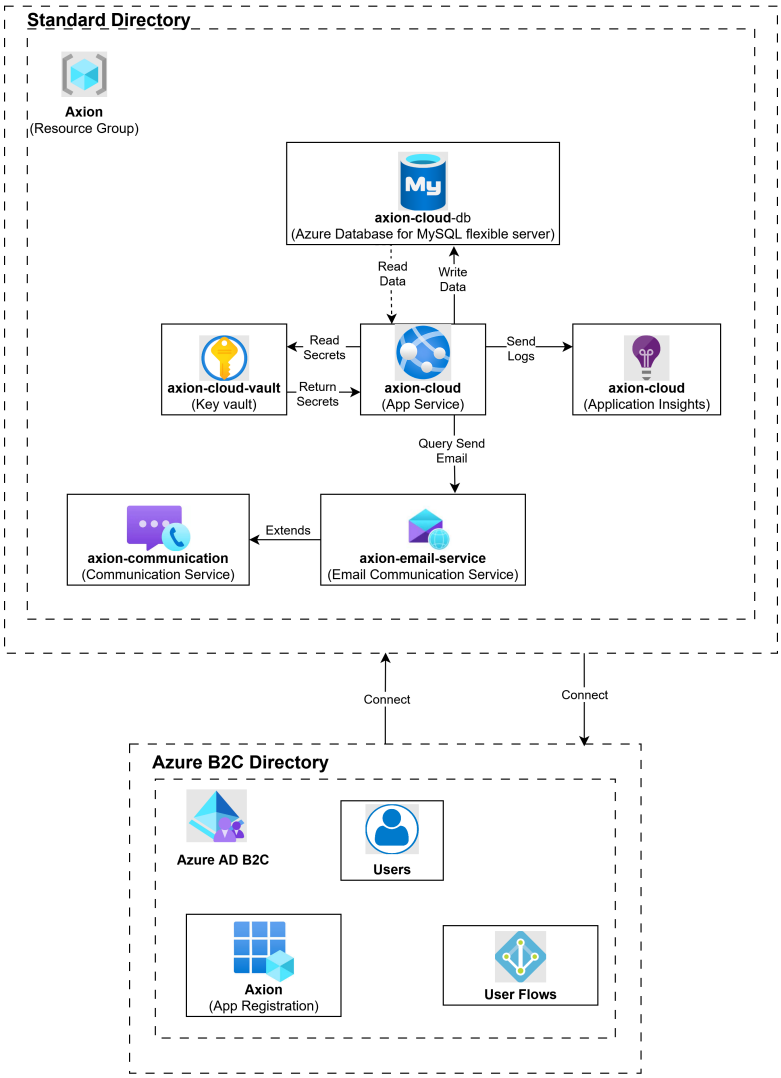


Figure 21: Overview of Azure Architecture

As shown in figure 21, the Azure architecture is divided into two directories: the standard directory and the Azure B2C directory. The standard directory manages all backend functionality, while the Azure B2C directory handles account management. This separation was intentional. Hosting Azure B2C within the same directory as the backend could potentially allow users who create an account to gain access to backend resources. This aligns with the recommended security and management best practices from Microsoft [47].

The different components are hosted on servers located in Europe. They are hosted in either *Germany West Central* region or *North Europe* region. This is to ensure that we can more easily follow GDPR, the General Data Protection Regulation. If we were to host the service outside of the European Union, we would have to follow the rules of transferring data outside of the EU [23].

7.5.1 Standard Directory

The Standard Directory hosts the backend services and resources. It organizes and manages these components securely and efficiently within Azure.

7.5.1.1 Resource Group

The resource group is created to group all the services. The resource group is defined to allow access between the different services, without having to set up an automated access flow. This allows all the services to freely connect with each other.

7.5.1.2 App Service

The App Service hosts the backend for the application. It runs the C# codebase on a Linux environment. The App Service is part of a Basic (B1) service plan, which provides a single instance and sufficient performance for development and small production workloads. As the user base scales, it would make sense to scale up to more instances for horizontal scaling, or upgrade to a service plan with more computational power, for vertical scaling.

To monitor system health, a built-in health check is configured on the App Service. The backend exposes a simple endpoint, `/api/health`, which responds with the word *UP*. This allows the App Service to verify that the system is operational. If the endpoint returns an error or no response, Azure automatically marks the issue and notifies us.

Application Insights is integrated too, providing detailed logging, performance metrics and diagnostics. More on this is covered in section 7.5.1.4 Application Insights.

7.5.1.3 Azure Database

The database is hosted using *Azure Database for MySQL Flexible Server*, a relational database service. It runs on a cost-effective compute tier with automatic storage scaling enabled, providing enough capacity for current usage and room to grow as needed. It handles daily automatic backups. These backups are retained for 7 days.

Security is ensured through SQL authentication with a strong password, with access limited to internal services in Azure to prevent public exposure.

7.5.1.4 Application Insights

Application insights provides real-time monitoring of the backend performance. It tracks individual endpoint response times, logs errors and collects general performance metrics. This helps quickly identify issues and ensures the server runs smoothly. While this has been ideal for some very specific errors, Sentry has proved the overall better monitoring tool for this project.

7.5.1.5 Communication & Email Communication Service

We use Azure Communication Services to handle email sending. It is linked to our domain `axionapp.io`, and sends all emails from `DoNotReply@axionapp.io`. This improves trust and credibility when users receive emails. The backend can trigger these emails when needed, such as during email change and account deletion. It can also be further expanded in the future to, for example, automatically send newsletters.

7.5.1.6 Key Vault - axion-cloud-vault

To keep our application secure, all sensitive data and keys are stored in the Key Vault on Azure. This is the recommended and most secure approach, as it prevents passwords from being exposed in the code or through configuration files. The backend connects securely to the Key Vault over the internal network on Azure. Since the App Service and Key Vault is in the same resource

group, the backend only needs the URL of the vault to access the passwords. It is not needed to publicly expose the key vault.

The vault currently holds five passwords:

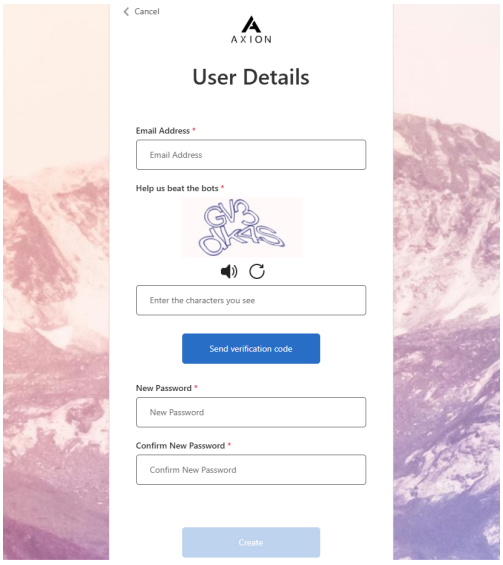
- `AzureClientSecret` – Used for authenticating with Azure B2C.
- `AzureEmailConnectionString` – Used to connect to the email service.
- `DBPassword` – Used to connect securely to the MySQL database.
- `SentryDSN` – Used to connect to Sentry.
- `GoogleSheetsCredentials` – Used to store user feedback in Google Sheets.

7.5.2 Azure B2C Directory

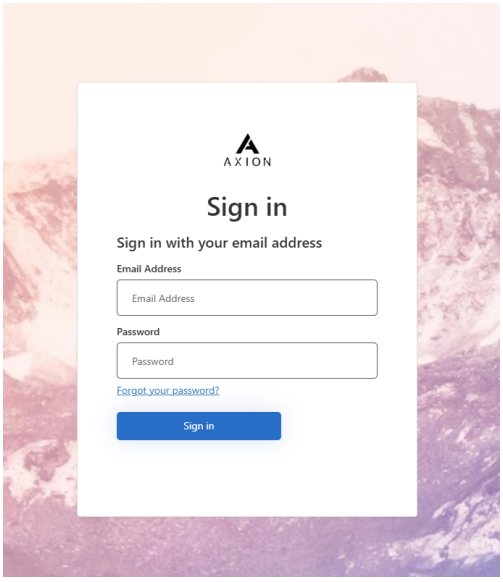
Azure Active Directory B2C is an identity management service that handles user authentication and account management. It manages password security, handles login and registration flows, securely stores credentials, rotates access, refresh tokens and follows best practices in identity protection. It also handles the initial verification of emails and handles the resetting of passwords. This removes a large security responsibility from our backend and significantly reduces development time, allowing us to focus on developing features instead.

The backend connects to Azure B2C through an *App Registration* called *Axion*. The app registration is configured to allow secure communication between B2C and our backend using a client secret, which is stored safely in our Key Vault under the name `AzureClientSecret`.

We have configured two user flows in Azure B2C: *sign up* and *sign in*, shown in figure 22a and 22b. These are website based and open as browser pages when interacted with inside the application. The user leaves our application and opens a browser. A downside of using Azure B2C is therefore that these flows are not fully native to our application and generally offer limited customization of its visual elements. It also lacks an easy extension of the flows, should other values be needed to collect on sign up.



(a) Sign Up Flow



(b) Sing In Flow

7.6 System Architecture - Database

The database serves as the central storage component of the system, handling all persistent data related to users and other essential entities, that need to be accessed across devices and user sessions. A relational database model was chosen due to its built-in functionality for enforcing data integrity. Integrity is upheld by specifying what should happen to data when references change or get deleted. This ensures consistent and reliable data management across the database.

Minimizing data redundancy and maintaining integrity are key goals in database design. Redundancy can lead to inconsistencies and increased storage costs, while integrity ensures that the data remains true. To address these concerns, the database design follows the principles of normalization, reducing duplications and ensuring data is organized logically.

This section outlines the conceptual design of the database, details the normalization process applied to its schema and describes the implementation using Entity Framework within `.NET`.

7.6.1 Conceptual Design

The database conceptual design is an abstract representation of the database, that captures the requirements for the structure of data.

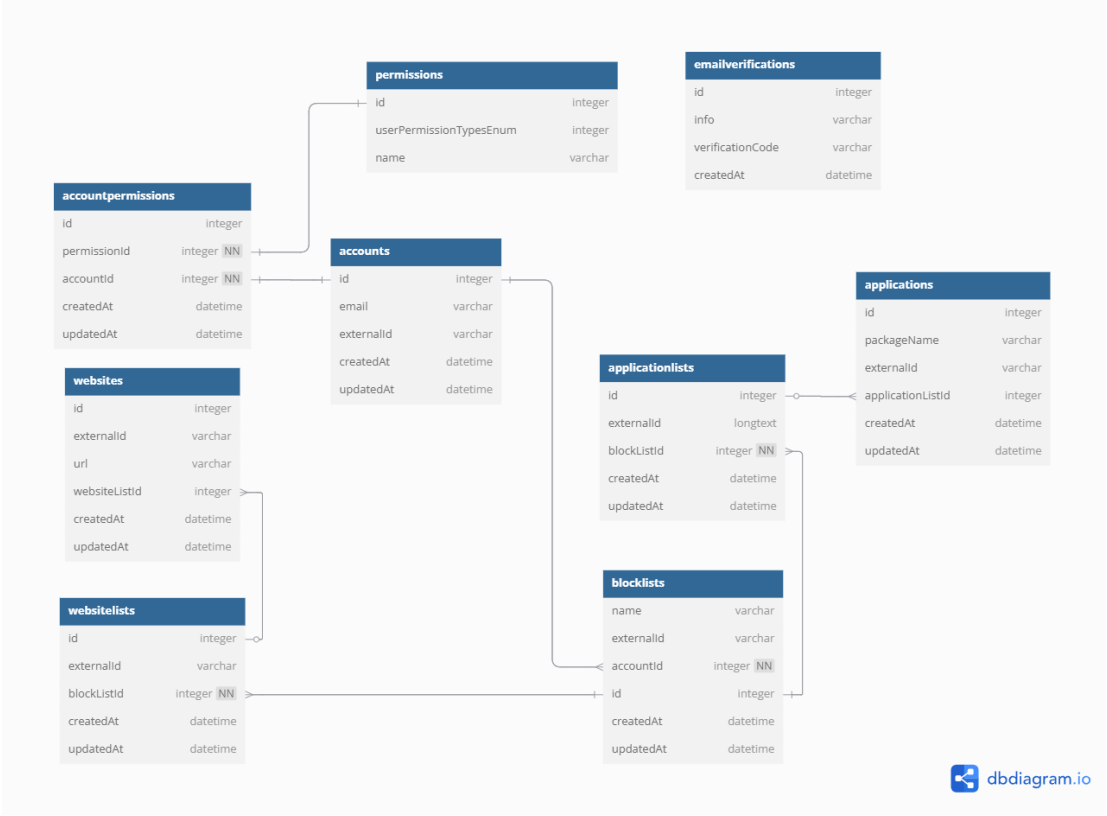


Figure 23: Conceptual Design of the Database

Accounts Contains users and their information, such as the external id from Azure and their email. This table can be expanded further with more user information in the future.

AccountPermissions Decomposed from `Accounts` to isolate permissions for easier maintenance. References the primary key `id` in the `Accounts` table via `accountId` with a one-to-one relation, as one account can only have one permission. References the primary key `id` in the `Permissions` table via `permissionId` with a one-to-one relation, as one permission can only be referenced to one permission type.

Permissions Decoupled into its own table to maintain a reference between `id`, the permission type and its name, improving maintainability and isolating permission updates.

BlockLists Contains all block lists associated with a user. Contains information such as `id`, `name` and a reference to the account that owns it via `accountId`. This references the primary key `id` in the `Accounts` table, by a one-to-many relation, as one account can have multiple block lists, but each block list belongs to a single account.

ApplicationLists Contains application lists associated with a specific block list. Serves as a link between `BlockLists` and `Applications`, designed to support future extensions. Contains an `id`, `extnernalId` and a reference to `blockListId`, referencing the primary key `id` in the `BlockLists` table. It follows a one-to-many relation, as one block list can have multiple application lists, while each application list belongs to a single block list.

Applications Contains applications that are part of an application list. Contains information such as `id`, `pakcagename`, `externalId` and a reference to `applicationListId`. This references the primary key `id` in the `ApplicationLists` table. It follows a zero-to-many relation, as application lists can have zero or many applications in it, but when an application entry is stored, it always needs to reference an application list.

WebsiteLists This table is the same as the `ApplicationLists` table in its structure and foreign key references.

Websites This table is the same as the `Applications` table in its structure and foreign key references. It simply holds different information, such as a `url` instead of a `packageName`.

EmailVerifications This table handles ongoing email verifications. It is intentionally decoupled from the rest of the database to remain independent of existing users. This allows for greater flexibility, such as supporting email verification before account creation or email recovery. It includes fields like `id`, `note` and `verificationCode`, where `note` can temporarily store the intended email address of a user without requiring a user reference.

To ensure the verification data is not stored forever, the `EmailVerifications` table uses a time-based cleanup event. A SQL event, `DeleteOldEmailVerifications`, is scheduled to run every five minutes and deletes any entries older than 10 minutes. This is implemented using the MySQL `EVENT` scheduler feature.

7.6.2 Normalization

After defining the database schemas, we verify that each table conforms to the first, second and third normal forms (1NF, 2NF, 3NF). Normalization minimizes redundancy and issues with updates by decomposing tables into smaller tables linked via foreign keys. Normalization ensures any insert, update or delete only affects one reference and maintains overall consistency [48].

Table 5: Normal Forms: Definitions and Examples

Form	Definition
1NF	All attributes depend on the primary key, each column holds a single, atomic value.
2NF	Meets 1NF. Every non key attribute depends on the entire (composite) key. Keys in single column automatically satisfies 2NF.
3NF	Meets 2NF. All non key attributes must depend on the (composite) key and nothing but the key.

All tables use only atomic columns and therefore satisfy 1NF. 2NF is automatically satisfied since each table uses a single column primary key. Every table complies with 3NF, as all non-key attributes depend directly and completely on the system generated keys, with no transitive dependencies. System generated keys is a reference to the `id` primary keys. By relying on system generated keys, partial or indirect attribute dependencies are avoided, ensuring full normalization and consistent data integrity throughout the database schema.

7.6.3 Implementation

Conceptualizing the database schema is one thing, implementing it in two environments and keeping them in sync is another. We run MySQL locally and on Azure. This requires all schema changes to synchronize, both with the local databases and Azure. To prevent version inconsistencies, we chose for the backend to handle migrations directly.

Manual SQL scripts for updating offer greater control, but are separated from the backend and require a custom migration system, as Azure does not provide it as an option. We instead chose a code-first approach using the `Entity Framework` (EF). EF integrates with the backend natively through `.NET`, which manages the migrations automatically. While EF lacks support for complex SQL setups, our simple schema made this compromise feasible.

EF ensures schema consistency, automates migrations when running the backend and enables simple rollbacks. The full update workflow is shown in Figure 24.

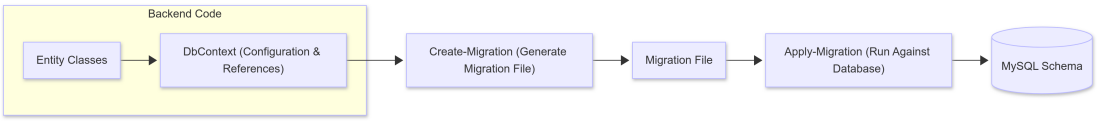


Figure 24: Workflow of Updating the Database Using Entity Framework

7.7 Website

The website currently functions as a simple landing page. It can be found on axionapp.io.

We purchased the domain `axionapp.io`, as .com was either unavailable or significantly more expensive. The ".io" domain is widely accepted in the tech industry and adding "app" aligns with common naming conventions among startups.

For hosting, we chose Cloud86 [49], a Netherlands-based provider with top performance scores for European users. This aligns with our initial marketing focus in Denmark. Other providers, like the Danish simply.com, were considered as well.

The site was built using WordPress, with a design focused on a sleek, metallic aesthetic using dark tones and image overlays.

We paid for a 3-year subscription, the first major expense besides the cost of incorporating. After purchase, the company balance was at around 15.000 DKK.

Website analytics have not yet been implemented, as efforts have been focused on the application. Once the website becomes more central, Google Analytics will be added to support optimization.

7.8 Sentry

To monitor both the application and backend effectively, Sentry was integrated for real-time error monitoring. It captures both handled and unhandled errors, providing valuable context, such as logs and showing user actions leading up to the issue. This made it significantly easier to reproduce and debug issues.

Sentry automatically assigns severity levels to errors based on both their impact and frequency. For example, a minor bug occurring for every user daily is marked as severe due to its frequency, even though the bug might not disturb the user. A critical crash experienced by a single user however, might also be severe due to its impact, even if only a single user experienced it. This prioritization helped us determine what we should focus on.

Each error log includes device information, Android version and the application version, allowing us to determine whether an issue was fixed in a later release. If an error shows up again in a version where it was marked as resolved, Sentry flags it with the highest severity to indicate a recurrent error.

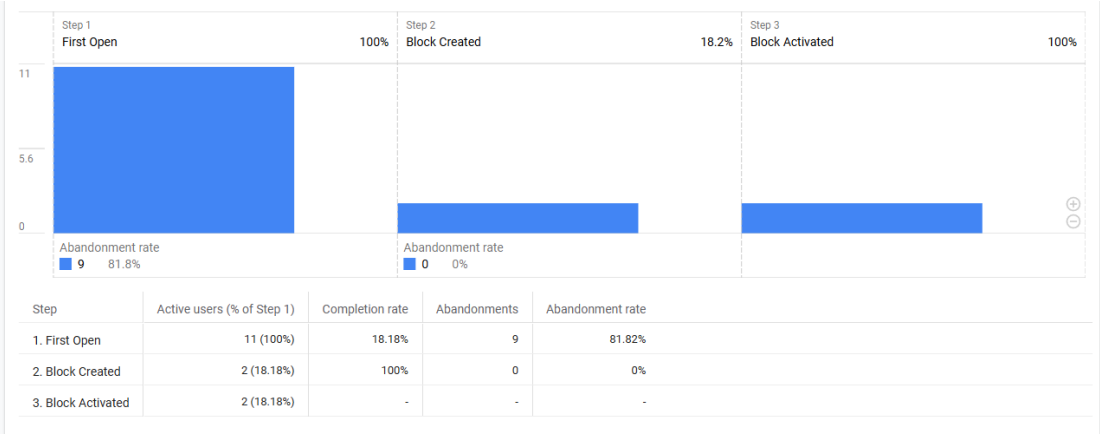
To keep the application and backend errors separate, the Sentry project was split in two. This separation made it easier to identify whether an error originated from the application or the backend, for better isolation.

7.9 Firebase Analytics & BigQuery

To effectively monitor and measure engagement that were explained in section 6.7 Key Performance Indicators, we used Firebase Analytics to capture events and get a quick overview over them. For an in-depth analysis of the data, we used Google BigQuery.

This section explains how user engagements can be measured and visualized. Note, however, that meaningful data cannot be shown due to the current lack of an active user base.

We used Firebase Analytics to capture events. Firebase automatically tracks certain events such as first application open, application launches afterwards, session duration and uninstalls. For more specific insights into user behavior, custom events are required. To get a full overview of all the custom events that we are currently tracking, refer to the appendix 10.2 Firebase Events. As an overview, the custom events are events that we choose. This can for example be to send an event when a block is activated, when the user creates an account, etc. Firebase Analytics can be used to get a quick overview and visualize certain flows, through funnel exploration. An example is shown on figure 25.



Step	Active users (% of Step 1)	Completion rate	Abandonments	Abandonment rate
1. First Open	11 (100%)	18.18%	9	81.82%
2. Block Created	2 (18.18%)	100%	0	0%
3. Block Activated	2 (18.18%)	-	-	-

Figure 25: Users Who Opened the Application After Installation, Also Created and Activated a Block

To measure the user engagement we use Google BigQuery. By setting up an automated export of data from Firebase to BigQuery, we have now set up measurement queries. BigQuery is a powerful tool, that through the use of the programming language SQL, can perform extensive and complex queries. For now, we have set up the following queries in BigQuery that can measure the following user engagements:

- Active Users
- Long Term Users
- Churn Rate
- User Growth Daily
- Long Term User Growth Daily

The actual queries and how they work is explained in the appendix 10.5 BigQuery User Engagement Measurements. It is important to note that, due to the limited user base at this stage of the project, these queries have not been thoroughly tested with a large amount of data. As such, they may require adjustments or might encounter errors once more data becomes available.

7.10 Version Control & CI/CD

Version control and CI/CD (Continuous Integration and Continuous Deployment) are core components of modern software development. CI/CD automates tasks such as building, testing and deploying systems. These tasks are often repetitive, but essential. To fully benefit from CI/CD, a version control system must be used.

Version control is used to track changes in the codebase over time. Each time code is committed, a version of the project is saved. This allows developers to revert to earlier states of the systems if errors occur. In this project, Git was used for version control, with GitHub serving as the third-party repository platform. GitHub provides additional features such as pull requests and visual tools for managing branches and commits.

A branching model was adopted to ensure that the main branch remains stable, tested and deployable at all times, as described in section 6.11 Development Guidelines & Principles. New features, bug fixes and improvements were developed on separate branches derived from the main branch. Once completed, tested, and validated, these branches were merged back into the

main branch via pull requests. This practice allowed us to isolate ongoing development from the main branch, which were always ready for deployment.

With version control, CI/CD pipelines were integrated to automate testing, validation and deployment processes. The testing and validation pipelines were configured to run on each push to the repository. When new commits were pushed to the application or backend, the pipelines automatically checked that the code compiled and all tests passed.

In addition to validation, CI/CD was also used for deployment. GitHub Actions was configured to build and deploy the backend to Azure. This allowed us to trigger deployment workflows directly from the GitHub Actions tab on the website. The workflow handled packaging the backend, uploading it and deploying them to Azure, making the process much simpler.

7.11 Challenges and Unimplemented Features

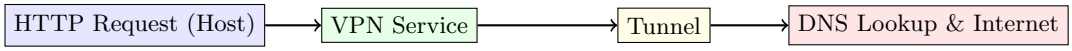
This section will go over some challenges faced during development. It also describes features which we investigated and attempted to implement, but failed to complete for various reasons.

7.11.1 Internet Slowdown

The Concept
The Internet Slowdown feature was intended as our first premium feature. Instead of fully blocking applications, it would allow slowing down the network connection of individual applications by a set amount when opened. This would allow access to the application, but make teh experience of scrolling less stimulating.

What Went Wrong?
While the idea was good, its implementation proved not to be feasible due to security issues and a lack of network expertise. We explored two implementation strategies: a VPN service and a local proxy server.

Using the built-in Android `VpnService`, we would establish a TUN (tunnel interface) to intercept raw IP packets, delay them, and then forward them. Figure 7.11.1 shows the flow.



In practice, forwarding each packet requires parsing IP/TCP/UDP headers, resolving DNS forwarding and lookups, managing sockets, reconstructing packets and writing responses back into the TUN interface. Existing libraries (`tun2socks-android` [50], `sockstun` [51], `VPNclient-engine` [52]) were outdated, UDP packets only, or required external servers, making them all unusable for our particular case.

As a fallback solution, we considered implementing a local HTTP proxy server to introduce delays without having to do packet level handling. This also introduced complexity due to HTTPS traffic. This is because HTTPS is encrypted with TLS, masking the packet content and complete destination. It would be necessary to decrypt and re-encrypt packets every time they were forwarded. This would introduce security issues and greater technical complexity, as exposing the packet content and complete destination, can easily introduce confidentiality and integrity risks [53], through the use of DNS tunneling attacks.

These things considered, we determined the feature was not feasible within the scope of the project. It requires the ability of an individual who specialize in network protocols and configuration building. It was therefore not implemented.

7.11.2 Category Blocking With DNS

The Concept
The Category Blocking With DNS was the initial idea for the category blocking, before it was changed to a predefined list. The idea was to use a DNS from a company, where all internet traffic would be routed through. The companies looked at was NextDNS [54] and OpenDNS [55]. OpenDNS use scraping technologies and artificial intelligence to categorize every single webpage on the internet. NextDNS takes a less complex route, by using artificial intelligence to determine the category of the website from the content that it has. The idea was to use either of the two DNS setups in our application to attempt to correctly identify what category a certain webpage or application belongs to for improved category blocking.

What Went Wrong?
The same technical and security challenges described in the section 7.11.1 Internet Slowdown applies here as well.

RESULTS & FUTURE VISION

"If I had asked people what they wanted, they would have said 'faster horses'"
- Henry Ford, Founder of Ford Motor

8.1 Results

To get a better understanding of how the product feels to use, we encourage the reader to watch our application showcase video, which can be found on our YouTube channel, Axion App Showcase — Your Personal Attention Manager, or as the supplementary file `app-showcase.mp4`.

After six sprints and five completed iterations, this section evaluates the application in its current state. First, we compare the product to the MVP requirements and the product vision. Then we examine the usability of the application from an end user perspective.

In section 5.1 Minimum Viable Product, the MVP was defined as a minimal, functional system capable of applying simple blocks to user-selected applications and websites. It required a scalable account system to support future cross platform synchronization.

In section 4.4 Blue Ocean Strategy - Applied, the product vision outlined long-term goals beyond the MVP. These included advanced blocking features, automated block configuration with usage data and fully implemented cross-platform synchronization.

The current state of the application exceed the requirements of the MVP. Beside simple blocking, users have the option to block predefined categories of apps and website. Users can configure their blocks in detail with advanced blocking mechanisms, such as delay locks and recurring blocks, explained in section 7.3.5 Advanced Blocking. The MVP did not include requirements for data reporting, but iteration five supports visual data reports, explained in section 7.3.7 Data Reporting.

Iteration five made considerable progress towards the product vision, which will serve as a plan for future development8.2.

The current state of the codebases can be seen in the supplementary files `axion-android-application.zip` for the Android application codebase and `axion-cloud.zip` for the backend codebase.

8.1.1 End User Perspective

For a visual walkthrough, refer to appendix 10.11 Screenshots of the Application and 10.7 Application Tutorial Screens.

The user experience begins with an onboarding tutorial. This introduces core functionalities and asks the required permissions from the user. The users then proceed to sign in or register through Azure B2C. If they are creating a new account, the users must accept the terms of service and the privacy policy.

After the users have signed up/in, they can now navigate to the `New Block` screen to create a new block. This could for example be for studying. They can manually select applications or websites to block, and lists containing the selected items are accessible for review and editing. Users can also use the `Block Categories` function to block entire categories.

After creating a block, the user is returned to the home screen, where the new block appears. If the user activates a block and accessibility permissions are missing, an alert will show to request the permission. Once enabled, users can customize locking, blocking and access settings in a large dialog box. These settings are saved and determine the behavior of the block after turning it on. Blocks can also be edited or deleted if they are not locked.

At any time, users can access the data reporting page for insights into their phone usage. This includes category specific usage, trends over time and an overview of the most used apps. Users can with this information identify areas for improvement to address with the application.

8.2 Future Vision

The product requires more development to reach the completion of the product vision outlined in Chapter 4. The future timeplan of product development can be seen on the Gantt-Diagram on figure 26.

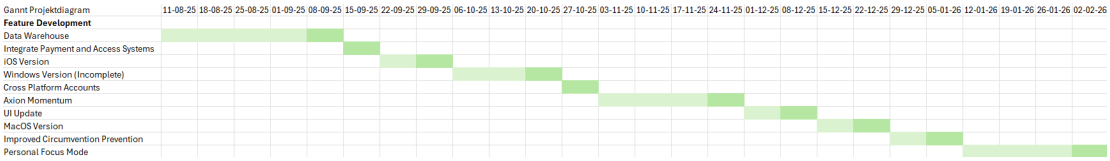


Figure 26: Gantt Timeline for Continued Development

The first feature is a custom built database and analytics system. Although this represents a substantial task, having the ability to perform granular measurements is essential for enhancing the user experience and take business and development decisions. The second feature is the integration of a payment system to enable monetization through subscriptions. The third feature is the development of an iOS version of the application. This is a significant step, as it provides access to the largest mobile market in Denmark. Although the application will then support both iOS and Android platforms, cross-platform synchronization is not considered essential at this stage, as we assume few users will operate both systems at the same time.

The fourth feature is the development of a Windows version of the application. Due to fundamental platform differences, this requires a complete redevelopment. As a result, the initial Windows release will focus only on implementing blocking functionality, excluding data reporting and cross-platform synchronization. The fifth feature is the implementation of cross-platform synchronization. With the application available on iOS, Android and Windows, ensuring synchronization between the platforms becomes a priority. Synchronization of block configurations and user accounts across platforms will be the primary focus.

The sixth feature is *Axion Momentum*, a conceptual idea for automating block creation based on user habit analysis. It involves collecting detailed usage data and utilizing behavioral psychology to suggest personalized blocking configurations. The seventh feature is a comprehensive overhaul of the user interface of the application. The goal is to commission a professional designer to produce concept designs, applying principles from user experience engineering to enhance the overall usability.

The eight feature is the development of a MacOS version of the application. The development process resembles the Windows version, and upon completion, cross-platform synchronization will already be implemented. The ninth feature is improving circumvention prevention. This involves addressing scenarios, where users can attempt to bypass active blocks, such as disabling the accessibility service. Any method that enables the bypass of an active block should be identified and mitigated.

The tenth and final planned feature for the product vision is a conceptual feature called the Personal Focus Mode. Unlike standard blocking functionality, this mode aims to minimize distractions more comprehensively by disabling notifications, messages, emails and other interruptive elements. It may include the altering of the user interface, such as enabling grey scale or relocating distracting applications. On desktop platforms, it is intended to minimize distracting windows, hide the taskbar and silence system notifications. Implementing this feature will require a re-imagining of the structure of the application, as it diverges significantly from the current core mechanisms.

EPILOGUE

"The greatest enemy of knowledge is not ignorance, it is the illusion of knowledge"
- Stephen Hawking, Theoretical Physicist and Cosmologist

9.1 Discussion

While problematic technology use is a multi-faceted problem not easily solved on a society-wide level, individuals could be empowered to solve it for themselves if they had better tools. The development of a new solution to be distributed directly to individuals has been started with this project. Success in further development of the product could help people struggling with compulsive technology usage.

The project was executed using wide range of disciplines; academic research, business strategy, computer engineering, startup methodology and agile planning. It has been difficult to tie the different parts together into a coherent project that serves a single objective. Nonetheless, the multi-disciplinary nature of the project has proven an interesting and complex challenge.

9.1.1 Problem Exploration

Finding sources to support the claims made in chapter 3 presented several challenges. As technologies of digital entertainment has evolved quickly, especially in the last ten years, the novelty of scientific articles carries importance. Older findings were not used if the subject they treated had changed significantly since. This was especially relevant with regards to articles treating the subject of potential plastic changes to the brain stemming from technology use, as the nature of entertainment commonly found online has become more intense in the last few years with the advent of short-form video content.

As argued in chapter 3, there is widespread awareness of the issue, but a lack of clear and simple solutions. Unless a major shift in the regulation of technology companies is eminent, our clear understanding is that the problem will continue to grow. Technology companies will engineer better algorithms, more stimulating experiences and new methods of delivering content through advanced user-interfaces. This will drive an increasing need for new solutions and innovation to counteract the unregulated progress in digital entertainment technology.

9.1.2 Solution Innovation

The Blue Ocean Strategy [1] was used with the intention of identifying openings in the market. It is a business strategy method, and thus rely on the ability of the team using the method to correctly identify which aspects of a product or service to focus on. When the Blue Ocean Strategy is used by large corporations, it is often accompanied by a considerable amount of market research, data and customer knowledge. We did not have the resources to do extensive information gathering work, and thus the conclusions drawn in chapter 4 are entirely based on our ability to apply the strategy and estimate certain qualities of competitors and customers accordingly.

While we had the goal of solving the problem presented in chapter 3 since the beginning, the solution was and will continue to be the result of a continuous process of innovation. The success of the current solution both technically and commercially depends on assumptions which are continuously validated or invalidated. This constantly shapes the direction in which we will steer innovation to achieve a better solution to the problem.

9.1.3 Development

The development strategy was centered around completing the MVP quickly. This was due to the belief that launching quickly would allow us to learn faster from user feedback. We found some errors in the application, and we incorporated the collected user feedback into the development; the tutorial, the placement of the accessibility service requests, and the navigation bar was all shaped by user feedback. While this gave us an advantage in development decision making, releasing an MVP quickly caused the application to lose the initial user base we got

after launch. As we had an undiscovered crash, as described in section 6.9 First Marketing Campaign & Feedback, the initial users stopped using the application quickly. Seemingly, the crash deterred them from using it, and it might have caused the invisible "ranking" of the application on Google Play Store to fall. This could explain why we had a low organic user acquisition when we released iteration five. While general startup advice is to release products quickly, it is possible that our application would have benefited from more thorough testing. We did learn from this mistake; as described in section 7.3.8 Global Exception Handler, more advanced crash monitoring was set up to avoid undetected crashes going forward.

While iterations were made quickly, they were not rushed in terms of code quality. From the beginning, the idea was to continue the project as a startup. Therefore, we had scalability and future complexity increases in mind when building the structure of the system. Prioritizing good architecture has already proven useful - some of the aspects of iteration five was not planned to begin with, but integrating them into existing code was simple due to a solid fundamental structure.

One of the major challenges in the project was the lack of user data after the initial surge of users was lost. While somewhat sophisticated monitoring systems has since been set up, we have not had the opportunity to collect any meaningful quantity of data. Thus, it is unknown how well iteration five of the application would perform with users. The lack of data after the initial surge had the consequence that it was not possible to let user feedback guide the development towards the end of the project period.

Since we are currently relying on third-party analytics systems like Firebase and Google Console, the metrics we can measure are delayed and limited in scope. Improving the ability to measure more specific events and obtain data faster would allow us to extract more data from fewer users. As described in Chapter 8, the immediate next task after ending this project will be to develop an in-house database and analytics system, to get improved measurements. Getting more data from fewer users will be a crucial step towards a better understanding of our KPI's, helping guide future growth efforts.

9.1.4 User Acquisition and Marketing

The marketing campaign on DTU, section 6.9, provided the feedback we were looking for - we got to see how the application was used live. This granted insights into issues with the onboarding flow, and we made relevant changes to the application. A substantial amount of development had to be done to accommodate the feedback we got. Therefore, we think it was the right choice to focus on development after the first user acquisition effort; spending time, effort or money on acquiring new users when they would likely tell us information we already knew would have been redundant. We do not yet have any idea how well iteration five will perform, as it does not suffer from the same shortcomings as previous iterations.

As the financial resources have been limited, due to the project being entirely founder-funded, paid marketing campaigns has been inaccessible. It is not yet clear what the Customer Lifetime Value will be, and it is therefore hard to justify paid marketing campaigns. This leads to slower user-acquisition, which in turn make it harder to estimate the CLV. To get around this, organic marketing will be used until CLV can be somewhat accurately estimated.

9.1.5 Future Perspectives

The project has provided an excellent start towards developing a novel solution to problematic technology use for individuals. The plan for continuous development, section 8.2, will be followed to the best of our ability. Alongside development, more effort will be put into user acquisition and marketing to validate or disprove assumptions. The completion of the plan will likely be

dependent on our ability to raise pre-seed funding, as such we are still in a vulnerable position. If the current plan is successful, we hope to build an attention manager with a broad appeal which could help many change their problematic technology habits.

Further work could explore how different technology habits interplay in more detail. Knowledge of how the newest entertainment technologies affect human behavior is yet incomplete. In this report, we argued for one possible solution to problematic technology use. The digital entertainment technologies are advancing at a rapid pace, and researching the impact and spearheading innovation which could protect customers remain relevant.

9.2 Conclusion

The original objective was to investigate problematic technology use, innovate a solution, formulate a plan and begin an iterative development process. The product quickly reached the MVP stage, and we reached iteration five within the project period. The agile methodology was executed well. The cross-disciplinary nature of the project allowed us to experiment with building an entire system and implement it with current industry infrastructure. Unlike in purely academic environments, the launch of the application led to a confrontation with the requirements of usability and simplicity demanded by real users.

The application in its current form could be used by an individual to monitor and manage their technology use. Therefore, we consider the project successful in achieving the original objective. The success of the solution in a broader sense require further efforts in marketing, product development and fundraising. Future work will focus on refining the product. This includes developing a behavioral algorithm for automated block creation, expanding the application to iOS and conducting controlled testing to gain insight into user behavior.

We are personally satisfied with the project. It provided an opportunity to apply our academic knowledge in a free but demanding setting. We believe that this project is a solid foundation for the completion of the product vision.

References

[1] W. C. Kim and R. Mauborgne, *Blue Ocean Strategy: How to Create Uncontested Market Space and Make the Competition Irrelevant*. Harvard Business Review Press, 2005.

[2] E. Henden, H. O. Melberg, and O. J. Røgeberg, "Addiction: choice or compulsion?," *Addictive Disorders*, vol. 4, 2013. DOI: https://doi.org/10.3389/fpsyt.2013.00077.

[3] J. Aagaard, "Beyond the rhetoric of tech addiction: why we should be discussing tech habits instead (and how)," *Phenomenology and the Cognitive Science*, vol. 20, pp. 559–572, 2021. DOI: https://doi.org/10.1007/s11097-020-09669-z.

[4] A. N. Gearhardt and A. G. DiFeliceantonio, "Highly processed foods can be considered addictive substances based on established scientific criteria," *Addiction*, vol. 118, pp. 589–598, 2023. DOI: https://doi.org/10.1111/add.16065.

[5] D. K. Hatsukami, L. F. Stead, and P. C. Gupta, "Tobacco addiction," *The Lancet*, vol. 371, pp. 2027–2038, 2008. DOI: https://doi.org/10.1016/S0140-6736(08)60871-5.

[6] A. Ezrachi, *1CIIntroduction*. Oxford University Press, 2021.

[7] J. G. Sidak and D. J. Teece, "Dynamic competition in antitrust law," *Journal of Competition Law & Economics*, vol. 5, no. 4, pp. 581–631, 2009.

[8] O. Bar-Gill, "Competition and consumer protection: A behavioral economics account." NYU Law and Economics Research Paper No. 11-42, Forthcoming in *The Pros and Cons of Consumer Protection*, Swedish Competition Authority, 2011. Available at https://ssrn.com/abstract=1974499.

[9] F. Parisi and V. L. Smith, *The Law and Economics of Irrational Behavior*. Stanford Economics And Finance, 2005.

[10] N. Tinbergen, *The Herring Gull's World: A Study of the Social Behavior of Birds*. Basic Books, 1960. Series: New naturalist monograph.

[11] N. Brandborg, *Vanedyr*. Grønningen 1, 1 ed., 2023.

[12] H. Davis and S. McLeod, "Why humans value sensational news: An evolutionary perspective," *Evolution and Human Behavior*, vol. 24, no. 3, pp. 208–216, 2003. DOI: https://doi.org/10.1016/S1090-5138(03)00012-6.

[13] S. Kemp, "Digital 2025: Global overview report." https://datareportal.com/reports/digital-2025-global-overview-report, 2025. Accessed: 2025-07-20.

[14] H. Allcott, M. Gentzkow, and L. Song, "Digital addiction," *American Economic Review*, vol. 112, p. 2424–63, 2022. DOI: https://doi.org/10.1257/aer.20210867.

[15] N. Khalili-Mahani, A. Smyrnova, and L. Kakinami, "To each stress its own screen: A cross-sectional survey of the patterns of stress and various screen uses in relation to self-admitted screen addiction," *Medical Internet Research*, vol. 21, 2019. DOI: https://doi.org/10.2196/11485.

[16] H. Allcott, L. Braghieri, S. Eichmeyer, and M. Gentzkow, "The welfare effects of social media," *American Economic Review*, vol. 110, no. 3, pp. 629–676, 2020. DOI: https://doi.org/10.1257/aer.20190658.

[17] H. Liu, X. Chen, M. Huang, X. Yu, Y. Gan, J. Wang, Q. Chen, Z. Nie, and H. Ge, "Screen time and childhood attention deficit hyperactivity disorder: A meta-

analysis," *Reviews on Environmental Health*, vol. 39, no. 4, pp. 643–650, 2023. DOI: https://doi.org/10.1515/reveh-2022-0262.

[18] Q. Fang, Y. Cai, J. Kang, Y. Zhang, and F. Ye, "The causal associations between screen exposure time and attention decit hyperactivity disorder: a two-sample mendelian randomization," 2024. DOI: https://doi.org/10.21203/rs.3.rs-4266434/v1.

[19] E. Schwaiger and R. Tahir, "The impact of nomophobia and smartphone presence on fluid intelligence and attention," *Cyberpsychology: Journal of Psychosocial Research on Cyberspace*, vol. 16, p. Article 5, 2022. DOI: https://doi.org/10.5817/CP2022-1-5.

[20] G. Frydenlund, J. D. Guldager, K. O. Frederiksen, and H. K. Egebæk, "Do young people perceive their smartphone addiction as problematic? a study in danish university college students," *Heliyon*, vol. 9, p. Issue 10, 2023. DOI: https://doi.org/10.1016/j.heliyon.2023.e20368.

[21] L. Carden and W. Wood, "Habit formation and change," *Current Opinion in Behavioral Sciences*, vol. 20, pp. 117–122, 2018. DOI: https://doi.org/10.1016/j.cobeha.2017.12.009.

[22] E. Ries, *The Lean Startup: How Today's Entrepreneurs Use Continuous Innovation to Create Radically Successful Businesses*. Crown Business, 2011.

[23] E. Union, "Data protection under gdpr." https://europa.eu/youreurope/business/dealing-with-customers/data-protection/data-protection-gdpr/index_en.htm, 2025. Accessed: 2025-07-17.

[24] Accelerace, "Home of the nordic founders building the great companies of tomorrow." https://accelerace.io/, 2025. Accessed: 2025-07-22.

[25] I. F. Denmark, "Innofounder." https://innovationsfonden.dk/en/p/innofounder, 2025. Accessed: 2025-07-22.

[26] T. Consulting, "Public funding." https://www.thursday.consulting/blog/public-funding, 2025. Accessed: 2025-07-23.

[27] Microsoft, "Microsoft for startups." https://foundershub.startups.microsoft.com/, 2025. Accessed: 2025-05-30.

[28] E. Union, "Right to be informed." https://gdpr-info.eu/issues/right-to-be-informed/, year = 2025, note = Accessed: 2025-07-25.

[29] Linkly, "Linkly." https://linklyhq.com/, 2025. Accessed: 2025-07-19.

[30] R. Nunkesser, "Implementing cross-platform business logic in mobile applications using hexagonal architecture," 2023. DOI: https://doi.org/10.36227/techrxiv.22770218.v1.

[31] J. Shrestha, "Evaluating the application of solid principles in modern ai framework architectures," 2025. DOI: https://doi.org/10.48550/arXiv.2503.13786.

[32] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*. Addison-Wesley, 4 ed., 2021.

[33] Kotlin, "Comparison to java." https://kotlinlang.org/docs/comparison-to-java.html, 2025. Accessed: 2025-05-29.

[34] Android, "Build better apps faster with jetpack compose." https://developer.android.com/compose, year = 2025, note = Accessed: 2025-07-23.

[35] C. for Ludomani, "Center for ludomani." https://ludomani.dk/, 2025. Accessed: 2025-07-14.

[36] C. Davenport, "shop-list." https://github.com/corbindavenport/shop-list, 2024. Accessed: 2025-05-18.

[37] S. Black, "hosts." https://github.com/StevenBlack/hosts, 2025. Accessed: 2025-05-18.

[38] IREK-szef, "games-blocklist." https://github.com/IREK-szef/games-blocklist, 2023. Accessed: 2025-05-18.

[39] E. D. Torre, "adult-hosts-list." https://github.com/emiliodallatorre/adult-hosts-list, 2021. Accessed: 2025-05-18.

[40] Spillemyndigheden, "Licence holders." https://www.spillemyndigheden.dk/en/tilladelsesindehavere/print, 2025. Accessed: 2025-05-18.

[41] WebshopListen, "Categories of danish webshops." https://webshoplisten.dk/baby-boern-og-teenager/, 2023. Accessed: 2025-05-18.

[42] Wikipedia, "Kategori:butikker i danmark." https://da.wikipedia.org/wiki/Kategori:Butikker_i_Danmark, 2021. Accessed: 2025-05-18.

[43] T. Preston-Werner, "Semantic versioning 2.0.0." https://semver.org/, 2023. Accessed: 2025-07-17.

[44] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. Ph.d. dissertation, University of California, Irvine, 2000.

[45] G. Blinowski, A. Ojdowska, and A. Przybyłek, "Monolithic vs. microservice architecture: A performance and scalability evaluation," *IEEE Access*, vol. 10, pp. 20357–20374, 2022. DOI: https://doi.org/10.1109/ACCESS.2022.3152803.

[46] Microsoft, "What is azure active directory b2c?." https://learn.microsoft.com/en-us/azure/active-directory-b2c/overview, 2025. Accessed: 2025-07-03.

[47] Microsoft, "Azure identity management and access control security best practices." https://docs.azure.cn/en-us/security/fundamentals/identity-management-best-practices, 2025. Accessed: 2025-07-17.

[48] A. Haxthausen and F. Schmidt, "7. normalization." https://learn.inside.dtu.dk/d2l/le/content/60224/viewContent/244519/View, 2021. Accessed: 2025-06-30.

[49] Cloud86, "Cloud86." https://cloud86.io/, year = 2025, note = Accessed: 2025-07-25.

[50] LondonX, "tun2socks-android." https://github.com/LondonX/tun2socks-android, 2023. Accessed: 2025-06-27.

[51] heiher, "sockstun." https://github.com/heiher/sockstun, 2025. Accessed: 2025-06-27.

[52] Anton-Dodonov, "Vpnclient-engine." https://github.com/VPNclient/VPNclient-engine, 2025. Accessed: 2025-06-27.

[53] Zenarmor, "Dns tunneling: Detecting dns tunneling attacks." https://www.zenarmor.com/docs/network-security-tutorials/what-is-dns-tunneling, 2023. Accessed: 2025-06-27.

[54] NextDNS, "The new firewall for the modern internet." https://nextdns.io/, 2025. Accessed: 2025-07-10.

[55] OpenDNS, "Improve your internet." https://www.opendns.com/, 2025. Accessed: 2025-07-10.

[56] Asana, "Asana." https://asana.com/, 2025. Accessed: 2025-07-14.

APPENDIX

10.1 Endpoints

10.1.1 POST /api/accounts

Description: Creates a new account.

Method	POST
Endpoint	/api/accounts
URL Params	None
Query Params	None
Request Body	JSON with email and external id
Response	JSON with external id

Example Body

```
{
  "email": "user@example.com",
  "externalId": "c2d4-4a4b-8f90-5ea1"
}
```

Example Response

```
{
  "ExternalId": "c2d4-4a4b-8f90-5ea1"
}
```

10.1.2 GET /api/accounts/{externalId}

Description: Gets a specific account from external id.

Method	GET
Endpoint	/api/accounts/{externalId}
URL Params	externalId, the external id of the user
Query Params	None
Request Body	None
Response	JSON as account object

Example Response

```
{
  "Email": "user@example.com",
  "ExternalId": "c2d4-4a4b-8f90-5ea1",
  "Permission": 0
}
```

10.1.3 PUT /api/accounts/{externalId}

Description: Updates a specific account.

Method	PUT
Endpoint	/api/accounts/{externalId}
URL Params	externalId, the external id of the user
Query Params	None
Request Body	JSON with old and new email
Response	HTTP Status 204, No Content

Example Body

```
{
  "oldEmail": "user@example.com",
  "newEmail": "new@email.com"
}
```

10.1.4 DELETE /api/accounts/{externalId}/delete/request

Description: Sets a request to delete an account. This sends the verification email to the email of the user.

Method	DELETE
Endpoint	/api/accounts/{externalId}
URL Params	externalId, the external id of the user
Query Params	None
Request Body	None
Response	HTTP Status 204, No Content

10.1.5 POST /api/accounts/{externalId}/delete/verify

Description: Verifies the verification code and deletes the account.

Method	POST
Endpoint	/api/accounts/{externalId}/delete/verify
URL Params	externalId, the external id of the user
Query Params	None
Request Body	JSON with verification code
Response	HTTP Status 204, No Content

Example Body

```
{
  "verificationCode": "123456",
}
```

10.1.6 POST /api/accounts/{externalId}/email/request

Description: Sets a request to change an email for an account. This sends the verification email to the email of the user.

Method	POST
Endpoint	/api/accounts/{externalId}/email/request
URL Params	externalId, the external id of the user
Query Params	None
Request Body	JSON, with the new and old email
Response	HTTP Status 204, No Content

Example Body

```
{
  "oldEmail": "user@example.com",
  "newEmail": "new@email.com"
}
```

10.1.7 POST /api/accounts/{externalId}/email/verify

Description: Verifies the verification code for the change email request.

Method	POST
Endpoint	/api/accounts/{externalId}/email/verify
URL Params	externalId, the external id of the user
Query Params	None
Request Body	JSON, with the new email and the verification code
Response	HTTP Status 204, No Content

Example Body

```
{
  "newEmail": "new@email.com",
  "verificationCode": "123456"
}
```

10.1.8 POST /api/feedback

Description: Sends feedback to Google Sheets.

Method	POST
Endpoint	/api/feedback
URL Params	None
Query Params	None
Request Body	JSON, with the feedback object
Response	JSON, with the id of the created feedback

Example Body

```
{
  "text": "I have experienced this bug here",
  "type": "Bug",
  "device": "Google Pixel 8",
  "date": "2025-07-10",
  "email": null
}
```

10.1.9 GET /api/health

Description: Health endpoint, for Azure to monitor if the server is actively running.

Method	GET
Endpoint	/api/health
URL Params	None
Query Params	None
Request Body	None
Response	PlainText, the status ”UP”

Example Response

```
UP
```

10.1.10 POST /api/accounts/{externalId}/blocklists

Description: Creates a block list.

Method	POST
Endpoint	/api/accounts/{externalId}/blocklists
URL Params	externalId, the external id of the user
Query Params	None
Request Body	JSON, with the block list object
Response	JSON, with the external id of the created block list

Example Body

```
{
  "name": "Study Time",
  "websites": [
  {
    "url": "facebook.com"
  }
  ],
  "applications": [
  {
    "packageName": "com.facebook.katana"
  }
  ]
}
```

Example Response

```
{
"ExternalId": "c2d4-4a4b-8f90-5ea1"
}
```

10.1.11 GET /api/accounts/{externalId}/blocklists

Description: Gets all the block lists of a user.

Method	GET
Endpoint	/api/accounts/{externalId}/blocklists
URL Params	externalId, the external id of the user
Query Params	None
Request Body	None
Response	JSON, with a list of block list objects

Example Response

```
{
    "BlockList": [
        {
          "Name": "Study Time",
          "Websites": [
          {
            "Url": "facebook.com"
          }
          ],
          "Applications": [
          {
            "PackageName": "com.facebook.katana"
          }
          ]
        }
    ]
}
```

10.1.12 DELETE /api/accounts/{externalId}/blocklists/{blockListId}

Description: Deletes a specific block list from a user.

Method	DELETE
Endpoint	/api/accounts/{externalId}/blocklists/{blockListId}
URL Params	externalId, the external id of the user and blockListId, the id of the block list
Query Params	None
Request Body	None
Response	HTTP Status 204, No Content

10.2 Firebase Events

Event	Explanation
trigger_website_blocked	Whenever the user tries to access a website that is currently blocked.
trigger_app_blocked	Whenever the user tries to access an application that is currently blocked.
privacy_policy_accepted	The privacy policy was accepted when creating a user.
privacy_policy_read	The privacy policy was read, optionally, when creating a user.
terms_of_service_accepted	The terms of service was accepted when creating a user.
terms_of_service_read	The terms of service was read, optionally, when creating a user.
accessibility_permission_required	The user tried turning on a block without accessibility service enabled, triggering the alert to enable it.
accessibility_permission_denied	The user canceled the alert with about enabling the accessibility service, or clicked agree but did not turn it on in the settings before going back to the application.
accessibility_permission_granted	The user clicked agree on the alert and enabled the accessibility service, before going back to the application.
delete_account_request	The user clicked on the delete account button.
delete_account_cancel	The user canceled the flow, before deleting their account.
delete_account_success	The user verified their email and confirmed deletion, deleting their account.
email_change_request	The user clicked on the change email button.
email_verification_cancel	The user canceled the flow, before changing their email.
email_changed_success	The user verified their email and confirmed the change, changing their email.

Table 6: Overview of All Firebase Events

Event	Explanation
accessibility_service_disabled _with_active_block	Accessibility service is currently disabled, but a block is active.
Screen Views	This is a special event. Every time the user changes the page inside of the application, an event is sent for that specific screen, so we know how the user moves through the application.
app_crash	Whenever the global exception handler captures an error, that would have lead to a crash.
adult_content_blocked	Turned on the category blocking for adult content.
adult_content_unblocked	Turned off the category blocking for adult content.
gambling_blocked	Turned on the category blocking for gambling.
gambling_unblocked	Turned off the category blocking for gambling.
social_media_blocked	Turned on the category blocking for social media.
social_media_unblocked	Turned off the category blocking for social media.
shopping_blocked	Turned on the category blocking for shopping.
shopping_unblocked	Turned off the category blocking for shopping.
games_blocked	Turned on the category blocking for games.
games_unblocked	Turned off the category blocking for games.
logout	The user has logged out.
login	The user has logged in.
signup	The user has created an account.

Table 7: Overview of All Firebase Events

Event	Explanation
block_created_event	The user has created a new block.
block_activated_event	A block has been activated. This is both from manually turning it on and through recurring.
block_deactivated_event	A block has been deactivated. This is both from manually deactivating it and through recurring.
block_activation_constant	The block was turned on, with the activation setting of constant.
block_activation_recurring	The block was turned on, with the activation setting of recurring.
block_lock_none	The block was turned on, without a lock type.
block_lock_timed	The block was turned on, with the timed lock type.
block_lock_delay	The block was turned on, with the delay lock type.
feedback_sent	The user has sent feedback through the application.
notification_permission_granted	The notification setting was granted, when the user was prompted or if they clicked on the permission on the tutorial screen.
notification_permission_denied	The notification setting was denied, or the user canceled the prompt for the permission.
exact_alarm_permission_granted	The exact alarm setting was granted, when the user was prompted to turn it on in their settings.
exact_alarm_permission_denied	The exact alarm setting was denied, by either the user canceling the prompt or by agreeing and going to the settings, but not granting the permission before going back to the application.
active_user	The user satisfies the criteria of an active user.
inactive_user	The user does not satisfy the critieria of an active user.

Table 8: Overview of All Firebase Events

10.3 Version Display

10.3.1 Version 1.0.0

- Date: 2025-05-23
- Features:
  - This is the first version of the app, providing a solid foundation for blocking unwanted apps and websites.
  - Users can now block specific apps and websites to enhance productivity and focus.
  - The app allows users to edit their blocks easily, providing flexibility in managing distractions.
  - Designed with a user-friendly interface to ensure a smooth experience for all users.

10.3.2 Version 1.1.0

- Date: 2025-06-09
- Features:
  - Users can now block entire categories of apps and websites, making it easier to manage distractions. Current categories include Social Media, Shopping, Adult Content, Gambling and Games.
  - Account management features have been added, allowing users to create and manage their accounts.
- Improvements:
  - Improved the stability of the app, preventing it from crashing during usage in the background.
- Bug Fixes:
  - Fixed an issue where users could block off internal apps like Settings, which could lead to confusion and accessibility issues.
  - Fixed an issue where turning on multiple blocks that blocked the same website/app and then turning one off, would cause the block to not work, even if one block was still active with the app/website.

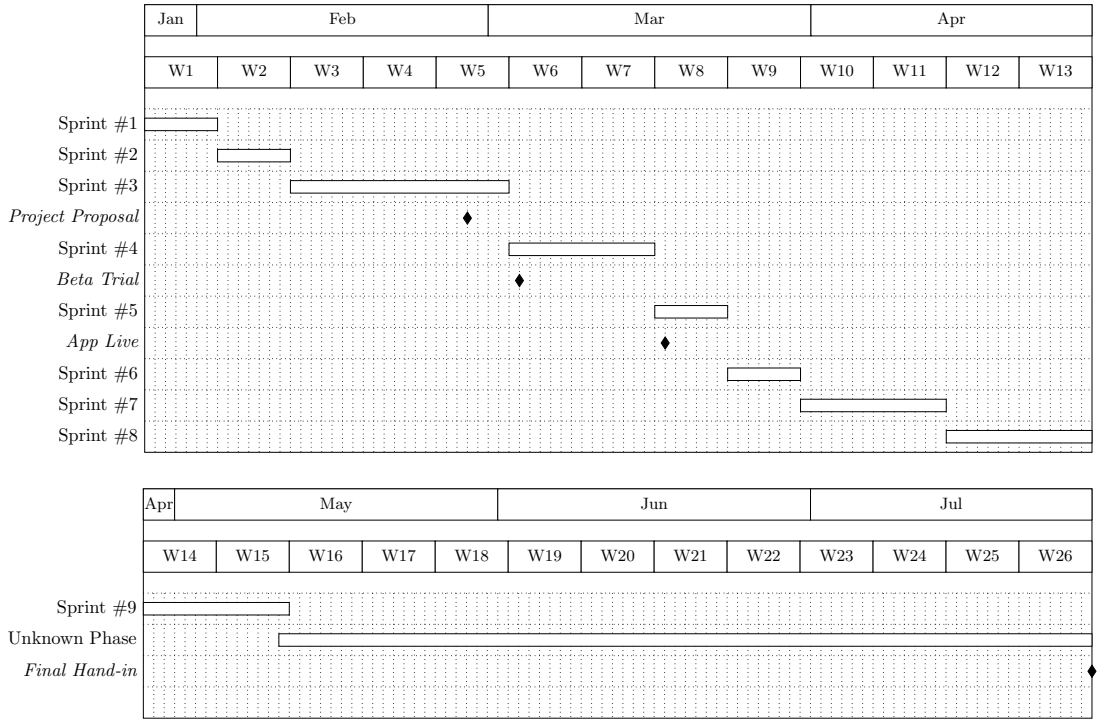
10.3.3 Version 1.1.1

- Date: 2025-06-24
- Features:
  - Added an onboarding tutorial to help new users understand how to use the app. The tutorial guides users through the main features and functionalities, ensuring a better start.
- Bug Fixes:
  - Fixed an issue where new users were not correctly created in the database, causing them to be unable to use the app.
  - Fixed an issue when categorizing installed apps in the category blocking feature, which caused some apps to be categorized incorrectly.

10.3.4 Version 1.2.0

- Date: 2025-07-14
- Features:
  - Users can now view data reporting and analytics to understand their usage patterns and the effectiveness of their blocks.
  - The user interface has been redesigned for better usability, making it easier to navigate and manage blocks. This includes the changing of the navigation bar, the home screen and added permission introduction in the onboarding tutorial.
  - Recurring blocks have been introduced, allowing users to set up blocks that automatically activate at specified times.
  - A new setup method for lock and blocking methods has been introduced, providing more flexibility in how blocks are applied. This includes the ability to set up blocks that has a delay, timed, or recurring blocks.
- Improvements:
  - Introduced better handling of errors in general, that prevents the app from crashing when an error occurs.
- Bug Fixes:
  - Fixed an issue where new users were sometimes not correctly created and could not use the app.

10.4 Initial Project Plan



10.4.1 Pre-Release Phase

Before releasing the application, sprints can be well documented, as the vision and features are already clearly defined.

10.4.1.1 Sprint #1

This sprint focuses on setting up the entire project and developing an initial prototype capable of blocking a single website and a single installed application on an Android emulator. This sprint includes project planning to establish a clear roadmap.

10.4.1.2 Sprint #2

The goal of this sprint is to expand the prototype by improving the user interface and enabling customization of blocking rules. This includes allowing users to define which websites and applications they want to block. No tasks outside of the technical scope are planned in this sprint.

10.4.1.3 Sprint #3

This sprint involves implementing account management and further refining the user interface. The backend, hosted on Azure, will be developed to support secure account creation and login.

This sprint prepares for the beta trial by recruiting users, maintaining communication and setting up the required infrastructure on Azure. Since this is the final sprint before the beta trial, a public-facing website for account creation will also be developed. Sentry, an error monitoring and performance tracking tool, will be integrated to analyze application performance during beta testing. Given the importance of data collection in this phase, Android analytics will be implemented to track app usage.

10.4.1.4 Sprint #4

This sprint is dedicated to the beta trial. The primary focus is identifying and resolving errors or performance issues that happens during the trial period. Qualitative feedback will be collected from users through interviews, to understand their experience with the application.

10.4.2 Post-Release Phase

Once the application is live, sprints become less planned, as real user interactions may change priorities and focus. While the project has an idea of a roadmap, going live potentially introduces unexpected features and adjustments may happen.

Each sprint in this phase will introduce one key metric:

- **Number of Users** – The number of paying customers.

While this metric guides development, it is not strict criteria for sprint completion.

Our goal is to find metrics which are accurate proxies for churn, virality factor and projected user growth. Ideally, these metrics are directly connected to actions we can take to improve them, and hence also improve important KPI's.

10.4.2.1 Sprint #5

Number of Users: 0

This sprint focuses on the official release of the application. Tasks include publishing the app on the store, ensuring updates can be pushed correctly and verifying that everything functions as expected for end users.

10.4.2.2 Sprint #6

Number of Users: 10

This sprint aims to get the first customers while monitoring error handling and performance. Churn analysis will be conducted to identify missing features or usability issues. Additionally, a new feature allowing users to block entire categories (for example all social media sites) with a single click will be developed.

10.4.2.3 Sprint #7

Number of Users: 20

This sprint focuses on improving the user interface. The team will reach out to designers to explore potential UI enhancements for a better user experience.

10.4.2.4 Sprint #8

Number of Users: 30

This sprint introduces cross-platform support, developing versions of the application for Windows and iOS while ensuring synchronization of accounts and settings across devices.

10.4.2.5 Sprint #9

Number of Users: 60

This sprint shifts focus to marketing strategies and outreach. The team will engage with public relations channels, including journalists and podcast teams, to increase visibility to potential users. Additionally, expanding the customer base will be prioritized to gather more feedback, identify performance issues and refine the project scope based on user demand.

10.4.3 Unknown Phase

Beyond this point, no sprints are pre-planned. The project is planned until **2025-05-11**, but after this date, the direction remains uncertain.

While long-term goals include implementing AI for personalized blocking recommendations, a lot of unpredictable challenges and opportunities may come forward. Potential unknowns include scalability issues, inaccurate Customer Acquisition Cost (CAC) calculations and predictions or misalignment of customer research with the actual target audience. Due to these uncertainties, this phase remains open-ended for now.

10.5 BigQuery User Engagement Measurements

10.5.1 Active Users

To measure the active users, we execute the following query against the data we get from Firebase analytics.

```
WITH date_range AS (
  SELECT day
  FROM UNNEST(GENERATE_DATE_ARRAY('2025-06-15', CURRENT_DATE())) AS day
),
daily_active AS (
  SELECT
    DATE(TIMESTAMP_MICROS(event_timestamp)) AS event_date,
    COUNT(DISTINCT user_pseudo_id) AS active_users
  FROM
    `axionapp-7b35a.analytics_483826613.events_*`
  WHERE
    event_name = 'active_user'
    AND _TABLE_SUFFIX BETWEEN '20250615' AND FORMAT_DATE('%Y%m%d',
        CURRENT_DATE())
  GROUP BY event_date
)

SELECT
  d.day AS event_date,
  IFNULL(a.active_users, 0) AS active_users,
  IFNULL(a.active_users, 0) - LAG(IFNULL(a.active_users, 0)) OVER (ORDER
      BY d.day) AS change_from_previous_day
FROM
  date_range d
LEFT JOIN
  daily_active a ON d.day = a.event_date
ORDER BY event_date;
```

This query first gets all the dates, from the all the exports we get from Firebase. It goes from the first export, 2025-06-15 till the date that it is executed. From all the dates, it then, for each date, counts the number of distinct users that got the `active_event` event sent. After that, it then goes over all the dates to place a 0 in the dates, where no users was registered with the event. At the end of the script, it then goes over the changes in the count for each day, to visualize how many active users were gained or lost, for each of the dates. This gives us a full overview of all days, to see the active users either grow or decline.

10.5.2 Long Term Users

```sql
WITH daily_users AS (
  SELECT
    DATE(TIMESTAMP_MICROS(event_timestamp)) AS event_date,
    user_pseudo_id
  FROM
    `axionapp-7b35a.analytics_483826613.events_*`
  WHERE
    event_name = 'active_user'
    AND _TABLE_SUFFIX BETWEEN '20250615' AND FORMAT_DATE('%Y%m%d',
        CURRENT_DATE())
  GROUP BY event_date, user_pseudo_id
),

user_streaks AS (
  SELECT
    user_pseudo_id,
    event_date,
    COUNT(*) OVER (
      PARTITION BY user_pseudo_id
      ORDER BY event_date
      ROWS BETWEEN 13 PRECEDING AND CURRENT ROW
    ) AS streak_14_days
  FROM daily_users
)

SELECT
  event_date,
  COUNT(DISTINCT user_pseudo_id) AS long_term_users
FROM
  user_streaks
WHERE
  streak_14_days >= 14
GROUP BY event_date
ORDER BY event_date;
```

To measure how many users qualify as long term users, we define a long term user as someone who has been active for 14 consecutive days. The query above calculates this by first identifying each day a user triggered the `active_user` event.

The `daily_users` selects all user ids and their corresponding activity dates. The `user_streaks` then uses a function to count the number of days the user has been active within the past 14 days, including the current day. This allows us to determine if a user has maintained a 14 day streak. The query then filters out all records where the user has not met the requirement of 14 consecutive active days and counts the distinct users per day who did. This gives us a daily measurement of users that can be considered long term users.

10.5.3 Churn Rate

```
WITH daily_long_term_users AS (
  SELECT
    event_date,
    user_pseudo_id
  FROM (
    SELECT
      user_pseudo_id,
      event_date,
      COUNT(*) OVER (
        PARTITION BY user_pseudo_id
        ORDER BY event_date
        ROWS BETWEEN 13 PRECEDING AND CURRENT ROW
      ) AS streak_14_days
    FROM (
      SELECT
        DATE(TIMESTAMP_MICROS(event_timestamp)) AS event_date,
        user_pseudo_id
      FROM
        `axionapp-7b35a.analytics_483826613.events_*`
      WHERE
        event_name = 'active_user'
        AND _TABLE_SUFFIX BETWEEN '20250615' AND FORMAT_DATE('%Y%m%d',
            CURRENT_DATE())
      GROUP BY event_date, user_pseudo_id
    )
  )
  WHERE streak_14_days >= 14
),

prev_day_users AS (
  SELECT
    event_date AS prev_date,
    user_pseudo_id
  FROM daily_long_term_users
),

curr_day_users AS (
  SELECT
    event_date AS curr_date,
    user_pseudo_id
  FROM daily_long_term_users
),

joined_users AS (
  SELECT
    prev.prev_date,
    curr.curr_date,
    prev.user_pseudo_id AS prev_user,
    curr.user_pseudo_id AS curr_user
  FROM prev_day_users prev
  LEFT JOIN curr_day_users curr
    ON prev.user_pseudo_id = curr.user_pseudo_id
    AND curr.curr_date = DATE_ADD(prev.prev_date, INTERVAL 1 DAY)
)
```

```
SELECT
  DATE_ADD(prev_date, INTERVAL 1 DAY) AS event_date,
  COUNT(DISTINCT prev_user) AS long_term_users_previous_day,
  COUNT(DISTINCT curr_user) AS long_term_users_current_day,
  COUNT(DISTINCT prev_user) - COUNT(DISTINCT curr_user) AS
      long_term_users_lost,
  SAFE_DIVIDE(COUNT(DISTINCT prev_user) - COUNT(DISTINCT curr_user),
      COUNT(DISTINCT prev_user)) AS long_term_user_churn_rate
FROM joined_users
GROUP BY prev_date
ORDER BY event_date;
```

To calculate the churn rate of long term users, the query begins by selecting users who qualify as long term users, meaning they were active for 14 consecutive days. This is done in the `daily_long_term_users`. It then creates two separate datasets: one representing the long term users from a previous day and another from the following day. These are then joined to track which users remained long term active across these consecutive days.

The query then compares the number of users present on both days to determine how many were lost and calculates the churn rate by dividing the number of users lost by the number of users from the previous day. This gives a day to day overview of the percentage of long term users who stopped being active, to effectively track our churn rate.

10.5.4 User Growth Daily

```
WITH date_range AS (
  SELECT day
  FROM UNNEST(GENERATE_DATE_ARRAY('2025-06-15', CURRENT_DATE())) AS day
),
daily_active AS (
  SELECT
    DATE(TIMESTAMP_MICROS(event_timestamp)) AS event_date,
    COUNT(DISTINCT user_pseudo_id) AS active_users
  FROM
    `axionapp-7b35a.analytics_483826613.events_*`
  WHERE
    event_name = 'active_user'
    AND _TABLE_SUFFIX BETWEEN '20250615' AND FORMAT_DATE('%Y%m%d',
        CURRENT_DATE())
  GROUP BY event_date
),
filled_active AS (
  SELECT
    d.day AS event_date,
    IFNULL(a.active_users, 0) AS active_users
  FROM date_range d
  LEFT JOIN daily_active a ON d.day = a.event_date
)

SELECT
  event_date,
  active_users,
  LAG(active_users) OVER (ORDER BY event_date) AS prev_active_users,
  CASE
    WHEN LAG(active_users) OVER (ORDER BY event_date) IS NULL THEN NULL
    WHEN LAG(active_users) OVER (ORDER BY event_date) = 0 AND
        active_users > 0 THEN 1.0
    WHEN LAG(active_users) OVER (ORDER BY event_date) = 0 AND
        active_users = 0 THEN 0.0
    ELSE SAFE_DIVIDE(active_users - LAG(active_users) OVER (ORDER BY
        event_date), LAG(active_users) OVER (ORDER BY event_date))
  END AS user_growth_rate
FROM filled_active
ORDER BY event_date;
```

This query calculates the daily user growth rate by comparing the number of active users from one day to the next. It starts by generating all dates. For each of those dates, it then counts how many unique users triggered the `active_user` event.

To make sure all dates are included in the output, even if no users were active on a given day, the query fills in missing days with zero active users. This is simply for better readability. It then calculates the number of users from the previous day using the `LAG` function and compares it to the count of the current day to compute the growth rate. The LAG function simply takes

the values from the previous day, making that computation much easier.

Special care is taken to handle edge cases: if the value of the previous day is missing, growth is shown as NULL. If there were no users yesterday but there are today, growth is defined as 1.0 or 100%. If there were no users on both days, the rate is set to 0.0. This gives a daily view of how the user base is increasing or decreasing over time.

10.5.5 Long Term User Growth Daily

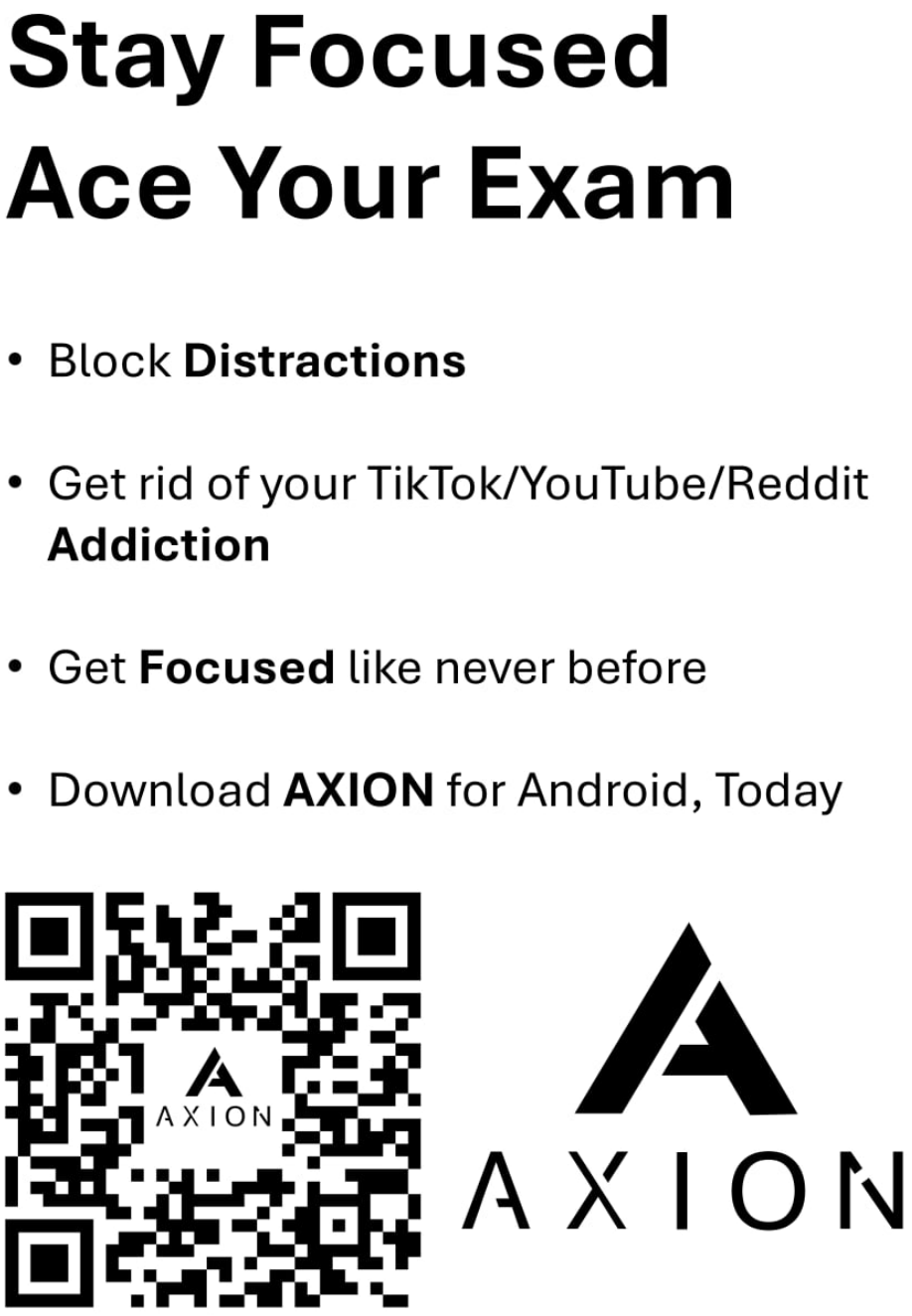
```
WITH date_range AS (
  SELECT day
  FROM UNNEST(GENERATE_DATE_ARRAY('2025-06-15', CURRENT_DATE())) AS day
),
daily_long_term_users AS (
  SELECT
    event_date,
    user_pseudo_id
  FROM (
    SELECT
      user_pseudo_id,
      event_date,
      COUNT(*) OVER (
        PARTITION BY user_pseudo_id
        ORDER BY event_date
        ROWS BETWEEN 13 PRECEDING AND CURRENT ROW
      ) AS streak_14_days
    FROM (
      SELECT
        DATE(TIMESTAMP_MICROS(event_timestamp)) AS event_date,
        user_pseudo_id
      FROM
        `axionapp-7b35a.analytics_483826613.events_*`
      WHERE
        event_name = 'active_user'
        AND _TABLE_SUFFIX BETWEEN '20250615' AND FORMAT_DATE('%Y%m%d',
            CURRENT_DATE())
      GROUP BY event_date, user_pseudo_id
    )
  )
  WHERE streak_14_days >= 14
),
daily_long_term_counts AS (
  SELECT
    event_date,
    COUNT(DISTINCT user_pseudo_id) AS long_term_users
  FROM daily_long_term_users
  GROUP BY event_date
),
filled_long_term AS (
  SELECT
    d.day AS event_date,
    IFNULL(l.long_term_users, 0) AS long_term_users
  FROM date_range d
  LEFT JOIN daily_long_term_counts l ON d.day = l.event_date
)
```

```
SELECT
  event_date,
  long_term_users,
  LAG(long_term_users) OVER (ORDER BY event_date) AS
      prev_long_term_users,
  CASE
    WHEN LAG(long_term_users) OVER (ORDER BY event_date) IS NULL THEN
        NULL
    WHEN LAG(long_term_users) OVER (ORDER BY event_date) = 0 AND
        long_term_users > 0 THEN 1.0
    WHEN LAG(long_term_users) OVER (ORDER BY event_date) = 0 AND
        long_term_users = 0 THEN 0.0
    ELSE SAFE_DIVIDE(long_term_users - LAG(long_term_users) OVER (ORDER
        BY event_date), LAG(long_term_users) OVER (ORDER BY event_date))
  END AS long_term_user_growth_rate
FROM filled_long_term
ORDER BY event_date;
```

To measure the growth rate of long term users, we follow the same 14 day activity definition. The query begins by selecting all days within our period. It then identifies users that qualify as long term users per day, the same way as the other queries.

After identifying long term users for each date, the query fills in any missing days with zeroes to ensure the data is well formed. It then uses the `LAG` function to compare the long term user count of each day with the previous day. Based on this comparison, we calculate the daily growth rate, the same way we did for the active users.

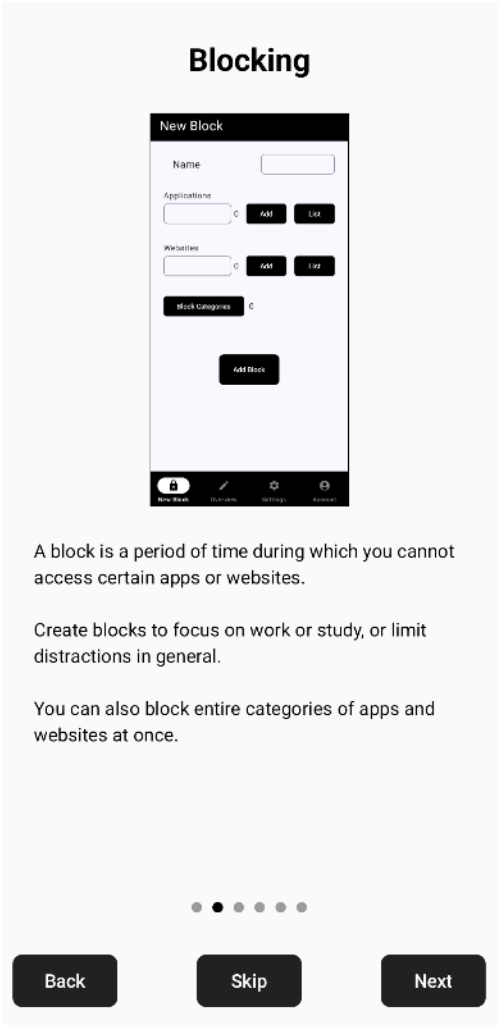
10.6 DTU Flyer



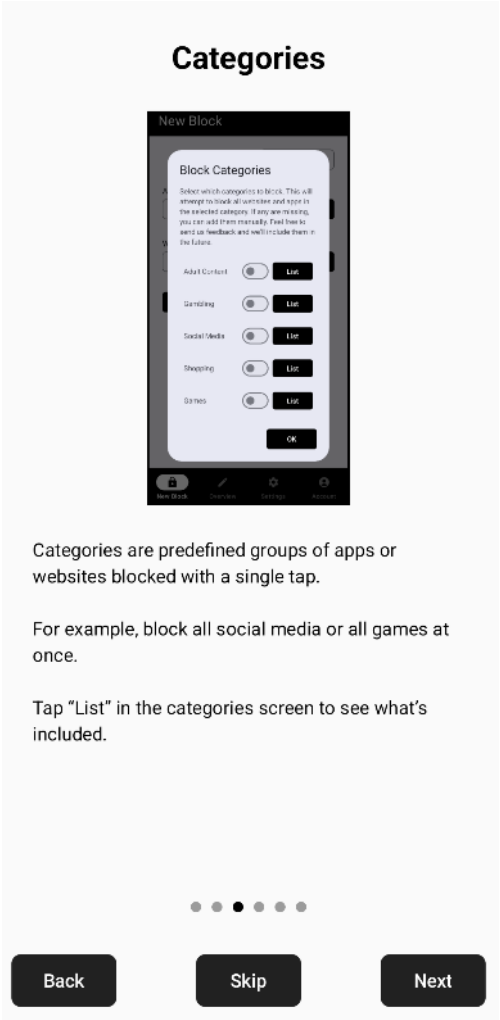
10.7 Application Tutorial Screens



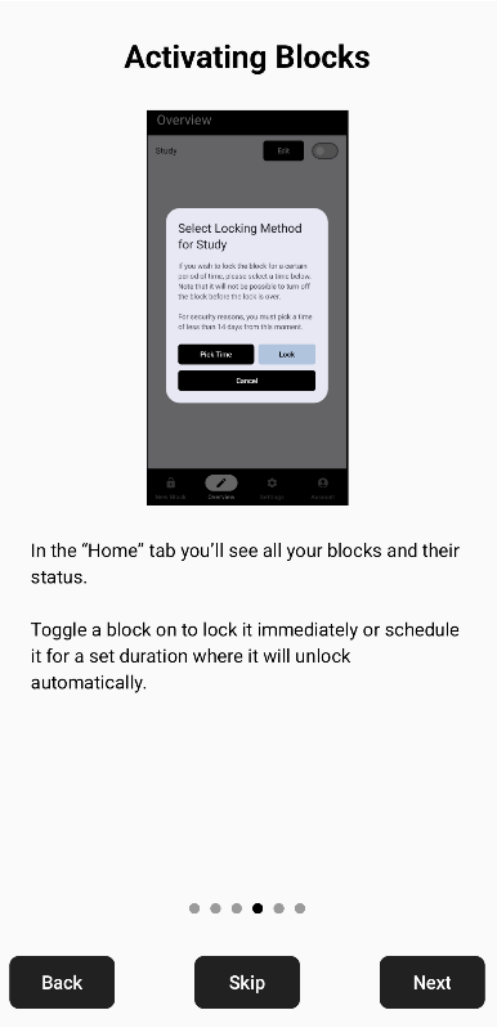
(a) First Screen



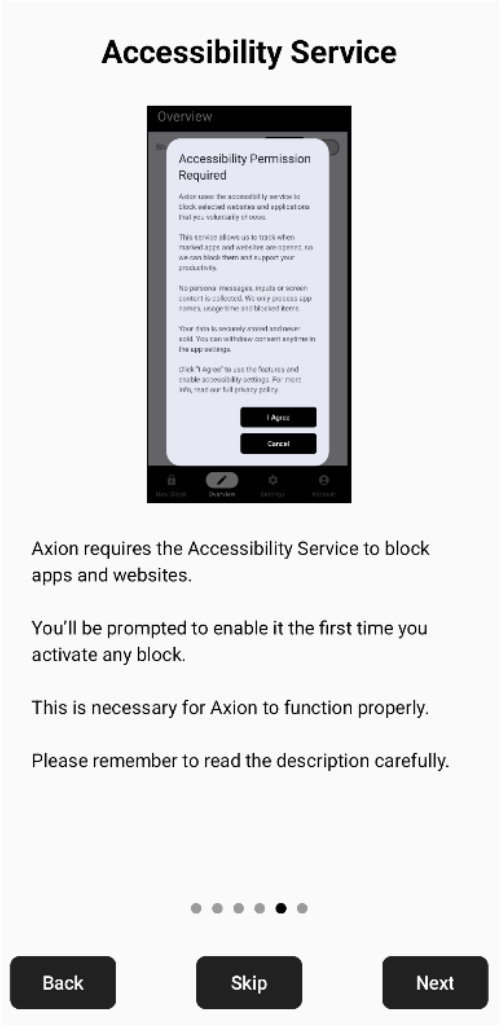
(b) Second Screen



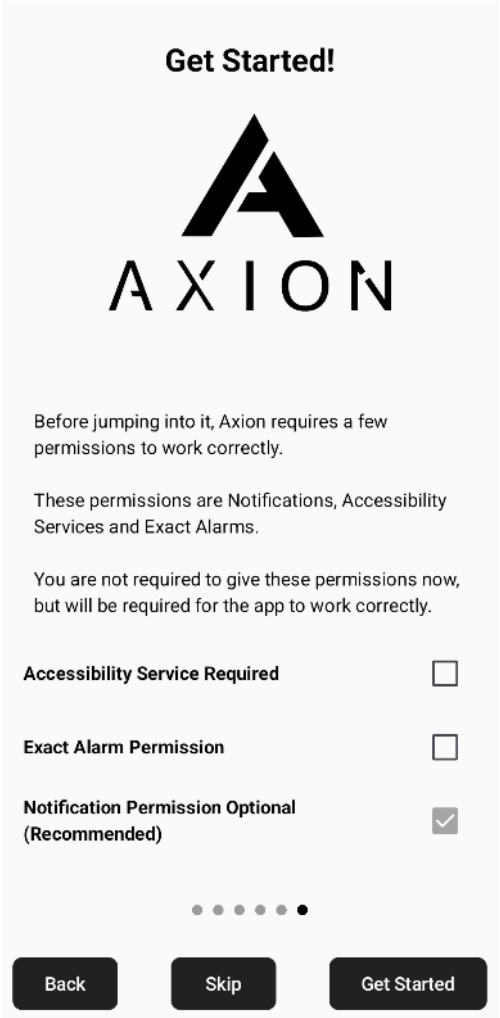
(a) Third Screen



(b) Fourth Screen



(a) Fifth Screen



(b) Sixth Screen

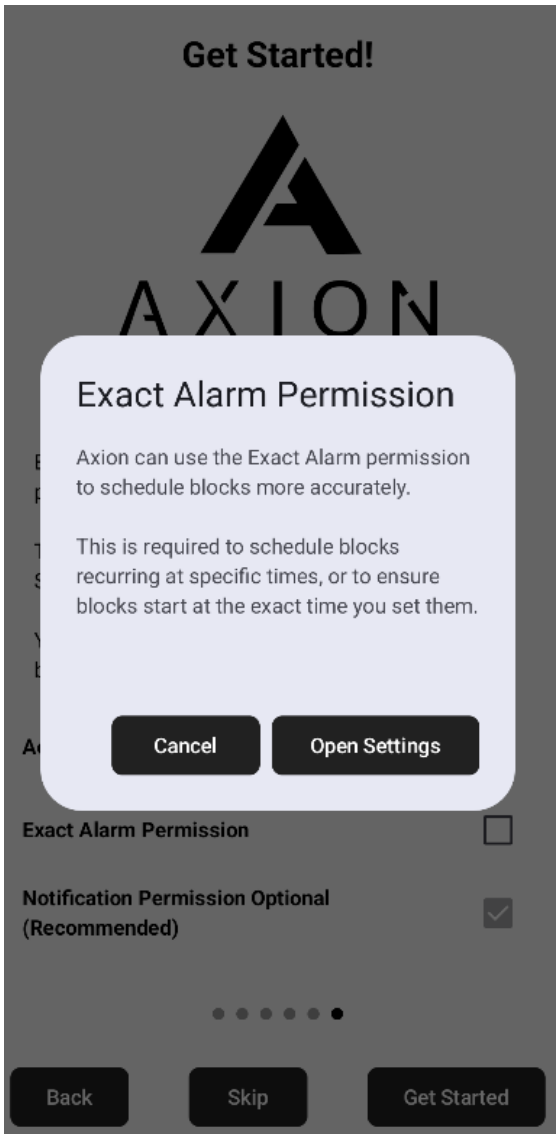


Figure 30: Exact Alarms Permission Alert

10.8 Board of Advisors

10.8.1 Board Members



Jan Borg

Has worked with IT and project management for over 10 years. Educated as a cand.it and has taught at CPH Business Academy. Currently working as a consultant for the Danish Armed Forces. Jan has his own company **Borg Interactive**, which develops games and apps for smaller businesses.



Morten Lahrmann

Currently working as a manager at Fujifilm, previously a manager at Novo Nordisk. Morten has experience from two earlier start-ups - one focused on gathering offers from craftsmen and another application based ordering system for cafés called **Butler**. While both companies were ultimately unsuccessful due to challenges with selling the product, they provided valuable entrepreneurial insight. Alongside his professional career, Morten has continuously engaged in business development activities.



Tina Braunstein

CEO at **Greve Vand**, a private water supply company with 11 employees. Tina is an educated cand.polyt and has extensive experience in project management across various sectors.

10.8.2 Meeting Summaries

10th of January - 2025
An initial board meeting was held at DTU Skylab. The purpose was to present the project concept, business plan and technical solution to the board members. The topics included market analysis, competitors, monetization go-to-market strategy and the project roadmap. The session also involved a collaborative market GAP analysis and a discussion on the future role of the board and each members competencies.

The board meeting resulted in valuable feedback regarding technical feasibility, potential user targeting and the market positioning of the project. The board members emphasized on starting with a minimal viable product, focusing only on core features and strong data security. Suggestions included strongly advised avoiding terms like "addiction", due to the negative stigma surrounding them. It also included further research into first development on iOS or Android, taking into account technical challenges and user potential. Lastly it was suggested to explore personalization and behavioral insights and how this could be used in the application. Several ideas for the application and it could evolve was discussed, including the application learning and understand the individual user habit through collected data, integration with other type of hardware like a smartwatch and to further improve on customization features to fit as many individual use cases as possible. The importance of clear KPIs, real user testing and often prototyping was highlighted, along with input on funding strategies and relevant technical competencies potentially needed going forward.

27th of January - 2025
The second board meeting was held virtually via Microsoft Teams. The agenda centered on the technical solution, beginning with a brief presentation of a potential approach. This was followed by a joint discussion on its feasibility and improvements. The meeting concluded with a review of potential third party resources that could support the development.

The second board meeting provided valuable insights into the technical direction of the project. The discussion focused on the proposed technical stack, which included an Android application written in Kotlin, a C# backend hosted on Azure and a MySQL database. The board supported the choice of Android for the initial release, emphasizing the importance of focusing on a single platform to reduce complexity and increase the amount of features feasible to create, within the project timeline.

Sprint planning was presented and discussed, outlining five sprints with specific deliverables, including early blocking functionality, integration with analytics tools and user testing. The board advised strongly towards the value of iterative development, clear milestones and both qualitative and quantitative user research. Focus group methodology and interview techniques were discussed in depth, with strong emphasis on uncovering real user pain points and behavioral patterns. Additional feedback also included developing technical mockups for early testing and using academic methods for sketching the solution, by utilizing known modeling techniques such as UML and use case diagrams.

11th of February - 2025
The third board meeting was held virtually via Microsoft Teams. The agenda included a presentation of the application and created sequence diagrams, followed by an updated roadmap and strategic division of the functions of the company. The meeting concluded with a focused discussion on defining KPIs to measure objective time savings, subjective user experience, irritation caused by the application and other relevant metrics.

The board meeting resulted in positive feedback on the quick progress of the application, especially noting the implemented ability to block a predefined application and website, despite the limited development time so far. The presented sequence diagrams effectively demonstrated

usability, though the board recommended additional diagrams to enhance documentation and clarify end-to-end workflows. The updated roadmap and strategic division of company functions into areas such as business, organization, development, etc. were well received. A bigger part of the discussion focused on defining KPIs, especially around minimizing user irritation to encourage continous use of the application. The board emphasized strategies like positive "nudging". This could for example be displaying encouraging messages when users encounter blocked content to promote other, potentially more health, behavior. There was also a thorough conversation on measuring subjective user experiences and setting up analytics tools to monitor usage accurately. Overall, the board highlighted the critical importance of understanding user interactions to guide continuous improvements and future development.

4th of March - 2025
The fourth board meeting was held virtually via Microsoft Teams. The agenda included a brief progress update, followed by a presentation and discussion of alternative monetization strategies, focusing on a freemium model versus a free trial approach. The meeting also addressed the possibility of launching the MVP earlier than planned if adopting the freemium model, discussing the pros and cons of a full release versus a beta test. The meeting would conclude with a development plan outlining feature priorities, which was presented and discussed.

The board meeting resulted in positive feedback on the significant progress of the application, now featuring a more developed user interface and the ability to set multiple blocks for various apps and websites. With the MVP getting closer to completion, the board engaged in a thorough discussion on monetization strategies. They generally recommended launching the application using a freemium model to gain early user feedback, noting that converting to a free trial or paid model could be considered later. The board also advised exploring a beta release to manage application ratings, as this could drastically decrease due to potential bugs and broken features. Feature priorities focused on category blocking, internet slowdown, data reporting and recurring blocks, which were seen as the best ideas for further development.

10.9 MVP User Stories

User stories are short, simple descriptions of a feature told from the perspective of the end user. They help bridge the gap between technical requirements and user needs by focusing on who the user is, what they want to do and why they want to do it. Using user stories allows us to better understand the intended functionality and prioritize development based on actual user need. They are easy to communicate, align well with agile development environments and support iterative design.

Each user story typically follows the format:

> As a [type of user], I want to [do something] so that [goal or benefit].

This format ensures that both the purpose and context of each feature is clear. In the following, we outline key user stories that guide the development of our system.

Authentication and Account Management:

- As a new user, I want to sign up with an account, so that I can save my blocks and preferences.
- As a returning user, I want to stay signed in between sessions, so that I can use the application without logging in every time.
- As a user, I want to securely sign out of my account, so that my data stays protected when I stop using the application.

Blocking Functionality:

- As a user, I want to create a block for distracting apps and websites, so that I can stay focused.
- As a user, I want to be redirected to home screen if I try to open a blocked application, so that I am reminded of my active block.

Analytics and Error Reporting:

- As a developer, I want to track when users sign up, start blocks and other interesting events, so that I can understand how the application is being used and improve upon it.
- As a developer, I want to collect error reports, so that bugs can be identified and fixed more efficiently.

Backend Infrastructure:

- As a developer, I want the backend to validate all incoming requests, so that unauthorized access is prevented.
- As a developer, I want the backend to scale with more users, so that performance remains stable over time.

10.10 Sprint Planning

Sprint planning and tracking were managed using Asana [56], a project management tool. Tasks were categorized by domain and visualized using a Kanban board, with states such as *To Do*, *In Progress*, *Blocked* and *Done*. At the end of each sprint, tasks that were not completed were evaluated. They were either moved to the backlog, postponed to a different sprint or resulted in an extension of the current sprint.

Each sprint either contained or concluded with an event (for example board meetings, supervisor sessions, user feedback), which helped shape the direction of the next sprint. The initial sprint plan can be found in appendix 10.4 Initial Project Plan. However, as is common in agile development, the plan evolved significantly over time. This means, that the final planned sprints, skew a lot from the original plan. Below is an explanation for each of the sprints, that gives an inside into how things quickly changes and adapt, in an agile environment.

10.10.0.1 Sprint #1 — Application Prototype

Focused on developing the first working prototype and laying the foundation of the application. It included basic blocking functionality for a single predefined application and website.

10.10.0.2 Sprint #2 — Customization Features

Expanded the prototype to allow users to define blocking rules, like setting which applications and websites to block, and improved the user interface.

10.10.0.3 Sprint #3 — Account Management & MVP Completion

Introduced account management, creating the backend on Azure, analytics integration and error-monitoring integration. A meeting with the Board of Advisors led to a slight pivot in the original release plan. Instead of first creating a closed beta in Sprint #4, the application would be released via Google Play Stores open testing. This was done to follow the principle of *fail early*. We wanted as many users as possible, to gain insight into user behavior and catch all the issues with the application. Open testing allowed broad testing while preventing public ratings from affecting the application store listing. Discussions also resulted in dropping iOS and Windows support at launch, focusing solely on a robust Android release. By building the Android application well, it it would give a better foundation for the other platforms in the future. Rather than simply releasing all three, with minimal features and issues, which would then needs to be fixed on all three.

With finishing the implementation of the account management, this sprint also marked the completion of the MVP. Reaching the MVP was crucial for an effective launch. This therefore marks the milestone, for the first complete foundation of the application for further improvements. Each sprint from here will therefore evaluate new features and improvements, to determine which to implement. When determined, requirements will be discussed and agreed upon.

10.10.0.4 Sprint #4 — Launch (Europe)

Prepared for and executed the European launch. Tasks included bug fixes, better analytics setup and creating a public website, to further create and grow the Axion brand. This sprint also featured the first, primitive implementation of the data reporting feature, which could simply collect the data, but could not show it. Feedback from a meeting with "Center for Ludomani" [35] (the Danish center for gambling addiction) prompted an increase in the need for the feature for category based blocking, extending this sprint to include its implementation. The application gained approximately 10 users through this soft launch, also giving a small insight into user behavior, while also discovering bugs.

10.10.0.5 Sprint #5 — Refinement & User Growth

Focused on improving the application based on initial feedback from the European launch. Further discussed and implemented features such as more blocking options (recurring, delay, etc.), expanded on the data reporting tool in the application with visuals and calculations and further advanced the analytics setup for KPI tracking. A small marketing push at DTU involved distributing flyers with QR codes to onboard more users and speaking with some directly, to gain immediate feedback. The feedback and the ideas behind it, is further explained in section 7.3.3 Introduction & Tutorial.

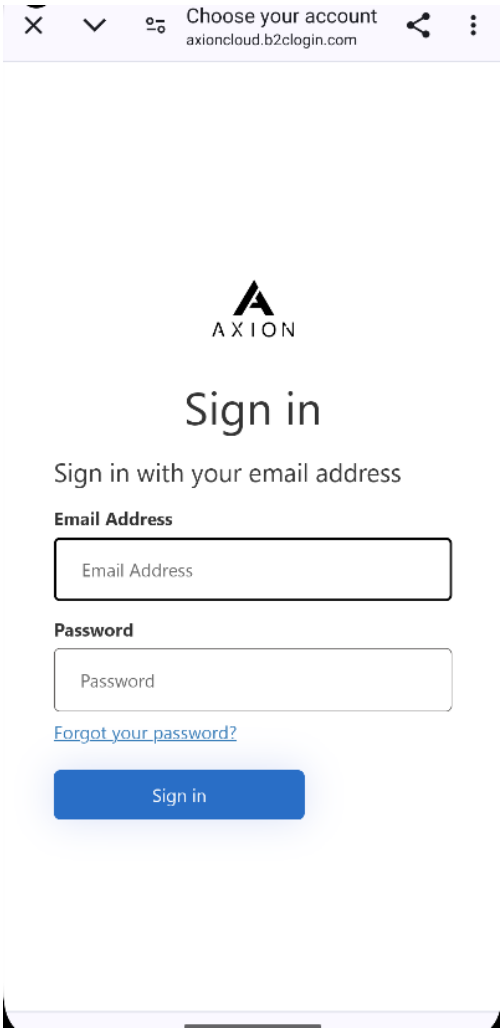
10.10.0.6 Sprint #6 — Global Launch & Finalization

Centered on releasing the application globally and finalizing the thesis. Final bug fixes and documentation were completed.

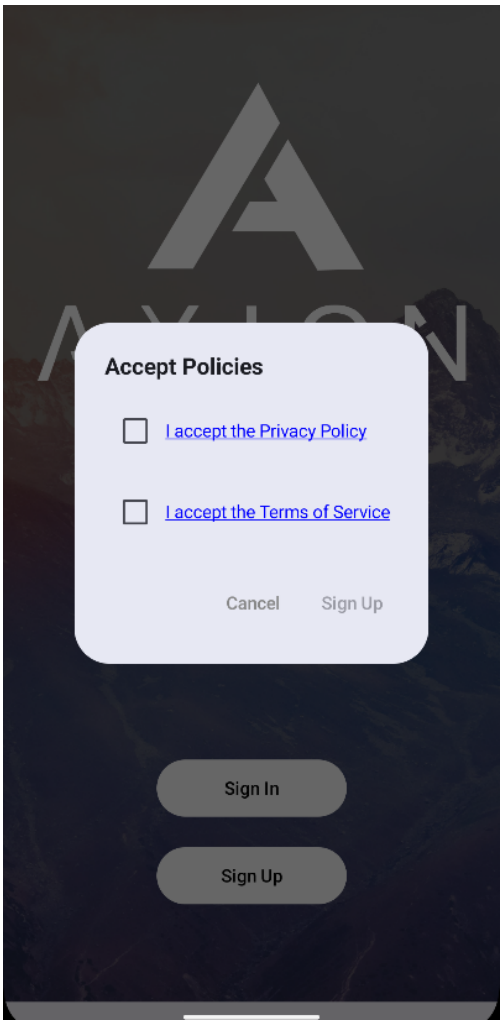
10.11 Screenshots of the Application



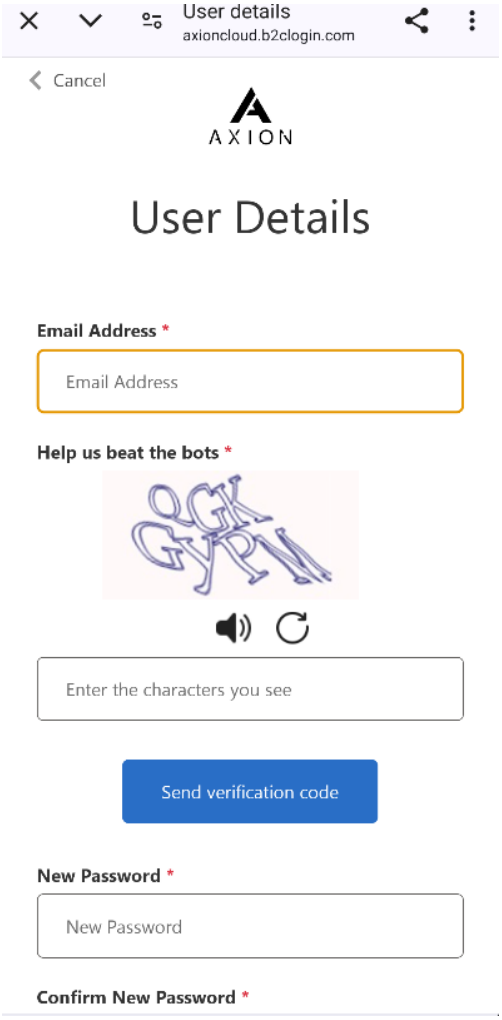
(a) Login Screen



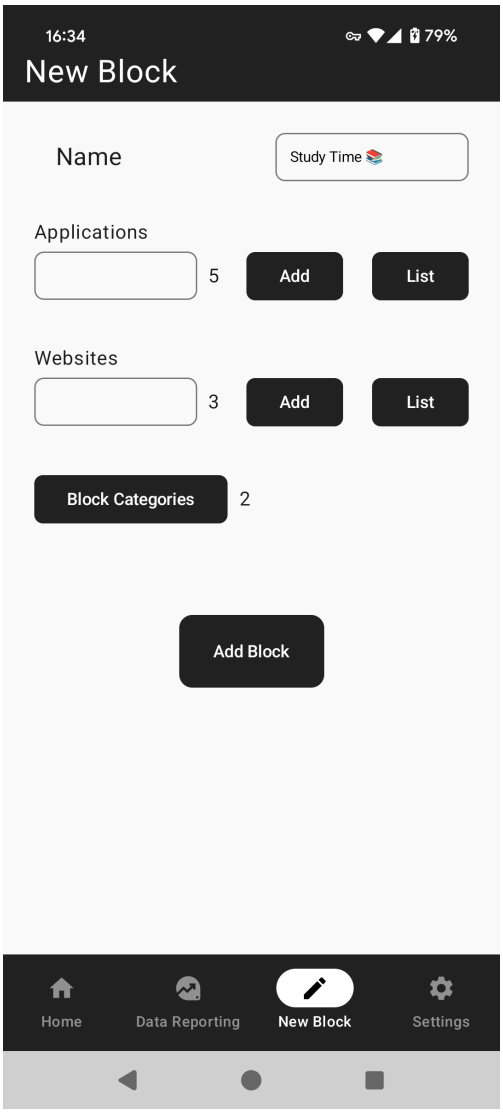
(b) Azure B2C Sign In



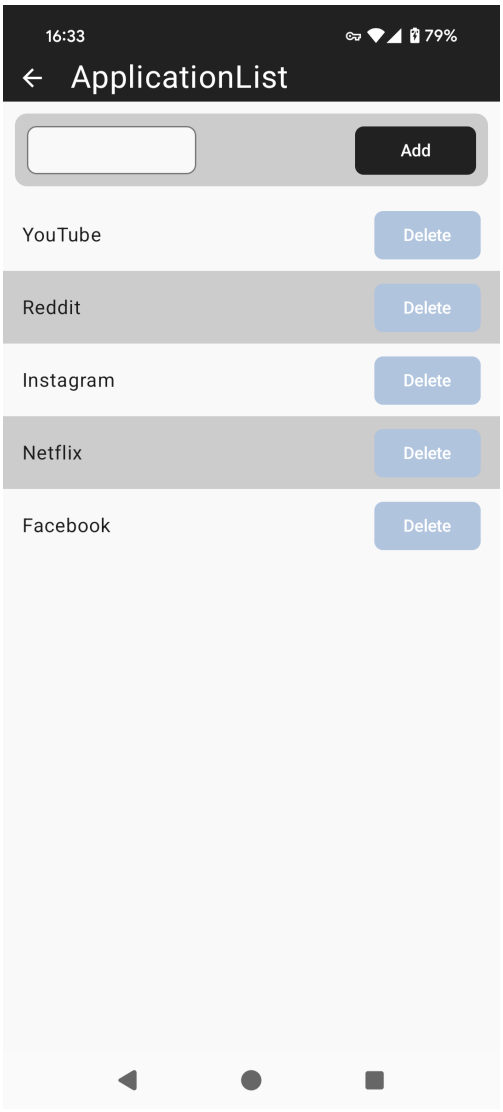
(a) Privacy and ToS



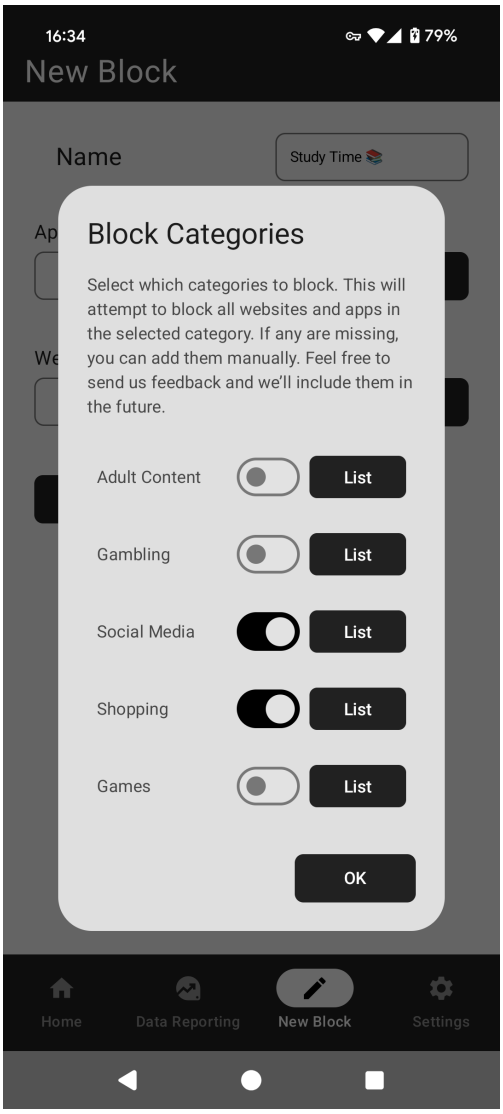
(b) Azure B2C Sign Up



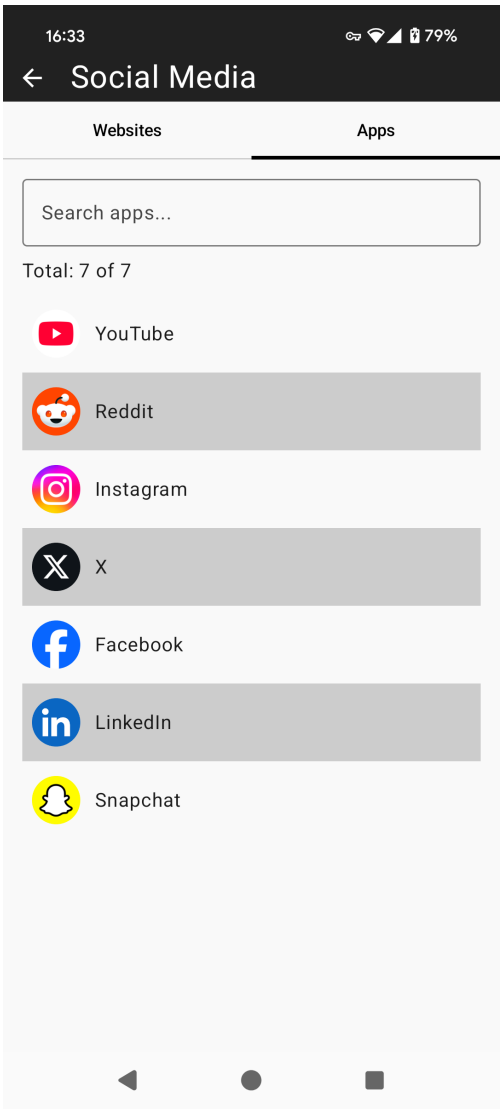
(a) New Block Screen



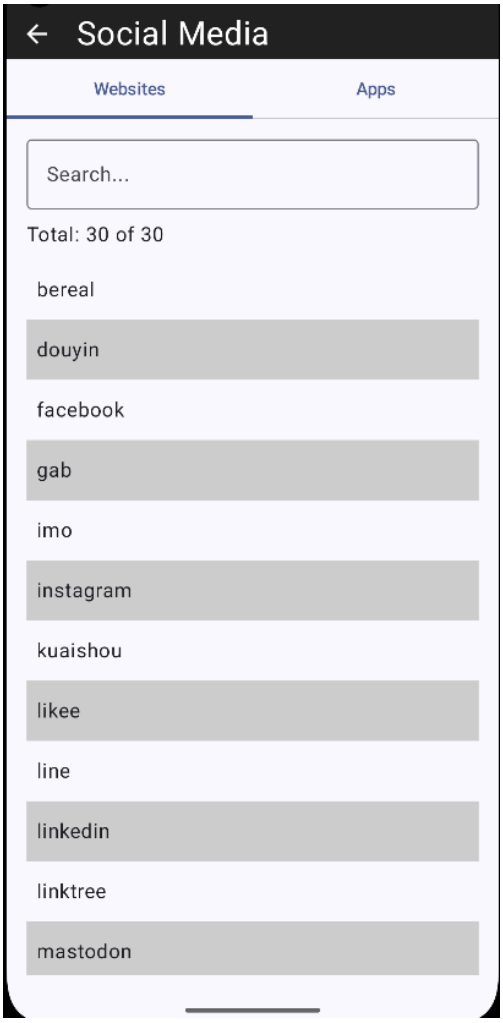
(b) Application List



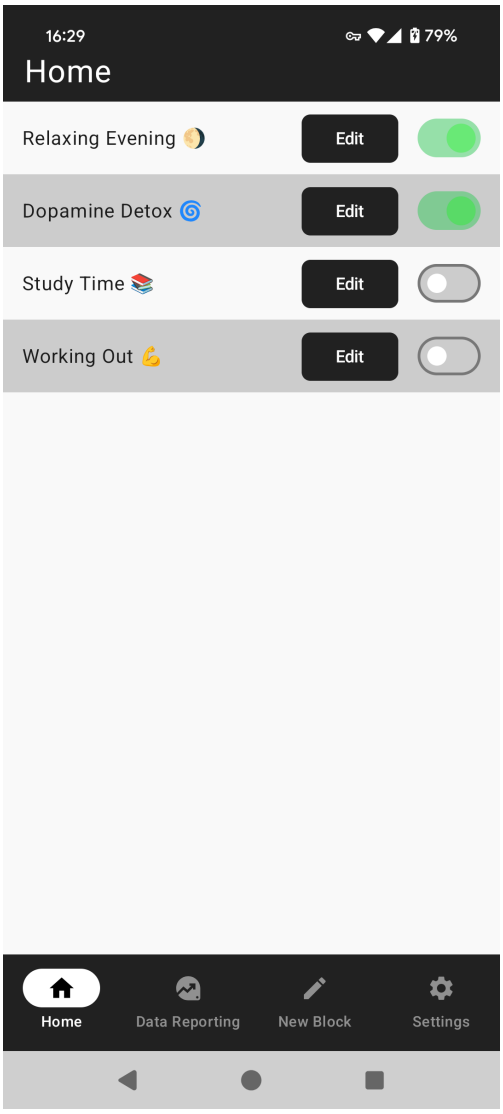
(a) Block Categories



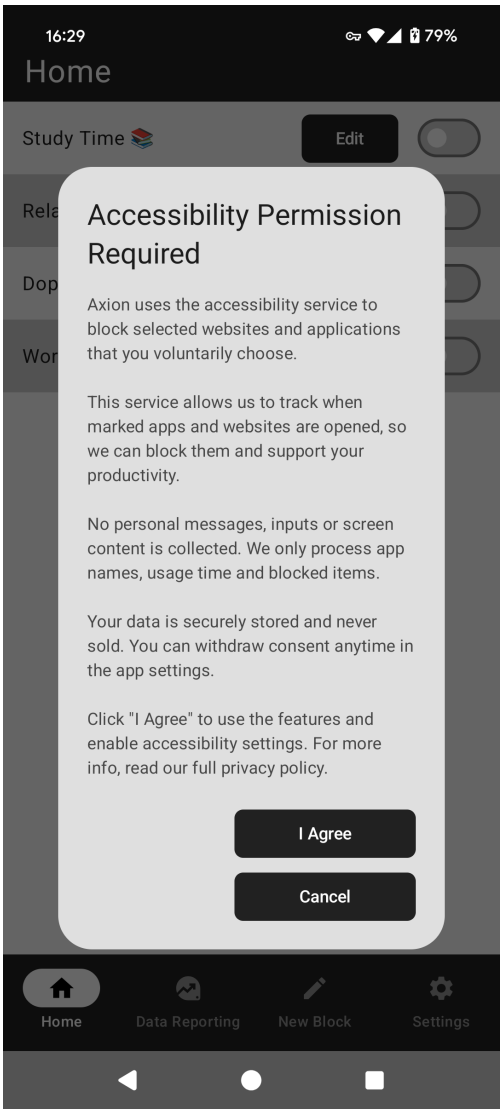
(b) Social Media - Apps



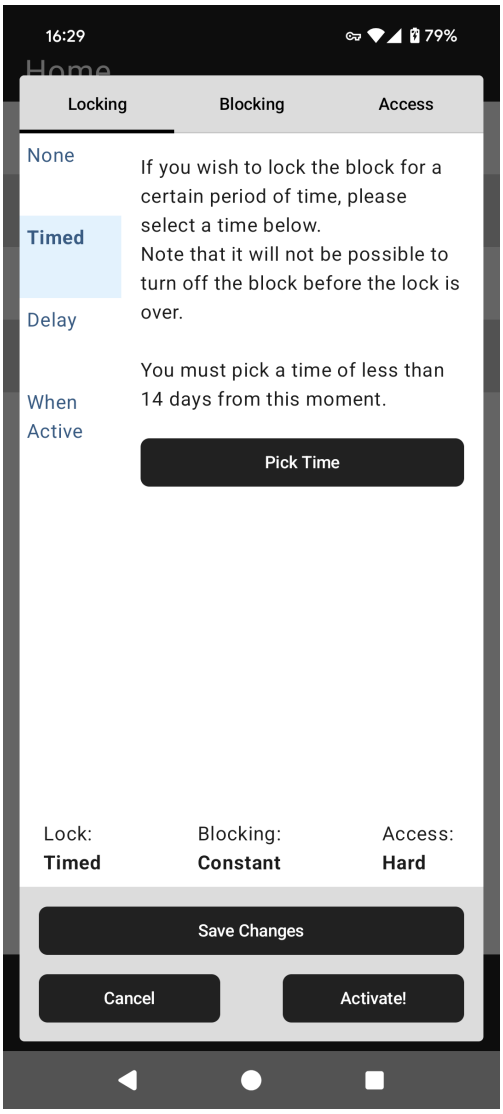
(a) Social Media - Websites



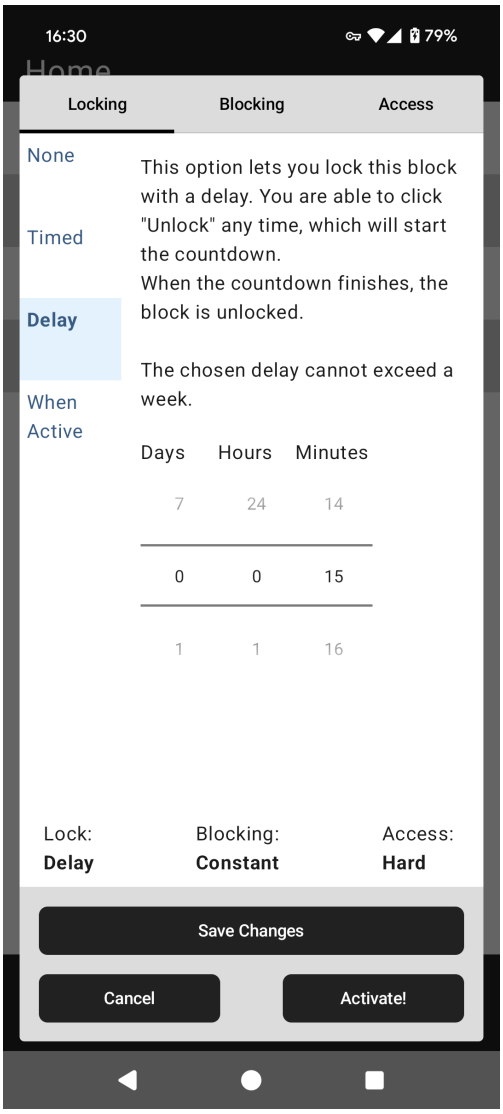
(b) Blocks Overview



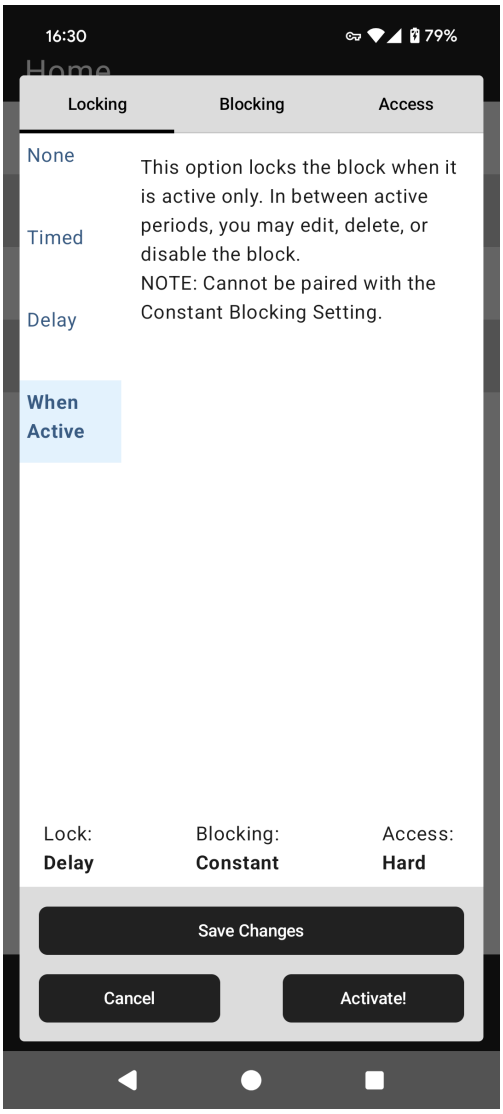
(a) Accessibility Alert



(b) Timed Locking



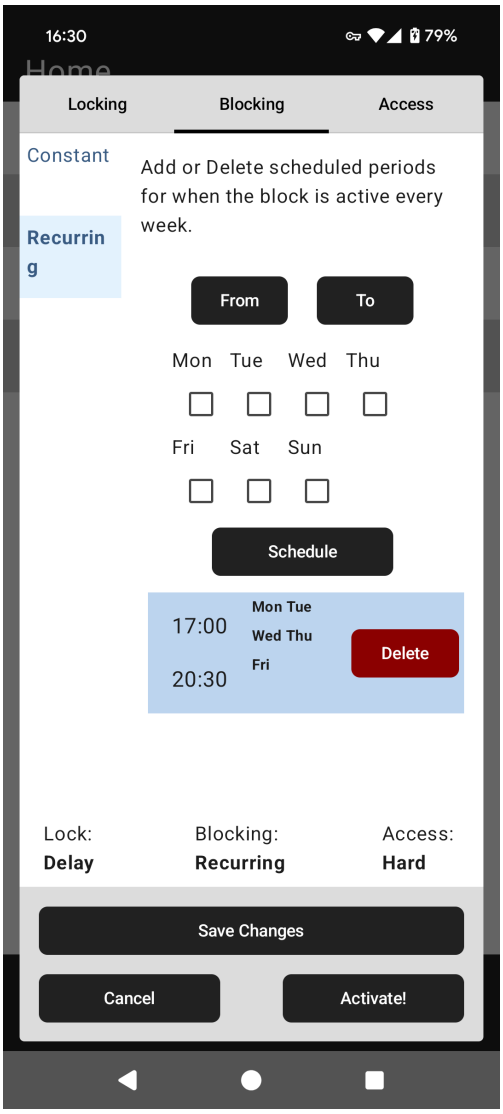
(a) Delayed Locking



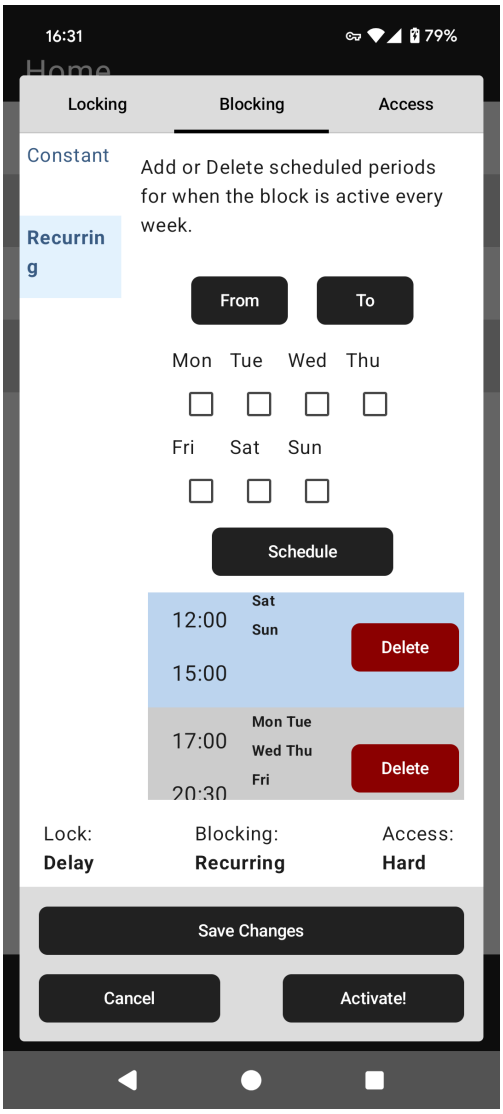
(b) When Active Locking



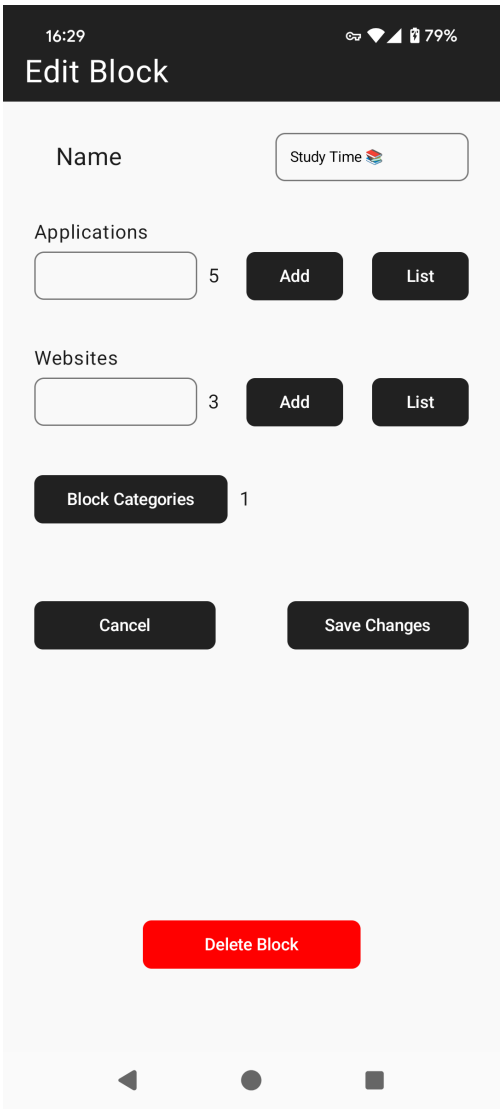
(a) Recurring Method



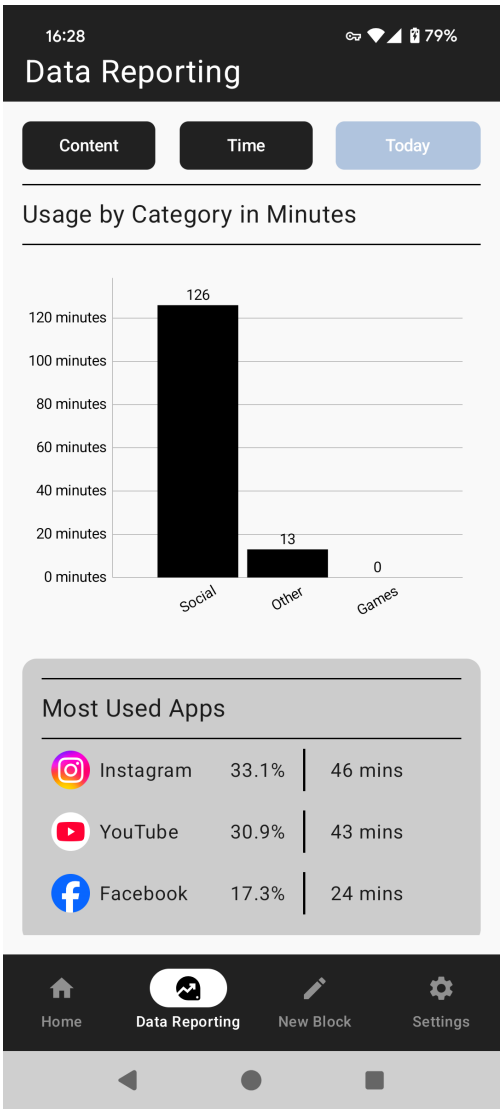
(b) One Recurring Block



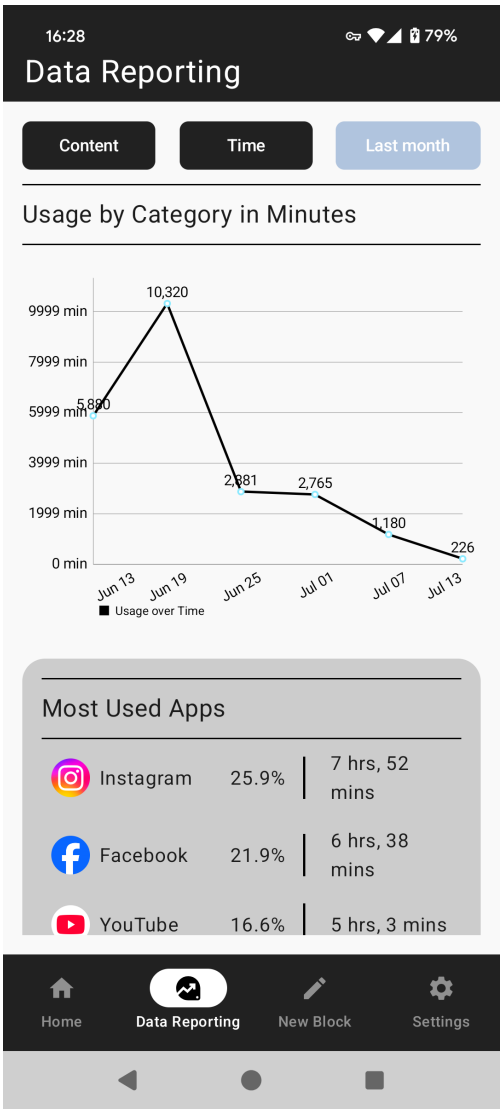
(a) Two Recurring Blocks



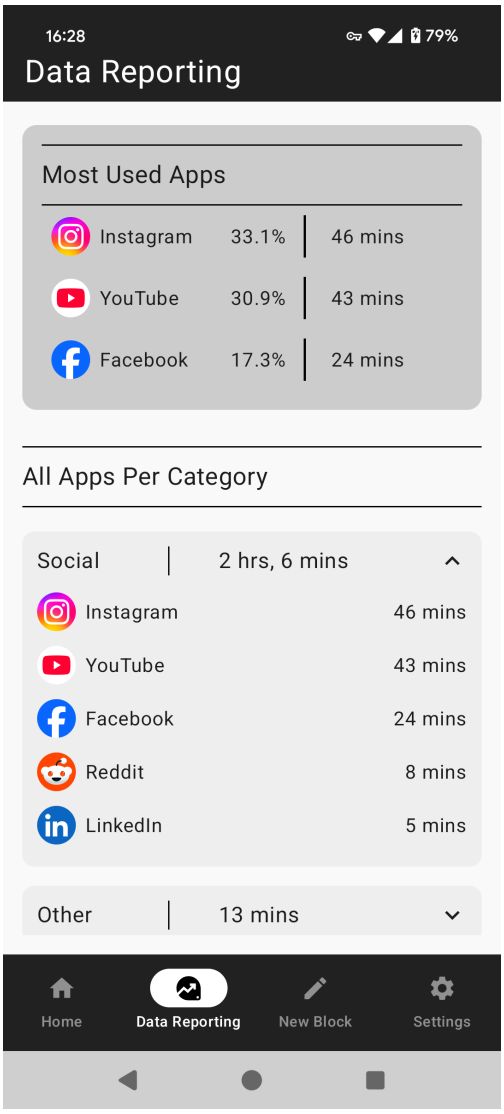
(b) Edit Block Screen



(a) Categories Today



(b) Total Last Month



(a) Most Used & All Apps

10.12 Class Diagram of Block

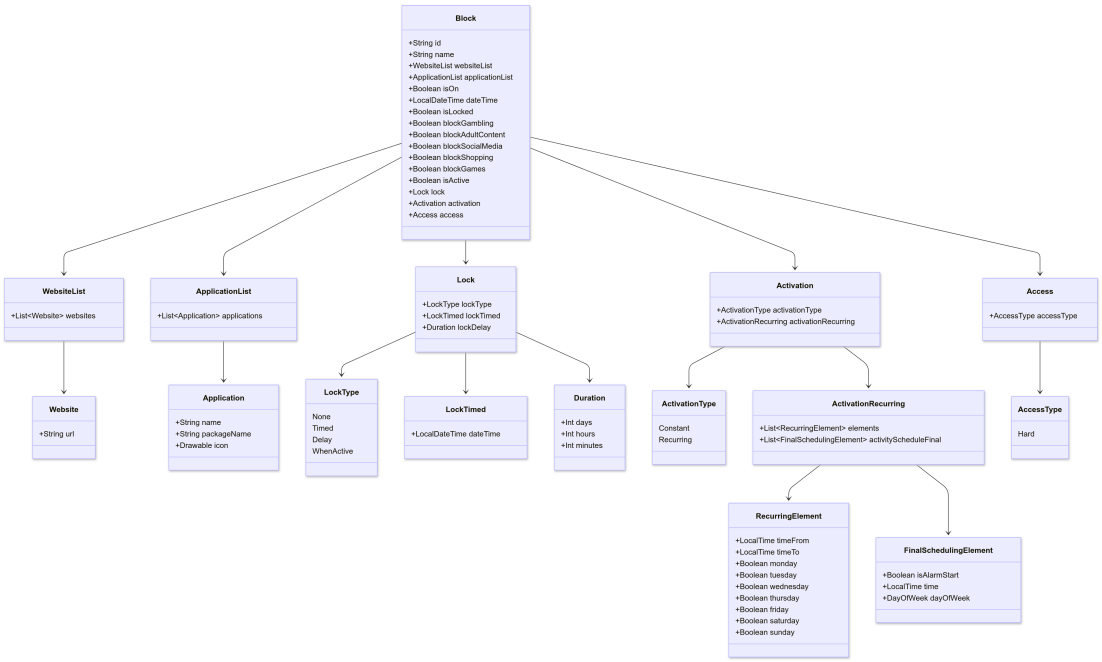


Figure 42: The Fundamental Structure of a Block

10.13 Backend HTTP Statuses

HTTP Status	Meaning
200 (OK)	The request was successful, and data was returned
204 (No Content)	The request was successful, and no data needed to be returned
400 (Bad Request)	The request is malformed in some way (body can be missing required entries)
401 (Unauthorized)	The access token given for the request is not valid
403 (Forbidden)	The access token given for the request is valid, but not allowed to view this resource
404 (Not Found)	The resource requested was not found
409 (Conflict)	The request is in conflict with the current state of the resource
429 (Too Many Requests)	There was sent too many requests to an endpoint in a given time frame
500 (Internal Server Error)	An error occurred when processing the request

Table 9: Currently Used HTTP Statuses

10.14 Accessibility Permission Alert

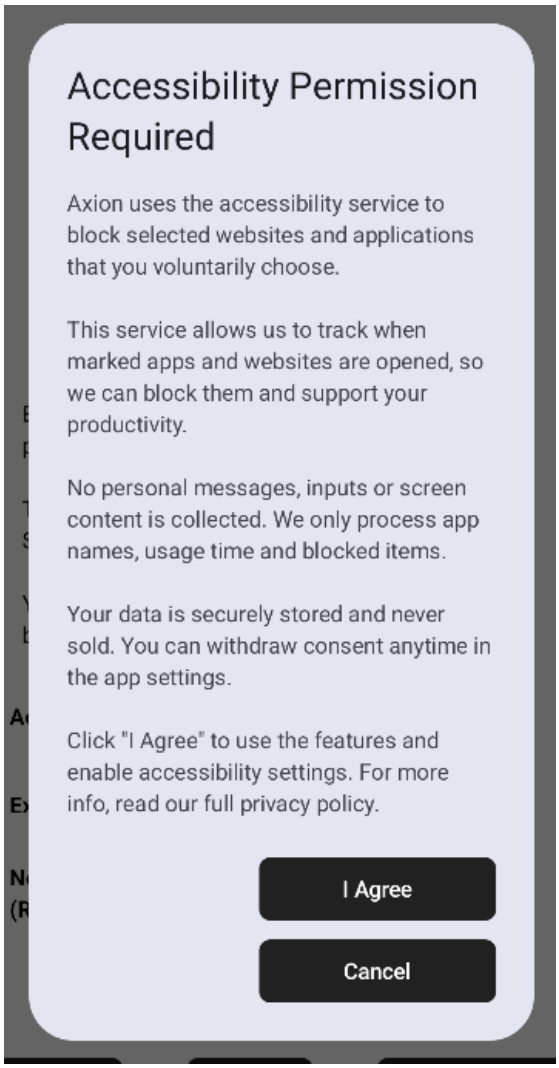


Figure 43: Alert for Accessibility Permission Required

10.15 Navigation Bar Comparison

While it might seem obvious that the application should be structured this way, it is actually a very minimalistic design compared to our competitors. They have entire screens in the navigation bar which has completely different purposes than supporting core functionality. Take a look at the navigation bar from the freedom app, for example, shown on figure 44.



Figure 44: Freedom's Navigation Bar

The today tab works a little like our home-screen, but with a lot more confusing features, for example a blog with hundres of articles with links to the web. It also shows blocks which happened in the past, which we believe is of little interest to the users. The "Start Session" tab is like our "New Block tab, but has one major difference. Freedom does not have a "block" as a data class, like we do, which then contains content and blocking options. Instead, they have "block session" which contains "block lists" and blocking options. The user has to first create a block session, then create any number of block lists which each contains lists of websites and applications. Then, they have to select any number of those block lists and attach them to the block session. Thus, there are extra steps in creating a block in freedom - it is very confusing, and we tried letting people we knew create a simple 1 hour block of YouTube on freedom. Several of them could not figure out how to do on their own, suggesting an unnecessarily overcomplicated data structure.

The next tab is block lists - this tab creates the 2nd element of freedoms blocks. Unneccesary tab for our application, as we just have one data class: the "block" class. On their 4th navigation tab, they made the choice of have a library of what they call "focus sounds". It is a collection of recordings from woods, cafes, or other ambient settings. While some might like this feature, dedicating 20 % of the applications navigation space in a mobile UI seems like it compromises the core functionality and simplicity for the user, and thus like an illogical design choice. Their final tab is settings, just like ours. Freedom chose to prioritize giving the users a sound library, over for example a screen time rapport.

Opals navigation bar is shown on figure 45.

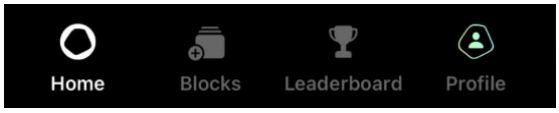


Figure 45: Opal's Navigation Bar

Opal's Home screen contains a acreen time rapport - it is not very detailed, does not show categories, on does not aid the user significantly in clearly marking which applications generally takes up the most of their time. It does have a good-looking design. Their blocks screen handles both the creation and management of blocks. While it does make navigation a little more difficult, it means that creating and managing blocks will only take up one slot in the navigation bar instead of two. The third tab is "Leaderboard", which provides the user with the option of gaining friends and competing in reducing screen time. It is clearly an attempt to increase traditional engagement metrics with the application and benefit from some kind of virality gained by integrating a little social network. Again, the choice of spending 25 % of the navigation space on features which have nothing to do with the core features is certainly a

compromise. We do not know how much they benefit from the engineered virality of the social network, but it seems like a detour from the functionality and simplicity of the application. The profile tab is much like our settings tab.

Hence, compared to the industry, our application has a simplistic, minimalist and easily usable design. Prioritizing less features, a simple data structure and no non-core features lets the tool be efficient and minimalistic.

10.16 Supplementary Files

Supplementary files uploaded for the thesis, is the following:

- `axion-android-application.zip` - The Android Application Codebase
- `axion-cloud.zip` - The Backend Codebase
- `User Growth & Revenue Projection.xlsx` - The financial projection for user growth and revenue
- `Pitch Deck.pptx` - The newest version of the pitch-deck
- `app-showcase.mp4` - The showcase of the application